

小さな VLA を自作して学ぶ

PyTorch で作る Vision-Language-Action —— 図表と実装で学ぶ版

2026 ・ github.com/sige0002/vla-learn

目次

1	全体像とセットアップ	5
1.1	VLA とは何か（一文で）	5
1.2	ロボット模倣学習 (imitation learning) の流れ	6
1.3	タスクの仕様	6
1.4	環境の入口を読む	8
1.5	環境構築 (uv)	8
1.6	本書の歩き方とリポジトリ地図	9
1.7	学習マップ (M0 → M6)	10
1.8	用語の整理（最重要）	10
2	PyTorch 速習	12
2.1	テンソル (tensor) —— PyTorch の基本データ	12
2.2	ブロードキャスト (broadcasting)	14
2.3	自動微分 (autograd)	15
2.4	nn.Module / nn.Linear / nn.Sequential	16
2.5	学習ループ（損失と optimizer）	17
2.6	「1 バッチに過学習できるか」 —— 学習デバグの鉄則	18
2.7	Dataset と DataLoader	19
3	最小の模倣学習 (behavior cloning)	22
3.1	この章で作る「最小の VLA の祖先」	22
3.2	エキスパート（お手本）とは	22
3.3	デモを集める: generate_episodes	23
3.4	最小の模倣学習（その 1）: state[3] → action[3]	24
3.5	【山場】なぜ素朴な模倣は閉ループで崩れるのか	25
3.6	【対策】ノイズ注入で「リカバリーのお手本」を作る（DART / DAgger 風）	27
3.7	学習デバグの鉄則: 1 バッチに過学習できるか	30
4	行動表現とデータ	32
4.1	なぜ「行動表現とデータ」を独立した章にするのか	32
4.2	正規化 (normalization)	32
4.3	時間方向の窓と行動チャンク (action chunking)	34
4.4	言語のトークン化 (CharTokenizer)	35
4.5	すべてを束ねる SyntheticVLADataset	36
4.6	仕上げ: pad_mask を使った損失 masked_mse	38
4.7	LeRobot 風データ辞書への伏線	39
5	最小 VLA を自作する (MSE 回帰版 TinyVLA)	41
5.1	全体像 — VLABackbone (3 エンコーダ + 融合)	41
5.2	3 つの設計教訓 — なぜこの作りなのか	43
5.3	TinyVLA 本体 — head と forward	45
5.4	学習ループ — masked_mse で教師あり回帰	46
5.5	スクリプトで学習する — train_mse.py + configs/m4_mse.json	49
5.6	閉ループ評価 — 損失ではなく成功率を見る	49
5.7	「損失は下がるのに動かすと失敗」 — distribution shift の再来	51
5.8	章末チェック — 1 バッチに過学習できるか	51

6	flow matching 化 (MSE ヘッド → rectified flow ヘッド)	53
6.1	この章で「足す」のは 1 点だけ	53
6.2	なぜ MSE では不十分か (多峰性を平均で潰す)	54
6.3	rectified flow の実装対応 (座学 → コード)	55
6.4	時刻埋め込みと速度ネット (shape を完全に追う)	57
6.5	推論: $t=0 \rightarrow 1$ を Euler 積分してサンプルする	58
6.6	学習と評価 (M4 との差分は本当に 2 箇所だけ)	59
6.7	本物の VLA とのつながり (次章 M6 へ)	61
6.8	章末: 1 バッチ過学習 で健全性を確かめる	61
7	LeRobot と有名 VLA	63
7.1	LeRobotDataset とは何か (フレーム列という考え方)	63
7.2	変換ロジックを読む (<code>map_episode_to_frames</code> ・ lerobot 不要)	65
7.3	SmolVLA 精読 — 自作 VLA との対応表	66
7.4	π_0 / $\pi_{0.5}$ — flow matching と action expert (実機データ規模)	68
7.5	OpenVLA — 離散トークン自己回帰 + 大規模 VLM	69
7.6	GR00T / MolmoAct2 — dual-system と action expert (LeRobot 統合)	70
7.7	比較表 — 行動表現で並べる	72
7.8	卒業課題 (capstone challenge)	73

本書は、小さな VLA（画像＋言語→行動）を PyTorch でスクラッチ自作するハンズオンの「図表＋実装を読む」版です。題材は実機・物理エンジン不要の **Tiny Tabletop 2D Pick-and-Place** で、ノート PC の CPU だけで最後まで動きます。各章の「実装を読む」枠は、次に読むべき ソースファイル（自作 `src/vla_learn/` と、有名 VLA の公式 repo）への地図です。対応する実行可能コード・演習・解答はリポジトリにあります。

全体像とセットアップ

この章のゴール

- VLA (Vision-Language-Action) が「何を入力に、何を出力するもの」かを一言で言えるようになる。
- ロボット模倣学習 (imitation learning) の流れと、本書で繰り返し出てくる用語 (action / state / episode / 行動チャンク / policy / rollout) を理解する。
- 本書の完成物 **Tiny Tabletop 2D Pick-and-Place** が何をするタスクか分かる。
- 開発環境 (uv) を用意し、テストと小さな学習が手元で回ることを確認する。
- M0 → M6 の地図を持って、次に何を学ぶか分かる。

本書は「まず動かす → 理解する」方針です。数式から入るのではなく、小さな VLA を **自分の手で** 作って学習・評価し、最後に SmolVLA / $\pi 0$ など本物の VLA を「同じ部品の大規模版」として読みます。PyTorch は知らなくても大丈夫です（次章 M1 でやさしく入門します）。diffusion / VLM の座学（概念・数式）は既習前提ですが、M0 では使いません。

1.1 VLA とは何か（一文で）

VLA とは「画像 (Vision) と言語指示 (Language) を入力に取り、ロボットの行動 (Action) を出力するモデル」です。画像分類は「画像 → ラベル」、VLM は「画像 + テキスト → テキスト」でした。VLA はその出力を **テキストではなく「行動」** に変えたものの、と捉えると分かりやすいです。「ロボットの目と耳と手をつなぐモデル」だと思ってください。

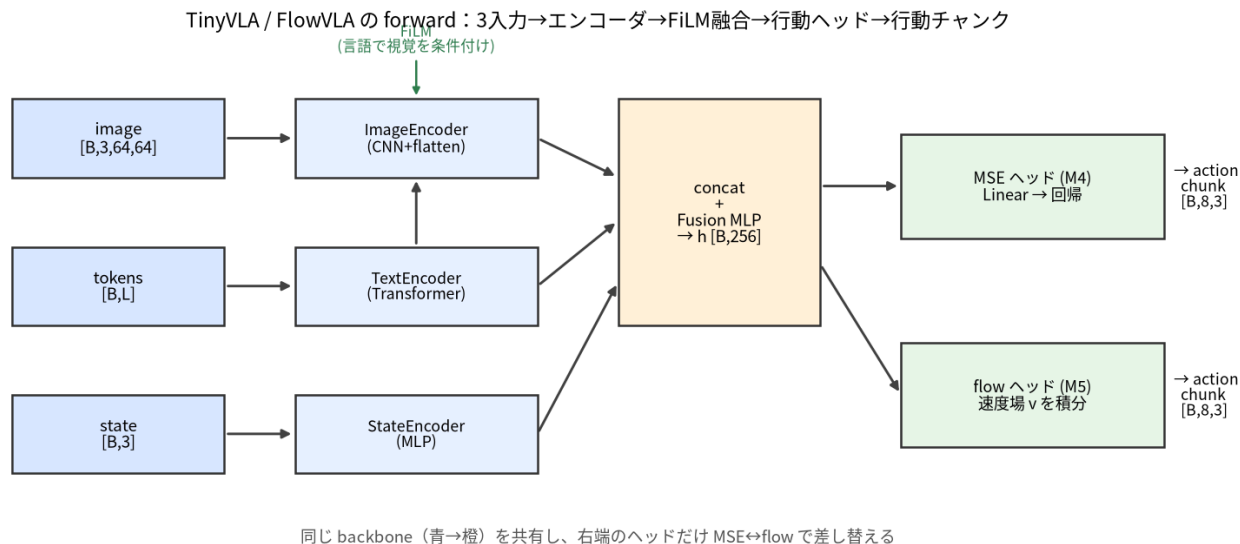


図 1: VLA の骨格。画像・言語・状態の 3 入力をそれぞれ エンコードし、言語で視覚を条件付け (FiLM) してから融合し、行動ヘッドで **行動チャンク** を出す。本書ではこの図の全部品を自作する。完成形の実装は `src/vla_learn/models/tiny_vla.py` の `VLABackbone`。

座学とのつながり

既習の VLM（画像とテキストを融合して表現を作る部分）は、VLA の「入力を理解する側」にほぼそのまま流用できます。VLA で新しく要るのは、その表現から **連続値の行動を生成するヘッド** です。本書では後半 M5 で、座学の flow matching / 拡散をこの「行動ヘッド」として実装回収します。

1.2 ロボット模倣学習 (imitation learning) の流れ

本書は **模倣学習 (imitation learning)**、特に **行動クローニング (behavior cloning)** という最も基本的な方法を使います。強化学習のように「試行錯誤して報酬を最大化」するのではなく、「**お手本 (expert) の行動をそっくり真似する**」教師あり学習です。流れは4ステップです。

ステップ	本書での自作内容
1. お手本を集める	ルールベースの 解析エキスパート (expert_action) がタスクを 100% 成功させ、その軌跡を記録する。ニューラルネットは使わない。世界の状態を直接読めるので毎回成功する。
2. データセットにする	記録した軌跡を (画像, 状態, 言語, 行動チャンク) の組に整形する (M3)。
3. 方策を学習する	VLA に「観測 → お手本の行動」を回帰 (MSE) で真似させる (M4)。さらに flow matching 版 (M5) に発展させる。
4. 評価する	学習した方策を環境で実際に動かし (ロールアウト rollout)、成功率を測る。

実装を読む `src/vla_learn/envs/expert.py` → `expert_action`

ステップ1の「お手本」を作る心臓部。 `if not holding:` ブロックへ近づいて掴む / `else:` ゴールへと運んで置く

いう単純な状態機械です。20行ほどなので、最初に読むと「学習が何を真似するのか」が具体的につかめます。NNを一切使わず **ワールド状態を直接読む** から 100% 成功する、という点が「お手本は賢くてよい (カンニングしてよい)」という模倣学習の発想そのものです。

座学とのつながり

行動チャンク (action chunking) は VLA 頻出の重要概念なので直感だけ掴んでおきましょう。1手ずつ予測すると、予測の小さなブレが毎ステップ積み重なって軌道が崩れがちです。そこで「今の観測から未来8手をまとめて予測 → そのうち数手だけ実行 → また観測して予測」とすると軌道が安定します。SmolVLA や $\pi 0$ など本物の VLA でも標準的に使われます (M3 で実装します)。

1.3 タスクの仕様

最終的にあなたが自作・学習・評価できるようになるのは、次の小さな VLA です。

- **タスク:** 2D 平面 (テーブルを真上から見た図) で、**言語指示** に従い、**指定色のブロックを指定色のゴールへ運ぶ** Pick-and-Place (つかんで置く)。
- **指示は日本語。** 例: 「青のブロックを青のゴールに置いて」「赤ブロックをつかんで黄ゴールへ」。
- **観測** は画像・状態・言語指示の3つ。**行動** は $[d_x, d_y, \text{grip}]$ の3次元で、一度に未来8ステップ分 (行動チャンク) を予測します。

要素	形	意味
image	$[3, 64, 64]$	真上から見たテーブルの RGB 画像 (値域 0..1)
state	$[3]$	グリッパの (a_x, a_y) 位置と開閉 grip (固有受容感覚 proprioception)
instruction	str	日本語の指示文 (例「青のブロックを青のゴールに置いて」)
action	$[8, 3]$	未来8手の (d_x, d_y, grip) 。 d_x, d_y は各軸 ± 0.08 にクリップ、grip は 0.0=開 / 1.0=閉 (≥ 0.5 で閉)

Tiny Tabletop 2D：観測画像 [3,64,64] と言語指示

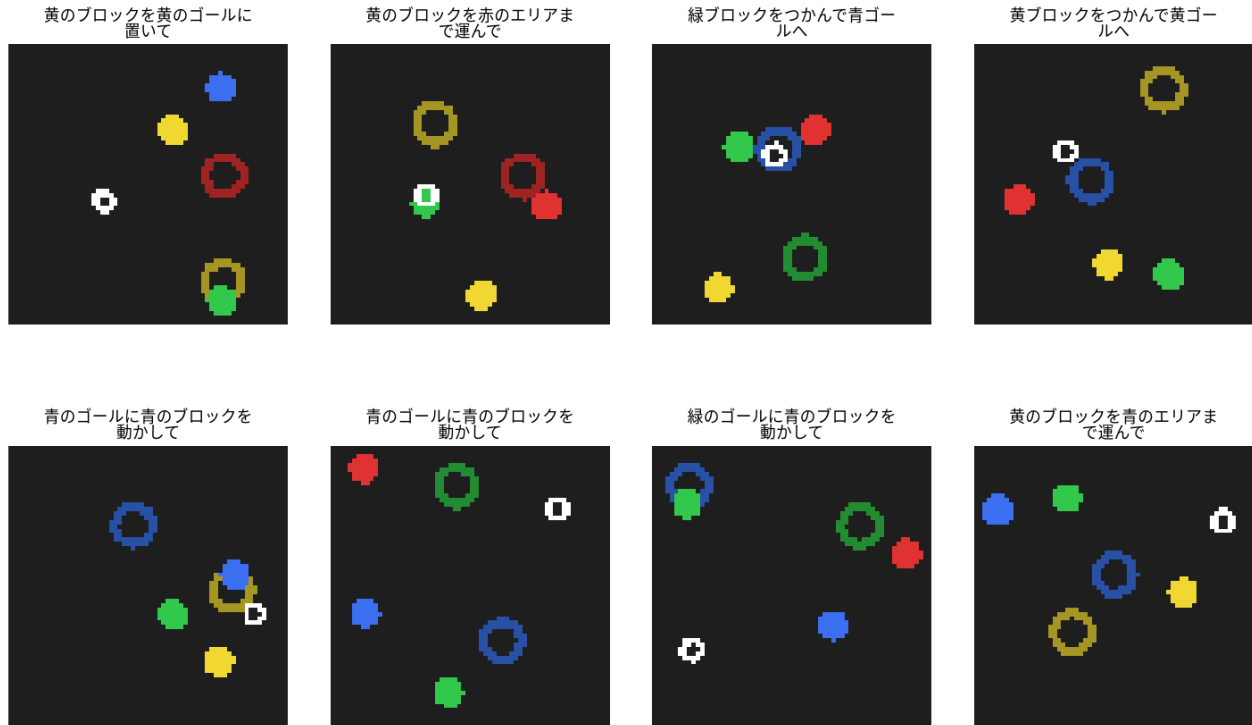


図 2: Tiny Tabletop 2D の観測例グリッド。色つきブロック（塗り円）を指定色のゴール（リング）へ運ぶ。白がグリッパ（開=リング / 閉=塗り円）。実装は `src/vla_learn/envs/render.py` の `render_world` と `src/vla_learn/envs/tabletop2d.py`。

これらの形・単位は `src/vla_learn/constants.py` に一元化されています（`IMG_SIZE=64` , `ACTION_DIM=3` , `STATE_DIM=3` , `MAX_STEP=0.08` , `SUCCESS_RADIUS=0.12` , `DEFAULT_CHUNK_LEN=8`）。環境・データ・モデル・学習・評価のあいだで shape がズレないための「唯一の真実の源」です。

つまずき / バグの定番

この環境の「成功」は、対象ブロックが対象ゴールの半径 (`SUCCESS_RADIUS=0.12`) 内に入った時点で判定します（`Tabletop2DEnv.is_success` は距離だけを見ます）。掴んだまま運べば成功で、実機タスクのように「グリッパを開いて離す」ことまでは要求しません。物理エンジンも無く当たり判定は距離計算だけ——本物の VLA の骨格は保ちつつ、CPU で完結するよう思い切って単純化しています。

このタスクの良いところは、**実機もシミュレータ（物理エンジン）も要らず、NumPy だけで世界が完結すること**です。画像はルールに従って描画され、当たり判定も単純な距離計算なので、手元の CPU だけで本物の VLA と同じ骨格（画像・言語・状態を融合 → 行動チャンクを生成）を最後まで体験できます。

版	モデル名	行動ヘッド	学ぶ章
MSE（回帰）版	TinyVLA（約 0.42M）	全結合で行動を直接回帰、損失は MSE	M4
flow matching 版	FlowVLA（約 0.58M）	rectified flow で行動を生成（座学の実装回収）	M5

補足

数値の注意: 成功率やパラメータ数は乱数・環境差でぶれます。「学習で loss が下がり、成功率が上がる」という傾向を見てください。目安として解析エキスパート約 100%、TinyVLA (MSE) の成功率

は約 0.76、FlowVLA は約 1.0、パラメータ数は TinyVLA 約 0.42M・FlowVLA 約 0.58M です (CPU で数分学習可能)。皆さんの環境で多少前後します。

1.4 環境の入口を読む

まずこの 1 ファイルを読むと、観測・行動・成功の定義が一望できます。

実装を読む `src/vla_learn/envs/tabletop2d.py` → `Tabletop2DEnv.step`

環境の `reset` / `step` はここ。`step(action)` がグリッパを動かし (`dx`, `dy` を `MAX_STEP` にクリップ)、掴み判定 (最近傍の物体が `GRASP_RADIUS` 以内ならグリッパを閉じて追従) と成功判定 (`_is_success`) を行い、(`obs`, `reward`, `done`, `info`) を返します。Gym 風の最小 API です。

```
from vla_learn.envs import Tabletop2DEnv

env = Tabletop2DEnv(seed=0)
obs = env.reset()
print(obs["instruction"])      # 例: 青のブロックを青のゴールに置いて
print(obs["image"].shape)     # (3, 64, 64)
print(obs["state"])           # [ax, ay, gripper] float32

# お手本(エキスパート)で 1 手進めてみる
from vla_learn.envs import expert_action
action = expert_action(obs["world"])      # [dx, dy, grip]
obs, reward, done, info = env.step(action)
print(action, reward, done, info["success"])
```

つまづき / バグの定番

テンソルとモデルは同じ `device` に置く、画像は `[B, 3, 64, 64]`・行動チャUNKは `[B, 8, 3]` —— この「形」と「device」を最初に押さえると以降がぐっと楽になります (M1 で練習します)。

1.5 環境構築 (uv)

補足

重要: この教材は **CPU だけ・実機なし・物理エンジンなし** で完結します。GPU は不要です。

パッケージと仮想環境の管理に `uv` を使います。`uv` は「速い・1 ツールで完結・`uv.lock` で全員が同じバージョンを再現できる」のが利点で、`pip` と `venv` を別々に覚えなくても `uv sync` 一発で環境がそろいます。

```
# 0) uv を入れる (未導入なら)。入っていればスキップ。
# macOS / Linux:
curl -LsSf https://astral.sh/uv/install.sh | sh
# Windows (PowerShell):
# powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# 1) リポジトリのルートに移動
cd vla_learn

# 2) 依存を同期する。これ 1 つで
# 「.venv の作成 → PyTorch(CPU版) の導入 → vla_learn の editable 導入 → pytest 導入」
```

```
# まで全部やってくれる。
uv sync

# 任意) 可視化(matplotlib) も使いたい場合は extra を足す
uv sync --extra viz
```

補足

uv sync は何をしている? カレントの `pyproject.toml` と `uv.lock` を読み、ロックされたバージョンで `.venv` を作り直します。本パッケージ `vla_learn` は **editable** で導入されるので、どこからでも `import vla_learn` が通り、`src/` を書き換えればすぐ反映されます。PyTorchは**CPU専用index**から入る設定(`pyproject.toml` の `[tool.uv.sources]`)なので、GPU/CUDAは不要・余計な手動 `pip install` も不要です。**uv run** は `uv run python ...` のように使うと、必要なら環境を自動同期してから実行します (`.venv` を手で `activate` しなくてよい)。

1.5.1 動作確認 1: テストが通るか (数十秒)

```
uv run pytest -q
```

テストには「環境とエキスパート」「データの shape」「正規化」「モデルの forward」「**1 バッチに 過学習できるか**」が含まれます。最後の項目は学習機構が健全かを確かめる鉄則のテストで、本書で繰り返し登場します (M1 で考え方を導入します)。全部 `passed` と出れば環境は OK です。

実装を読む `tests/test_overfit_tiny_batch.py` → `test_mse_overfits_one_batch`

「1 バッチ過学習」テストの実体。TinyVLA を 16 サンプルの 1 バッチで 200 ステップ学習し、**loss が最初の 20% 未満まで下がること**を `assert` します。これが本書の「健全性テスト」で、各章の演習にも必ず 1 問入ります。中身は M1 で全部読み解きます。

1.5.2 動作確認 2: 小さく学習が回るか (1 分程度)

```
uv run python scripts/train_mse.py --config configs/smoke.json
```

`configs/smoke.json` は **ごく小規模な設定** (60 エピソード・3 epoch) で、「学習ループが最後まで回るか」だけを確認するためのものです (成功率を競う設定ではありません)。エラーなく学習が進み、最後にチェックポイントが `checkpoints/smoke/` に保存されれば、ひとつおりの配線 (データ生成 → 学習 → 保存) が動いています。本番の学習・評価 (`configs/m4_mse.json` など) は M4・M5 で扱います。

つまずき / バグの定番

困ったとき

- `ModuleNotFoundError: No module named 'vla_learn'` → `uv sync` を実行したか、コマンドを `uv run` 経由で動かしているか確認。
- `ModuleNotFoundError: No module named 'torch'` → `uv sync` 未実行か、`uv run` を付け忘れ。
- `command not found: uv` → `uv` 自体が未導入。手順 0 のインストールコマンドを実行。

1.6 本書の歩き方とリポジトリ地図

本文・演習・実装は次のように分かれています。

ディレクトリ	中身
<code>lessons/</code>	Markdown の短縮版教材 (M0~M6)。閲覧用。enriched 版はこの Typst 本

book/	この Typst 教科書 (chapters/ ・ figures/ ・ lib/template.typ)
exercises/	章ごとの演習 (mX/README.md と雛形 .py)
solutions/	演習の解答と解説
src/vla_learn/	検証済みの実装 (envs / datasets / models / training / evaluation)
scripts/	make_dataset / train_mse / train_flow / eval_policy など
configs/	学習設定 (m4_mse.json , m5_flow.json , smoke.json)
tests/	pytest (shape ・ 正規化 ・ forward ・ 1 バッチ過学習)

各章は「本文を読む → 演習 (exercises/mX/) を解く → 解答 (solutions/mX/) で答え合わせ」の順で進めます。演習はどの章も「**shape 確認** → **穴埋め** → **バグ修正** → **小実装** → **実験**」の 5 型で構成され、毎章「1 バッチに過学習できるか」を確認します。本書では各章に **実装を読む** 枠 (緑) を置き、「次に開くべき src/ のファイルと関数」を必ず指します。図も「飾り」ではなく、対応する実装 ファイルへの地図として読んでください (各図キャプションに src パスを書いています)。

1.7 学習マップ (M0 → M6)

章	テーマ	主に学ぶこと
M0 (この章)	全体像とセットアップ	VLA とは / action ・ state ・ episode ・ chunk / 完成物デモ / 環境構築
M1	PyTorch 速習	tensor ・ autograd ・ nn.Module ・ 学習ループ ・ Dataset/ DataLoader
M2	最小の模倣学習	状態 → 行動 ・ 画像 → 行動の回帰、なぜ素朴な模倣は崩れるか
M3	行動表現とデータ	正規化 ・ 時間窓 ・ 行動チャンク ・ LeRobot 風データ辞書
M4	最小 VLA をスクラッチ	画像 + 言語 + 状態 → 行動チャンク (MSE 版) を自作して学習 ・ 評価
M5	flow matching 化	行動ヘッドを生成モデル (rectified flow) へ。座学の実装回収
M6	LeRobot と有名 VLA	自作データの LeRobot export / SmolVLA 精読 + $\pi 0$ ・ GR00T ・ OpenVLA ・ MolmoAct 概観 / 卒業課題

1.8 用語の整理 (最重要)

本書で繰り返し出てくる言葉です。ここで一度そろえておきます。

用語	英語	本書での意味
行動	action	エージェントが 1 ステップで取る出力。ここでは $[d_x, d_y, \text{grip}]$ の 3 次元
状態	state	エージェント自身が感じ取れる内部状態 (固有受容感覚 proprioception)。ここでは $[a_x, a_y, \text{gripper}]$ の 3 次元
観測	observation	モデルへの入力一式。ここでは 画像 + 状態 + 言語指示
エピソード	episode	タスク 1 回分の連続した記録 (reset から done まで) の時系列

方策	policy	「観測を入れると行動を返す」関数そのもの。学習する VLA は方策
行動チャンク	action chunking	次の 1 手だけでなく未来の数ステップ分の行動をまとめて予測すること。本書では 8 ステップ分
ロールアウト	rollout	学習した方策を環境で 1 エピソード分、最後まで動かすこと

まとめ

- **VLA = 画像 + 言語 (+ 状態) → 行動** を出すモデル。VLM の出力を「行動」に置き換えたもの。
- 本書は **模倣学習 (行動クローニング)** : お手本を集める → データ化 → 回帰で真似 → ロールアウトで評価、の 4 ステップ。
- 完成物は **Tiny Tabletop 2D Pick-and-Place**。CPU・物理エンジン不要で本物の VLA の骨格を自作。
- 重要用語: action / state / observation / episode / policy / 行動チャンク / rollout。
- 環境構築は `uv sync` → `uv run pytest` → `configs/smoke.json` で学習が回る確認まで。
- 次章 M1 で道具 (PyTorch) を握ります。tensor・autograd・学習ループ・Dataset/DataLoader を、本書の題材 (状態 [3] や行動チャンク $[T, 3]$) に寄せて入門します。

PyTorch 速習

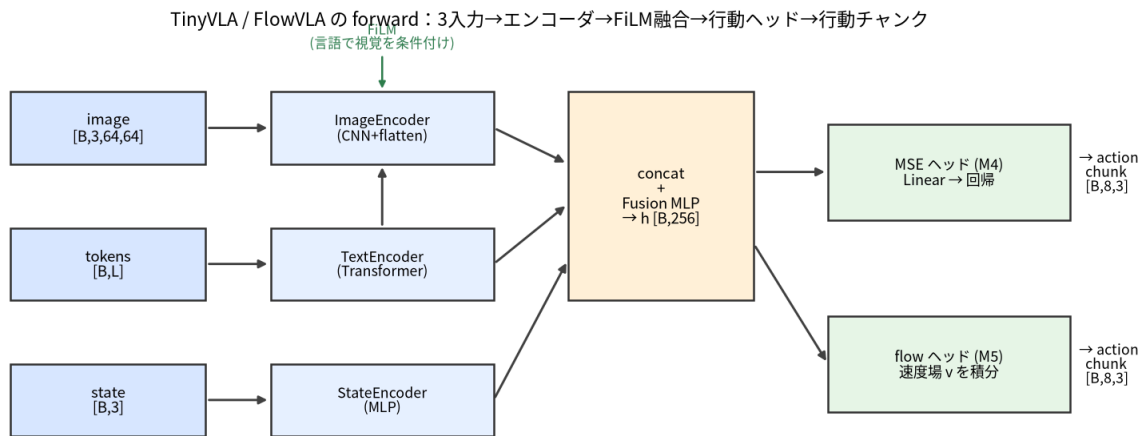
この章のゴール

- テンソル (tensor) を作り、`shape` / `dtype` / `device` を読めるようになる。
- ブロードキャスト (broadcasting) の規則を理解し、形の違う配列の演算を予測できる。
- 自動微分 (autograd): `requires_grad` / `backward()` / `.grad` / `no_grad` を使える。
- `nn.Module` / `nn.Linear` / `nn.Sequential` でモデルを定義できる。
- 損失と optimizer (Adam / `zero_grad` / `backward` / `step`) で学習ループを書ける。
- Dataset / DataLoader (`__len__` / `__getitem__` / バッチ化 / `shuffle`) を使える。
- 「1 バッチに過学習できるか」という学習デバグの鉄則を理解する。

この章は **PyTorch の基礎に集中** します。diffusion / VLM の座学は既知前提なので触れません。代わりに、後の章で実際に使う **本書の題材** (状態 [3]・行動チャック [T, 3]・小さな回帰) に寄せて練習します。本章のコードはすべて **CPU・コピペで動く** ように書いています (対話シェルか短い `.py` を `python xxx.py` で実行)。

座学とのつながり

この章で握る道具 (`tensor` / `autograd` / `nn.Module` / 学習ループ / `DataLoader`) は、最終的に下図の VLA を組み立てるための部品です。いま全体像が分からなくて大丈夫です。「この層は `nn.Linear`」「この矢印は `forward`」「学習は損失を `backward`」——と後で対応が取れるよう、完成形を一度だけ眺めておきましょう。



同じ backbone (青→橙) を共有し、右端のヘッドだけ MSE↔flow で差し替える

図 3: 本章で握る道具が組み上がる先 (M4 / M5 の完成形)。3 入力をエンコードし融合して行動チャックを出す。各エンコーダは `nn.Module`、矢印は `forward`、学習は損失の `backward`。実装は `src/vla_learn/models/tiny_vla.py`。

2.1 テンソル (tensor) —— PyTorch の基本データ

テンソルは「GPU でも動く、自動微分に対応した多次元配列」です。NumPy の `ndarray` とほぼ同じ感覚で使えます。

2.1.1 作る

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])    # リストから (状態 state[3] のイメージ)
z = torch.zeros(8, 3)               # 0 埋め [8, 3] (行動チャUNK [T=8, A=3] のイメージ)
o = torch.ones(2, 3)                # 1 埋め
r = torch.randn(4, 3)               # 標準正規分布から [4, 3]
e = torch.arange(6)                  # 0,1,2,3,4,5
print(a)                             # tensor([1., 2., 3.])
print(z.shape, r.shape)              # torch.Size([8, 3]) torch.Size([4, 3])
```

2.1.2 shape (形)

shape は各次元の大きさです。VLA では shape を読めることが最重要スキルになります (観測画像 $[B, C, H, W]$ 、行動 $[B, T, A]$ など)。

```
import torch

x = torch.randn(4, 8, 3)            # 例: バッチ4, チャUNK長8, 行動次元3 = [B, T, A]
print(x.shape)                       # torch.Size([4, 8, 3])
print(x.shape[0], x.ndim)            # 4 3 (バッチサイズ と 次元数)

flat = x.reshape(4, 24)              # 形を変える (要素数は不変) [4, 8*3] = [4, 24]
print(flat.shape)                    # torch.Size([4, 24])

s = torch.randn(3)                   # [3]
print(s.unsqueeze(0).shape)           # [1, 3] 先頭にバッチ次元を足す
print(s.unsqueeze(0).squeeze(0).shape) # [3] 大きさ1の次元を消す
```

補足

読み方の規約 (本書共通)

- B = バッチサイズ、 T または C = チャUNK長 (時間ステップ数)、 A = 行動次元 (=3)、 D = 特徴次元。
- 画像は $[B, C, H, W]$ (C =チャUNK=3, $H=W=64$)。
- 行動チャUNKは $[B, T, A]$ (例 $[64, 8, 3]$)、状態は $[B, D]$ (例 $[64, 3]$)。

2.1.3 dtype (型)

dtype は数値の型です。float と int を混ぜると壊れるので注意します。

```
import torch

f = torch.tensor([1.0, 2.0])         # 既定は float32
i = torch.tensor([1, 2])              # 整数リストなら int64
print(f.dtype, i.dtype)               # torch.float32 torch.int64

g = torch.zeros(3, dtype=torch.float32)
h = i.float()                         # int64 -> float32
j = f.long()                          # float32 -> int64
print(g.dtype, h.dtype, j.dtype)      # torch.float32 torch.float32 torch.int64
```

つまずき / バグの定番

本書での型の約束 (`src/vla_learn/datasets/synthetic_dataset.py` の `__getitem__` より): 画像・状態・行動・`pad_mask` は **float32**、言語のトークン ID (`tokens`) は **int64** です。トークン ID が `int64`なのは、後で `nn.Embedding` (埋め込み表) の索引として使うから——索引は整数でなければなりません。ここを `float` にすると埋め込みでエラーになります。

2.1.4 device (CPU / GPU)

テンソルは CPU か GPU のどちらかに乗っています。**鉄則: 演算する者どうしは同じ device に置く** (混在は実行時エラー)。本書は CPU 完結なので基本は `cpu` のままで困りません。

```
import torch
from vla_learn.utils.device import get_device

x = torch.randn(2, 3)          # 既定は CPU
print(x.device)                # cpu

dev = get_device()             # GPU があれば cuda、無ければ cpu
x = x.to(dev)                  # その device へ移す
print(x.device)
```

実装を読む `src/vla_learn/utils/device.py` → `get_device`

`get_device()` は `torch.cuda.is_available()` を見て `cuda` か `cpu` を返すだけの数行です。ファイル冒頭コメントに「テンソルとモデルは同じ device に置く」という鉄則が書いてあります。本書の学習・評価スクリプトはみなこのヘルパで device を取ります。

2.1.5 NumPy との行き来

環境 (`envs/`) は NumPy で世界を作り、学習は PyTorch です。両者は橋渡しできます。

```
import numpy as np
import torch

arr = np.array([0.3, 0.7, 0.0], dtype=np.float32) # state [ax, ay, gripper]
t = torch.from_numpy(arr)                         # NumPy -> tensor (メモリ共有)
print(t, t.dtype)                                # tensor([0.3000, 0.7000, 0.0000]) torch.float32

back = t.numpy()                                  # tensor -> NumPy
print(type(back))                                 # <class 'numpy.ndarray'>
```

実際、後の章で作る `SyntheticVLADataset.__getitem__` は `torch.from_numpy(...)` で NumPy 画像・状態・行動を tensor に変換して返しています (M3)。

2.2 ブロードキャスト (broadcasting)

形の違うテンソルどうしの演算を、PyTorch が **自動で形を合わせて** 計算してくれる仕組みです。規則は「**末尾の次元から見て、大きさが等しい or どちらかが 1 なら OK**」。1 の側が引き伸ばされます。

```
import torch

x = torch.randn(4, 3)          # [B=4, D=3]
b = torch.tensor([10.0, 20.0, 30.0]) # [3] -> [1, 3] とみなされ各行に足される
print((x + b).shape)            # torch.Size([4, 3])

col = torch.tensor([[1.0], [2.0], [3.0], [4.0]]) # [4, 1] -> 各列にブロードキャスト
print((x + col).shape)          # torch.Size([4, 3])
```

本書で実際に使われる例です。パディング位置の誤差を 0 にする `masked_mse`（行動チャンクの損失）で、`[B, C]` のマスクを `[B, C, 1]` にしてから二乗誤差 `[B, C, A]` に掛けます。

```
import torch

se = torch.randn(4, 8, 3) ** 2 # 二乗誤差 [B, C, A]
mask = torch.ones(4, 8) # pad_mask [B, C] (1=有効, 0=パディング)
mask3 = mask.unsqueeze(-1) # [B, C, 1]
masked = se * mask3 # [B,C,1] が [B,C,3] にブロードキャストされる
print(masked.shape) # torch.Size([4, 8, 3])
```

`unsqueeze(-1)` で `[B, C]` を `[B, C, 1]` にしてから掛けると、行動次元 $A = 3$ 方向に同じマスクが伸びて、パディング位置の誤差をまとめて 0 にできます。「次元を 1 にして broadcast」は頻出パターンです。

実装を読む `src/vla_learn/functional.py` → `masked_mse`

この `unsqueeze(-1)` で broadcast するパターンが実際に使われている場所（`mask.unsqueeze(-1).expand_as(se)`）。行動チャンクは長さが足りない分を 0 でパディングし、`pad_mask` でそこを損失から除外します。M4 の学習損失そのもので、学習側からは `from vla_learn.training.losses import masked_mse` で使います。broadcast の練習として今のうちに眺めておくと M4 がスムーズです。

つまずき / バグの定番

よくある罠: `[8, 3]` と `[3, 8]` は broadcast できません（末尾から見て $3 \neq 8$ かつ 1 でもない）。「形が合わない」エラーの大半はこれです。落ち着いて `.shape` を print しましょう。

2.3 自動微分 (autograd)

ニューラルネットの学習は「損失を下げる方向にパラメータを少し動かす」の繰り返しです。その「方向」= 勾配 (gradient) を、PyTorch が自動で計算してくれるのが autograd です。座学で出てきた誤差逆伝播 (backpropagation) を、手で微分せずに使えます。

2.3.1 requires_grad / backward / .grad

```
import torch

x = torch.tensor([2.0], requires_grad=True) # 「微分の対象」として追跡される
y = x ** 2 + 3 * x # y = x^2 + 3x
y.backward() # dy/dx を計算 (逆伝播)
print(x.grad) # dy/dx = 2x + 3 = 2*2+3 = 7 -> tensor([7.])
```

ポイント:

- `requires_grad=True` のテンソルから計算した結果には、その履歴（計算グラフ）が記録されます。
- `loss.backward()` を呼ぶと、グラフを逆向きにたどって各 `requires_grad=True` のテンソルの `.grad` に勾配が貯まります。
- `.grad` は加算されていく（上書きではない）。だから毎ステップ `zero_grad()` でリセットが必要です（後述。忘れると壊れます）。

2.3.2 no_grad（勾配を切る）

推論時や勾配が要らない処理では `torch.no_grad()` で追跡を止めます。メモリと速度の節約になり、また「パラメータ更新の式に勾配を混ぜない」ために必須です。

```
import torch

w = torch.tensor([1.0], requires_grad=True)
with torch.no_grad():
    w2 = w * 5          # この計算は追跡されない
print(w2.requires_grad) # False
```

実装を読む src/vla_learn/models/tiny_vla.py → TinyVLA.predict

実装では TinyVLA.predict が @torch.no_grad() デコレータで推論し、評価ループ (evaluation/rollout.py) でも勾配を切って行動を予測します。**学習中は勾配を追跡、推論中は切る、**と覚えてください。同じファイルの forward と並べて読むと「学習用と推論用の入口」の違いが分かります。

2.4 nn.Module / nn.Linear / nn.Sequential

毎回パラメータを手で持つのは大変です。PyTorch では `nn.Module` を継承してモデルを作ると、パラメータ管理・device 移動・学習/評価モード切替などを面倒見てくれます。

2.4.1 nn.Linear (全結合層)

`nn.Linear(in, out)` は $y = xW^T + b$ を計算する層です。入力の**最後の次元**を `in` から `out` に変えます。

```
import torch
import torch.nn as nn

lin = nn.Linear(3, 64)          # 状態 state[*, 3] -> 特徴[*, 64]
s = torch.randn(4, 3)          # [B=4, 3]
out = lin(s)
print(out.shape)                # torch.Size([4, 64])
print(lin.weight.shape, lin.bias.shape) # torch.Size([64, 3]) torch.Size([64])
```

これは本書の StateEncoder (状態 $[*, 3] \rightarrow [*, 64]$) と同じ発想です (M4)。

2.4.2 nn.Module を継承する

```
import torch
import torch.nn as nn

class TinyMLP(nn.Module):
    def __init__(self, in_dim=3, hidden=32, out_dim=3):
        super().__init__()          # ← 必ず最初に呼ぶ
        self.fc1 = nn.Linear(in_dim, hidden)
        self.fc2 = nn.Linear(hidden, out_dim)
        self.act = nn.ReLU()

    def forward(self, x):              # 順伝播を書く
        h = self.act(self.fc1(x))
        return self.fc2(h)

model = TinyMLP()
x = torch.randn(4, 3)                # [B, 3]
y = model(x)                          # model(x) は forward(x) を呼ぶ
print(y.shape)                       # torch.Size([4, 3])
```

```
n_params = sum(p.numel() for p in model.parameters()) # パラメータは自動で集まる
print("params:", n_params)
```

つまづき / バグの定番

モデルを呼ぶときは `model.forward(x)` ではなく **`model(x)`** と書きます（フックなどが正しく動くため）。また `super().__init__()` を忘れると、`self.fc1 = nn.Linear(...)` を代入してもパラメータが登録されず、`model.parameters()` が空になって学習できません（最頻出の初心者バグ）。

実装を読む `src/vla_learn/models/tiny_vla.py` → `count_parameters`

上の `sum(p.numel() for p in model.parameters())` と同じ書き方が `count_parameters` です（`requires_grad` のものだけ数える点だけ違う）。これで TinyVLA が約 0.42M パラメータ、と測れます。モデル規模をすぐ把握できるので、自作モデルを作るたびに呼ぶ習慣をつけましょう。

2.4.3 nn.Sequential（層を直列に並べる）

分岐のない単純な積み重ねは `nn.Sequential` が簡潔です。

```
import torch
import torch.nn as nn

mlp = nn.Sequential(
    nn.Linear(3, 32), nn.ReLU(),
    nn.Linear(32, 32), nn.ReLU(),
    nn.Linear(32, 3),
)
print(mlp(torch.randn(4, 3)).shape) # torch.Size([4, 3])
```

実装を読む `src/vla_learn/models/tiny_vla.py` → `VLABackbone`

実際、VLA 本体 `VLABackbone` の融合 MLP は `nn.Sequential(nn.Linear(...), nn.ReLU(inplace=True), nn.Linear(...), nn.ReLU(inplace=True))` で書かれています（本章の書き方そのまま）。`forward` で3つのエンコーダ出力を `torch.cat` してこの MLP に通し、条件ベクトル h を作ります。冒頭の図の「融合」がこのクラスです。

2.5 学習ループ（損失と optimizer）

ここが PyTorch の心臓部です。毎ステップ次の4つを順番に呼ぶだけ、と覚えてください。

```
for ステップ in 学習:
    ① pred = model(x) # 順伝播（予測）
    ② loss = 損失(pred, target) # どれだけ間違ったか
    ③ optimizer.zero_grad() # 前回の勾配を消す（忘れると加算され壊れる）
        loss.backward() # 逆伝播（.grad を計算）
    ④ optimizer.step() # パラメータを勾配方向に更新
```

2.5.1 最小の完全な例: 小さな線形回帰

「 $y = 2x + 1$ を当てる」を、本書の道具（`nn.Linear`・MSE・Adam）で学習します。そのままコピペで動きます。

```
import torch
import torch.nn as nn
```

```

torch.manual_seed(0)

# 1) データ (答え  $y = 2x + 1$ 。少しノイズを足す)
x = torch.randn(128, 1)          # [N=128, 1]
y = 2.0 * x + 1.0 + 0.05 * torch.randn(128, 1)

# 2) モデル・損失・optimizer
model = nn.Linear(1, 1)           # 直線  $y = w x + b$  を学習
loss_fn = nn.MSELoss()            # 平均二乗誤差
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)

# 3) 学習ループ
for step in range(200):
    pred = model(x)                # ① 順伝播 [128, 1]
    loss = loss_fn(pred, y)        # ② 損失 (スカラー)
    optimizer.zero_grad()          # ③ 勾配リセット
    loss.backward()                 # 逆伝播
    optimizer.step()                # ④ 更新
    if step % 50 == 0:
        print(f"step {step:3d}  loss = {loss.item():.4f}")

w = model.weight.item()
b = model.bias.item()
print(f"学習結果:  $y \approx \{w:.2f\} x + \{b:.2f\}$  (正解は  $2.00 x + 1.00$ )")

```

出力例 (数値は環境でぶれます) :

```

step   0  loss = 10.1834
step  50  loss = 0.7888
step 100  loss = 0.0052
step 150  loss = 0.0017
学習結果:  $y \approx 2.00 x + 1.00$  (正解は  $2.00 x + 1.00$ )

```

各行の意味:

- `nn.MSELoss()`: 予測と正解の差の二乗の平均。回帰の定番 (本書 M4 も MSE です)。
- `torch.optim.Adam(model.parameters(), lr=...)`: 更新方法。 `model.parameters()` を渡すことで「どのテンソルを更新するか」を optimizer に教えます。 `lr` (学習率) は 1 歩の大きさ。
- `loss.item()`: スカラーのテンソルから Python の `float` を取り出します。 `print` や記録には `.item()` を使う (テンソルのまま貯めると計算グラフが残りメモリを食う/壊れる原因に)。

つまずき / バグの定番

`zero_grad()` を忘れるとどうなる? `.grad` は加算され続けるので、勾配がどんどん膨らみ、更新が暴れて `loss` が下がりません。「`loss` が下がらない」ときに真っ先に疑う定番バグです。

2.6 「1 バッチに過学習できるか」——学習デバッグの鉄則

VLA に限らず、学習コードを書いたら**最初に必ず確認すべきこと**があります。

座学とのつながり

鉄則: モデルと学習ループが正しければ、小さな 1 バッチ (数十サンプル) には必ず過学習できる。過学習すらできないなら、損失・shape・最適化・勾配のどこかにバグがある。

なぜか。汎化（未知データへの一般化）は難しい問題ですが、「**手元の固定された数十サンプルを丸暗記する**」のは、配線が正しければニューラルネットには簡単なはずだからです。つまり「1 バッチに過学習できるか」は **モデルの賢さのテストではなく、配線（パイプライン）の健全性テスト**です。大きなデータで延々と回す前に、これで素早くバグを切り分けます。

確認のしかた:

1. データを**ごく少数**（例 16 サンプル）に固定する（`shuffle` は切る or 同じバッチを使い続ける）。
2. 同じバッチで**何百ステップ**も学習を回す。
3. **loss がほぼ 0 近くまで下がれば合格**。下がらなければバグを探す。

上の線形回帰の例も、実は 128 サンプルを固定して回しているのだから「1 バッチ過学習」に近い形でした。loss が小さく下がりましたね。これが「配線は正しい」というシグナルです。

実装を読む `tests/test_overfit_tiny_batch.py` → `test_mse_overfits_one_batch`

本書の「健全性テスト」の実体。TinyVLA を 1 バッチ（16 サンプル）で 200 ステップ学習し、**loss が最初の 20% 未満まで下がること**（`loss.item() < 0.2 * first`）を `assert` します。`next(iter(DataLoader(ds, batch_size=16, shuffle=True)))` で 1 バッチを取り出す書き方（次節）もここで使われています。各章の演習でも必ず 1 問、この確認を入れます。

うまく下がらないときのチェックリスト:

- `optimizer.zero_grad()` を呼んでいるか（最頻出バグ）。
- 学習率 `lr` が極端でないか（大きすぎると発散、小さすぎると動かない。まず `1e-3` 付近から）。
- 予測と正解の **shape が一致**しているか（ズレていると `broadcast` で意図しない損失になる）。
- モデルの出力に `model.parameters()` が勾配でつながっているか（`detach()` / `no_grad` で切っていないか）。
- 入力・モデルが同じ **device / dtype** か。

2.7 Dataset と DataLoader

データが増えると、「どう 1 件を取り出すか」と「どうミニバッチにまとめるか」を分けて書きたくなります。PyTorch では **Dataset** が前者、**DataLoader** が後者を担当します。

2.7.1 Dataset: `__len__` と `__getitem__`

`Dataset` は 2 つのメソッドさえ実装すればよい、というルールです。

- `__len__(self)` … データ件数を返す。
- `__getitem__(self, idx)` … `idx` 番目の 1 件を返す（dict やタプル）。

本書の題材に寄せた最小例（状態 [3] → 行動チャंक [8, 3] の組を返す擬似データセット）:

```
import torch
from torch.utils.data import Dataset

class ToyVLADataset(Dataset):
    def __init__(self, n=100, chunk_len=8, action_dim=3):
        torch.manual_seed(0)
        self.states = torch.randn(n, 3)  # [N, 3]
        self.actions = torch.randn(n, chunk_len, action_dim)  # [N, 8, 3]

    def __len__(self):
        return self.states.shape[0]  # 件数

    def __getitem__(self, idx):
        return {
```

```

        "state": self.states[idx],          # [3]
        "action": self.actions[idx],       # [8, 3]
    }

ds = ToyVLADataset()
print(len(ds))                          # 100
sample = ds[0]
print(sample["state"].shape, sample["action"].shape)  # torch.Size([3]) torch.Size([8, 3])

```

実装を読む src/vla_learn/datasets/synthetic_dataset.py → SyntheticVLADataset.__getitem__

実物の SyntheticVLADataset も同じ形で、__getitem__ が {"image": [3, 64, 64], "state": [3], "tokens": [L], "action": [8, 3], "pad_mask": [8]} を返します（型は画像・状態・行動・pad_mask が float32、tokens が int64）。この章の ToyVLADataset を「本物の観測」に差し替えたものが M3 の到達点です。

2.7.2 DataLoader: バッチ化と shuffle

DataLoader は Dataset を包み、ミニバッチにまとめて (batch_size)、毎エポック順番をシャッフル (shuffle=True) して、繰り返し取り出せるようにします。1 件が [3] でも、batch_size=16 で取り出すと先頭にバッチ次元が付いて [16, 3] になります。

```

import torch
from torch.utils.data import DataLoader

loader = DataLoader(ds, batch_size=16, shuffle=True)

batch = next(iter(loader))          # 最初のバッチを 1 つ取り出す
print(batch["state"].shape)         # torch.Size([16, 3]) ← 先頭に B=16 が付く
print(batch["action"].shape)       # torch.Size([16, 8, 3])

for batch in loader:                # 学習ではこう回す (1 エポック = 全データを一巡)
    s = batch["state"]              # [16, 3]
    a = batch["action"]             # [16, 8, 3]
    # ここで model(...) と loss.backward() ...
    pass

```

ポイント:

- **batch_size**: 一度に何件まとめるか。大きいほど勾配が安定し速いが、メモリを食う。
- **shuffle=True**: 毎エポック並び替える。順序の偏りで学習がゆがむのを防ぐ。学習データは True、評価データは False が定石。
- 辞書もまとめてくれる: __getitem__ が dict を返すと、DataLoader がキーごとにスタックして {"state": [B, 3], "action": [B, 8, 3]} にしてくれます (賢い)。

補足

「1 バッチ過学習」との合わせ技: batch = next(iter(loader)) で 1 バッチだけ取り出し、それを使い回して数百ステップ回せば、前節の健全性テストがそのまま書けます。本書の tests/test_overfit_tiny_batch.py もまさに next(iter(DataLoader(ds, batch_size=16, shuffle=True))) で 1 バッチを取り出しています。

2.7.3 学習ループ + DataLoader (まとめ)

これまでの部品を全部つなぐと、本書で何度も書く形になります。

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader

torch.manual_seed(0)
ds = ToyVLADataset(n=256) # 上で定義した擬似データ
loader = DataLoader(ds, batch_size=32, shuffle=True)

# 状態[3] -> 行動チャック[8,3] を出すごく小さなモデル
model = nn.Sequential(nn.Linear(3, 64), nn.ReLU(), nn.Linear(64, 8 * 3))
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(5):
    running = 0.0
    for batch in loader:
        s = batch["state"] # [B, 3]
        a = batch["action"] # [B, 8, 3]
        pred = model(s).view(-1, 8, 3) # [B, 24] -> [B, 8, 3]
        loss = loss_fn(pred, a)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running += loss.item()
    print(f"epoch {epoch} mean loss = {running / len(loader):.4f}")

```

ここでは答え `a` がランダムなので「賢く当てる」のは無理ですが、配線 (Dataset → DataLoader → model → loss → backward → step) が一巡することを確認できます。本物のデータ (M3以降) に差し替えれば、そのまま VLA の学習になります。

まとめ

- **tensor**: 多次元配列。shape・dtype・device を読めることが最重要。float と int、CPU と GPU は混ぜない。
- **broadcast**: 末尾の次元から見て「等しい or 片方が 1」なら自動で形が合う。unsqueeze(-1) で 1 を作って掛けるのは頻出パターン (masked_mse)。
- **autograd**: requires_grad=True → 計算 → loss.backward() で .grad に勾配が貯まる。推論は no_grad。 .grad は加算されるので zero_grad が必要。
- **nn.Module / nn.Linear / nn.Sequential**: モデルを定義しパラメータを自動管理。呼び出しは model(x)。super().__init__() を忘れない。
- **学習ループ**: pred → loss → zero_grad → backward → step の 4 つ。Adam に model.parameters() を渡す。記録は .item()。
- **1 バッチ過学習**: 学習コードを書いたら最初に確認する鉄則。配線の健全性テスト。
- **Dataset / DataLoader**: __len__ と __getitem__ で 1 件を定義 → DataLoader でバッチ化・shuffle。バッチ化で先頭に B 次元が付く。
- 道具がそろいました。次章 M2 で「状態 → 行動」「画像 → 行動」をこの学習ループで回帰し、なぜ素朴な模倣はロールアウトで崩れるのか (行動チャックが必要になる理由) を体験します。

最小の模倣学習 (behavior cloning)

この章のゴール

- 模倣学習 (imitation learning / behavior cloning, BC) = 「お手本の行動」を教師にして「状態 → 行動」を回帰する、という考え方を体で覚える。
- まず `state[3] → action[3]` の最小モデルを学習し、次に `image → action` へ拡張する。
- 本章の山場: 素朴な BC が 閉ループで崩れる 理由 = 分布シフト (distribution shift) と 誤差蓄積 (compounding error) を図で理解し、`generate_episodes(action_noise=...)` の ノイズ注入 (DART / DAgger 風) で対策を体験する。
- 学習デバッグの鉄則「1 バッチに過学習できるか」を実際に確認する。

3.1 この章で作る「最小の VLA の祖先」

最終的な VLA は 画像 + 言語 + 状態 → 行動チャックを出します (M4 で完成)。でもいきなり全部入りだと、何が効いているのか分かりません。そこでこの章では 言語をいったん外し、次の 2 段階の「祖先モデル」を作ります。

段階	入力 → 出力	使う感覚
その 1	<code>state[3] → action[3]</code> (画像すら使わない)	固有受容のみ
その 2	<code>image[3, 64, 64] → action[3]</code>	視覚のみ

やることは M1 で学んだ学習ループの応用です。新しいのは「教師データがどこから来るか」= エキスパート (expert) の存在だけです。

座学とのつながり

これは強化学習ではありません。報酬を最大化するのではなく、「お手本の入力→出力ペア」をそのまま 教師あり回帰 で覚えるだけです。拡散モデルで「データ分布を真似る」のと同じく、ここでも エキスパートの行動分布を真似ます。発想は地続きです。

3.2 エキスパート（お手本）とは

このリポジトリには、ニューラルネットを一切使わない 解析エキスパート が入っています。ワールドの真の状態を直接読めるので、ルールベースで 毎回成功 します (成功率 ≈ 100%、配置でぶれます)。考え方はシンプルな状態機械です。

状況	行動 $[d_x, d_y, \text{grip}]$
対象ブロックを 持っていない	ブロックへ近づく。十分近ければ (0, 0, 1) で 掴む
対象ブロックを 持っている	ゴールへ近づく。十分近ければ (0, 0, 0) で 置く

実装を読む `src/vla_learn/envs/expert.py → expert_action`

この章の教師ラベルはすべてここから出ます。world から対象ブロック obj とゴール goal を読み、「持っているか (holding)」で分岐して、目標へのベクトル d を MAX_STEP=0.08 でクリップした 1 ステップ行動を返すだけ——50 行未満です。PICK_THRESH=0.10 / PLACE_THRESH=0.08 (掴む・置く判定

距離) が環境の GRASP_RADIUS=0.18 / SUCCESS_RADIUS=0.12 より内側なので、多少ブレても掴める/置けます。

模倣学習では、このエキスパートに環境を解かせて「**そのとき何を見て (観測)、何をしたか (行動)**」を大量に記録し、その 観測 → 行動 をニューラルネットに教師あり学習で覚えさせます。

補足

「エキスパートが必要なら最初からエキスパートを使えば？」と思うかもしれませんが。ポイントは、エキスパートは **真の状態 world** を覗いていること。一方、学習するモデルが見られるのは **画像と状態だけ** (本物のロボットと同じ条件) です。つまり「真の状態を知っている先生」から「センサしか見られない生徒」へ知識を移すのが模倣学習です。

3.3 デモを集める: generate_episodes

エピソード (1 回の試行) を集めるのは generate_episodes です。各時刻で **記録するラベルは常にクリーンなエキスパート行動** `a = expert_action(w)`、**実行だけ** を後でノイズで乱す、という構造を覚えてください (節 3.6 で効いてきます)。

実装を読む `src/vla_learn/datasets/synthetic_dataset.py` → `generate_episodes`

`while not done:` のループが本体。 `a = expert_action(w)` を `acts` に積み、 `state` を `agents` に積み、 `a_exec = a.copy()` の `dx, dy` だけにガウスノイズを足して `env.step(a_exec)` に渡します (`grip` は乱さない)。 `only_success=True` なので成功エピソードだけが残ります。ここで **ラベルと実行が分かれている** のが本章後半の肝です。

```
from vla_learn.datasets import generate_episodes

eps = generate_episodes(n_episodes=200, seed=0) # action_noise=0.0 が既定
print("エピソード数:", len(eps))
print("最初のエピソードのキー:", list(eps[0].keys()))
print("actions の shape:", eps[0]["actions"].shape) # [T, 3]
print("agent (state) の shape:", eps[0]["agent"].shape) # [T, 3]
print("instruction:", eps[0]["instruction"])
```

出力例 (数値はシードや環境でぶれます) :

```
エピソード数: 200
最初のエピソードのキー: ['agent', 'objects_pos', 'objects_color', 'goals_pos', 'goals_color',
'target_obj', 'target_goal', 'instruction', 'actions']
actions の shape: (9, 3)
agent (state) の shape: (9, 3)
instruction: 赤ブロックをつかんで黄ゴールへ
```

各エピソードは T (おおむね 5~16、平均 8~9 程度。配置でぶれます) ステップの時系列で、 `agent[t]` = そのときの状態 $[a_x, a_y, gripper]$ 、 `actions[t]` = そのとき取った行動 $[d_x, d_y, grip]$ です。

補足

既定の `action_noise` は関数 `generate_episodes` では **0.0** (まずノイズ無しで素直に集める)。一方、学習設定 `TrainConfig` の既定は `action_noise=0.03` です (M4 の本番学習はノイズ込みで集める)。この **2 つの既定値は別物** なので取り違えないでください。本章後半でこの差が成功率に効く理由を見ます。

補足

ここでは **行動チャンク**（まとめて8ステップ予測）はまだ使いません。「1ステップの行動」を1つ予測する素朴版から始めます。チャンクはM3で導入します。

3.4 最小の模倣学習（その1）：state[3] → action[3]

まずは画像も言語も使わず、状態だけから行動を当てる **いちばん小さい回帰** をやります。M1の学習ループがそのまま使えることを確認しましょう。

3.4.1 データを (state, action) のペアに平らにする

エピソードは時系列ですが、素朴BCでは時間構造を忘れて **全時刻の (state, action) をただ集めた集合** として扱えば十分です。

```
import numpy as np
import torch
from vla_learn.datasets import generate_episodes

eps = generate_episodes(n_episodes=300, seed=0)

# 全エピソード・全時刻を縦に積む
states = np.concatenate([ep["agent"] for ep in eps], axis=0) # [sumT, 3]
actions = np.concatenate([ep["actions"] for ep in eps], axis=0) # [sumT, 3]
print("学習サンプル数:", states.shape[0]) # 例: 4123 (ぶれます)

X = torch.from_numpy(states).float() # [N, 3]
Y = torch.from_numpy(actions).float() # [N, 3]
```

3.4.2 2層 MLP を定義する (nn.Module)

```
import torch.nn as nn

class StateToAction(nn.Module):
    def __init__(self, in_dim=3, hidden=64, out_dim=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, out_dim), # [B,3] を出力（回帰なので活性化なし）
        )

    def forward(self, state): # state: [B, 3]
        return self.net(state) # -> [B, 3]
```

つまずき / バグの定番

なぜ最後に活性化を付けないのか。出力は連続値の行動 $[d_x, d_y, \text{grip}]$ です。 d_x, d_y は負にもなりうるので、ReLU や Sigmoid で範囲を縛るとお手本を再現できません。回帰の出力層は素のまま（恒等写像）が基本です。

3.4.3 学習ループ（M1の応用そのまま）

```
from torch.utils.data import TensorDataset, DataLoader
from vla_learn.utils import set_seed
```

```

set_seed(0)
loader = DataLoader(TensorDataset(X, Y), batch_size=256, shuffle=True)

model = StateToAction()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

for epoch in range(20):
    total = 0.0
    for xb, yb in loader:
        pred = model(xb)          # [B, 3]
        loss = loss_fn(pred, yb)  # 平均二乗誤差
        opt.zero_grad(); loss.backward(); opt.step()
        total += loss.item() * len(xb)
    if epoch % 5 == 0 or epoch == 19:
        print(f"epoch {epoch:2d}  train MSE = {total/len(X):.4f}")

```

出力例（傾向。実際の値はぶれます）：

```

epoch 0  train MSE = 0.1707
epoch 5  train MSE = 0.0347
epoch 10 train MSE = 0.0323
epoch 19 train MSE = 0.0318

```

loss が下がれば、「状態 → 行動」をそれっぽく回帰できています（ある程度で頭打ちになるのは state[3] だけでは情報が足りないから。節 3.5 で詳しく）。ただし **MSE が下がること** と **タスクが解けること** は別物です。これが次節の主題です。

補足

ここで state を正規化していないことに注意してください ($a_x, a_y \in [0, 1]$, gripper $\in \{0, 1\}$ なので たまたま大きく外れた値はありません)。本番では正規化が効きます。M3 で扱います。

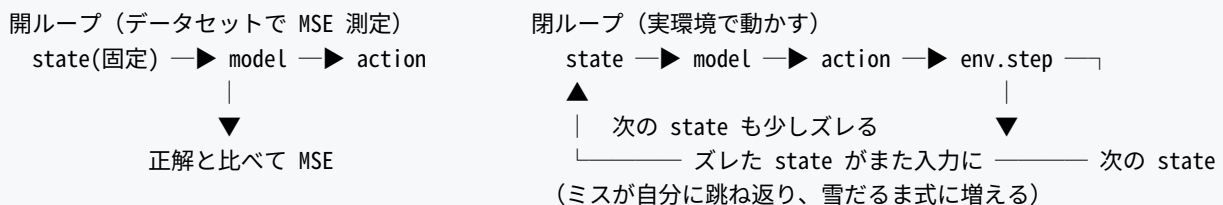
3.5 【山場】なぜ素朴な模倣は閉ループで崩れるのか

ここがこの章でいちばん大事なところですよ。前節の学習は **オフラインの MSE** を下げただけ。実際にロボットを動かす **閉ループ (closed-loop)** では、**自分の行動が次の観測を作る** という決定的な違いがあります。

3.5.1 開ループ評価と閉ループ評価

- **開ループ (open-loop)**: データセットの state を入れて action を当て、MSE を測る。状態は固定。
- **閉ループ (closed-loop)**: モデルの行動を環境に渡し、次の状態をモデル自身が作る。予測がほんの少しズレると、次に見る状態がデータに無かった状態になっていきます。

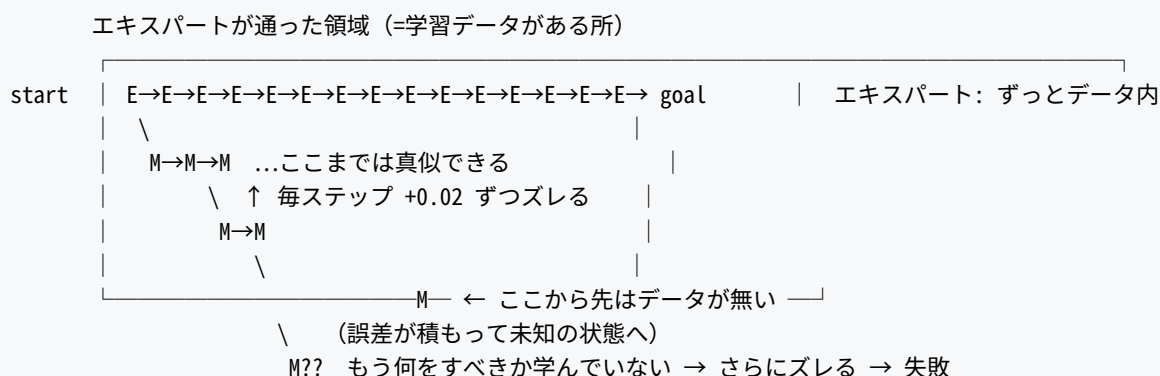
違いは「入力をどこから持ってくるか」の一点だけです：



- 開ループの入力は **データセットが用意した正しい state**。だから誤差は積もりません。
- 閉ループの入力は **モデル自身が作った(ズレているかもしれない) state**。ここでミスが跳ね返ります。

3.5.2 誤差蓄積 (compounding error) の図

エキスパートは画面の中央付近の「お手本の通り道」だけを通ります。学習モデルは各ステップでほんの少し (例: 0.02) ズれます。閉ループではそのズレが **積み上がり**、やがてエキスパートが一度も通らなかった領域に入り、そこではモデルは出鱈目を返します。



これが **分布シフト (distribution shift)** と **誤差蓄積 (compounding error)** です。

- **分布シフト**: 学習時に見た状態の分布 (=エキスパートの軌道) と、推論時にモデル自身が訪れる状態の分布が **ズれる**。
- **誤差蓄積**: 1 ステップの小さな誤差が次の観測をわずかに変え、その誤差が次の入力になり…と **時間方向に雪だるま式** に増える。BC の弱点の根本原因です。

座学とのつながり

直感: 「**お手本は失敗の直し方を見せてくれない**」。エキスパートは完璧なので、軌道から外れた状況 (=リカバリ) が教師データにほぼ現れません。だからモデルは「外れたときにどう戻るか」を学べないのです。

3.5.3 まずは崩れる様子を「お手本との誤差」で体感する

実機評価 (rollout) の完全版は M4 で `evaluate_policy` を使います。ここでは PyTorch だけで **ミニ閉ループ** を回して「ズレが積もる」のを観察します。新しい環境を `reset` し、`state` → `action` モデルだけで動かし、各ステップで「モデルの行動」と「(その状態での) エキスパートの正解行動」のズレを測ります。

補足

先に予想してみてください。前節で `state` → `action` の MSE は 0.03 付近まで下がりました。では、このモデルを実際に環境で動かしたら、**閉ループ成功率** は何 % くらいになると思いますか? (a) MSE が十分小さいので 80% 以上 / (b) そこそこで 40~60% / (c) ほぼ 0%。

```
import numpy as np
import torch
from vla_learn.envs import Tabletop2DEnv, expert_action

@torch.no_grad()
def mini_closed_loop(model, n_episodes=30, seed=1000):
    model.eval()
    gaps, successes = [], 0
    for k in range(n_episodes):
        env = Tabletop2DEnv(seed=seed + k)
        obs = env.reset()
```

```

ep_gap, done = [], False
while not done:
    state = torch.from_numpy(obs["state"]).float().unsqueeze(0) # [1,3]
    a = model(state).squeeze(0).numpy() # [3]
    a_star = expert_action(obs["world"]) # 正解 (参考)
    ep_gap.append(float(np.linalg.norm(a[:2] - a_star[:2]))) # dx, dy のズレ
    obs, _, done, info = env.step(a) # ← 自分の行動で次状態が決まる
    gaps.append(np.mean(ep_gap))
    successes += int(info.get("success", False))
return np.mean(gaps), successes / n_episodes

mean_gap, succ = mini_closed_loop(model)
print(f"閉ループでの平均ズレ(dx,dy) = {mean_gap:.4f} 成功率 = {succ:.2%}")

```

出力例 (state だけの素朴 BC。値は大きくぶれます) :

閉ループでの平均ズレ(dx,dy) = 0.0988 成功率 = 3.33%

予想の答え合わせ: 正解は (c)。 オフライン MSE は 0.03 付近まで下がったのに、閉ループ成功率はほぼゼロです。「MSE が小さい ≠ タスクが解ける」を数字で確認できました。

つまづき / バグの定番

state[3] は (a_x, a_y , gripper) だけで、**どのブロックが対象か・対象がどこか**が分かりません (画像や言語を見ていないので当然です)。つまり素朴 state → action はそもそも **情報不足** でもあります。次節で画像を足して情報不足を解消した上で、**分布シフトそのもの**への対策 (ノイズ注入) を入れます。

3.6 【対策】ノイズ注入で「リカバリのお手本」を作る (DART / DAgger 風)

分布シフトの正体は「軌道から外れた状態のお手本が無い」ことでした。ならばわざと軌道を少し外して、その「外れた状態」での正解行動を集めればよい——これが `generate_episodes(action_noise=...)` の発想です。「デモを集める」節で #readcode した `generate_episodes` の構造を、もう一度図で押さえます。

- `a = expert_action(w)` … 記録するラベルは常にクリーンなエキスパート行動。
- `a_exec = a + ノイズ` … 実行だけを乱して軌道をずらす (grip は乱さない)。

つまり「ずれた状態 → そこから正しく戻すための行動」というペアが自然に集まります。

ノイズなし (BC)	ノイズあり (DART/DAGger風)
通り道は1本だけ	通り道のまわりに「外れ→戻す」例が増える
E→E→E→E→goal	E→E→E→E→E→goal
	↗ ↘ ↗ ↘ ← 少しずらして実行
	各点でラベルは「中心へ戻す」クリーン行動
→ 外れたら未知	→ 外れても「戻し方」を学習済み → 崩れにくい

座学とのつながり

用語メモ:

- **DAgger (Dataset Aggregation)**: 学習中のモデルを実際に動かし、その訪問状態でエキスパートに正解を聞いて **データを足していく** 反復手法。
- **DART (Disturbances for Augmenting Robot Trajectories)**: 収集時に **外乱 (ノイズ)** を注入して訪問状態を広げる手法。本リポジトリの `action_noise` は **DART 風** (ラベルはクリーン) です。
- どちらも狙いは同じ「**エキスパートの軌道の“まわり”も教師にして分布シフトを埋める**」。

- ・ 拡散モデルで「ノイズを加えたサンプルから元へ戻す」ことを学ぶのと地続きの発想です。

3.6.1 実験: ノイズの有無で閉ループ成功率を比べる

「ノイズ注入したデータで学習したモデルのほうが、閉ループで崩れにくい」を自分の手で確認します。公平に比べるため **画像入力** で揃えます (情報不足を解消した上で、分布シフト対策の効果だけを見る)。まず `image[3, 64, 64] → action[3]` のモデルを用意します (M1 の CNN の応用。畳み込み 3 段 + 全結合)。

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from vla_learn.envs import Tabletop2DEnv, expert_action
from vla_learn.envs.render import render_world
from vla_learn.utils import set_seed

class ImageToAction(nn.Module):
    """[B,3,64,64] -> [B,3] の最小 CNN 回帰。"""
    def __init__(self, out_dim=3):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 16, 3, stride=2, padding=1), nn.ReLU(), # 64->32
            nn.Conv2d(16, 32, 3, stride=2, padding=1), nn.ReLU(), # 32->16
            nn.Conv2d(32, 32, 3, stride=2, padding=1), nn.ReLU(), # 16->8
        )
        self.head = nn.Sequential(
            nn.Flatten(), # [B, 32*8*8] = [B, 2048]
            nn.Linear(32 * 8 * 8, 128), nn.ReLU(),
            nn.Linear(128, out_dim), # [B, 3]
        )

    def forward(self, image): # image: [B, 3, 64, 64]
        return self.head(self.conv(image))

class ImgActionDataset(Dataset):
    """エピソード列から (image[3,64,64], action[3]) を返す素朴版 (1 ステップ・チャンク無し)。"""
    def __init__(self, episodes):
        self.eps, self.index = episodes, []
        for ei, ep in enumerate(episodes):
            for t in range(ep["actions"].shape[0]):
                self.index.append((ei, t))

    def __len__(self):
        return len(self.index)

    def __getitem__(self, i):
        ei, t = self.index[i]
        ep = self.eps[ei]
        img = render_world(
            ep["objects_pos"][t], ep["objects_color"],
            ep["goals_pos"], ep["goals_color"],
            ep["agent"][t, :2], float(ep["agent"][t, 2]), size=64,
        ) # [3,64,64]
```

```

a = ep["actions"][t].astype(np.float32) # [3]
return torch.from_numpy(np.ascontiguousarray(img)), torch.from_numpy(a)

```

学習と閉ループ評価を1つの関数にまとめ、`action_noise` だけを振って比べます:

```

from vla_learn.datasets import generate_episodes

def train_image_bc(action_noise, n_episodes=300, epochs=8, seed=0):
    set_seed(seed)
    eps = generate_episodes(n_episodes=n_episodes, seed=seed, action_noise=action_noise)
    loader = DataLoader(ImgActionDataset(eps), batch_size=128, shuffle=True)
    model = ImageToAction()
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    loss_fn = nn.MSELoss()
    model.train()
    for _ in range(epochs):
        for img, a in loader:
            loss = loss_fn(model(img), a)
            opt.zero_grad(); loss.backward(); opt.step()
    return model

@torch.no_grad()
def closed_loop_success(model, n_episodes=50, seed=2000):
    model.eval()
    succ = 0
    for k in range(n_episodes):
        env = Tabletop2DEnv(seed=seed + k)
        obs = env.reset(); done = False
        while not done:
            img = torch.from_numpy(np.ascontiguousarray(obs["image"])).float().unsqueeze(0) #
[1,3,64,64]
            a = model(img).squeeze(0).numpy() # [3]
            obs, _, done, info = env.step(a)
            succ += int(info.get("success", False))
    return succ / n_episodes

m0 = train_image_bc(action_noise=0.00)
m1 = train_image_bc(action_noise=0.03) # TrainConfig の既定と同じ値
print("noise=0.00 閉ループ成功率:", f"{closed_loop_success(m0):.2%}")
print("noise=0.03 閉ループ成功率:", f"{closed_loop_success(m1):.2%}")

```

出力例 (強くぶれます。重要なのは絶対値ではなく「ノイズありが同等以上、多くの試行で上回る」傾向):

```

noise=0.00 閉ループ成功率: 56.00%
noise=0.03 閉ループ成功率: 78.00%

```

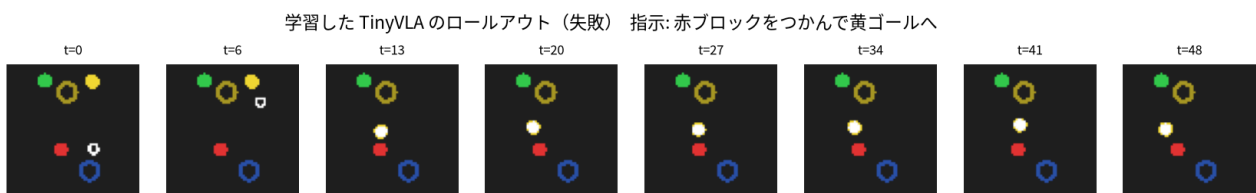


図 4: 画像入力で学習した方策の閉ループ・ロールアウト連続フレーム。各ステップの行動が次の観測を作る (=閉ループ)。情報源は `src/vla_learn/evaluation/rollout.py`、図は `scripts/make_figures.py` の `fig_loss_and_rollout`。

座学とのつながり

なぜ上がるのか: ノイズありのデータには「軌道から少し外れた状態+そこから戻す正解」が含まれるため、閉ループで誤差が積もっても**戻す行動**を出せるようになり、崩れにくくなります。逆にノイズが大きすぎると軌道が荒れてラベルとの対応が弱まり、かえって悪化することもあります（本リポジトリの既定は `action_noise=0.03` 付近。演習で振ってみましょう）。

つまずき / バグの定番

CPU でも `n_episodes=300, epochs=8` で 1 設定あたり数十秒～数分かかります。時間がなければ `n_episodes=150, epochs=4` に落としても傾向は見えます。成功率は**強くぶれる**ので、1 回の数字で「ノイズが悪い」と結論しないこと——複数シードで傾向を見ます。

3.7 学習デバグの鉄則: 1 バッチに過学習できるか

モデルと学習ループが正しければ、小さな 1 バッチには必ず（ほぼ 0 まで）過学習できるはずです。できないなら、損失・shape・最適化・勾配・`.train()/eval()` のどこかにバグがあります。これは毎章使う鉄則です。

実装を読む `tests/test_overfit_tiny_batch.py`

リポジトリのこのテストが同じ考え方を CI で守っています。「学習機構が壊れていない」ことの最小保証。自分のモデルを足したら、まずこの形のチェックを通すのが習慣です。

```
import torch
from torch.utils.data import TensorDataset, DataLoader
from vla_learn.utils import set_seed

set_seed(0)
# 前節の X, Y ([N,3]) から 1 バッチだけ取り出す
xb, yb = next(iter(DataLoader(TensorDataset(X, Y), batch_size=32, shuffle=True)))

model = StateToAction()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = torch.nn.MSELoss()

first = None
for step in range(300):
    loss = loss_fn(model(xb), yb)
    opt.zero_grad(); loss.backward(); opt.step()
    if first is None:
        first = loss.item()
print(f"最初の loss = {first:.4f} -> 最後の loss = {loss.item():.6f}")
assert loss.item() < 0.2 * first, "1 バッチに過学習できていない（どこかにバグ）"
print("OK: 1 バッチに過学習できました（学習機構は健全）")
```

出力例:

```
最初の loss = 0.2232 -> 最後の loss = 0.038375
OK: 1 バッチに過学習できました（学習機構は健全）
```

つまずき / バグの定番

もし下がらなければ、この順で疑います: ① `opt.zero_grad()` を忘れていないか ② `loss.backward()` の前に `pred` を `detach()` していないか ③ 入力と出力の `shape` が `[B,3]` で揃っているか ④ `model.train()` 状態か。(M1 のデバッグ手順)

補足

なお、ここで観察した「学習曲線が下がる」様子は、行動チャンク版の TinyVLA でも同じです。全データでの学習曲線（1 バッチではなく）の例が次の図です。

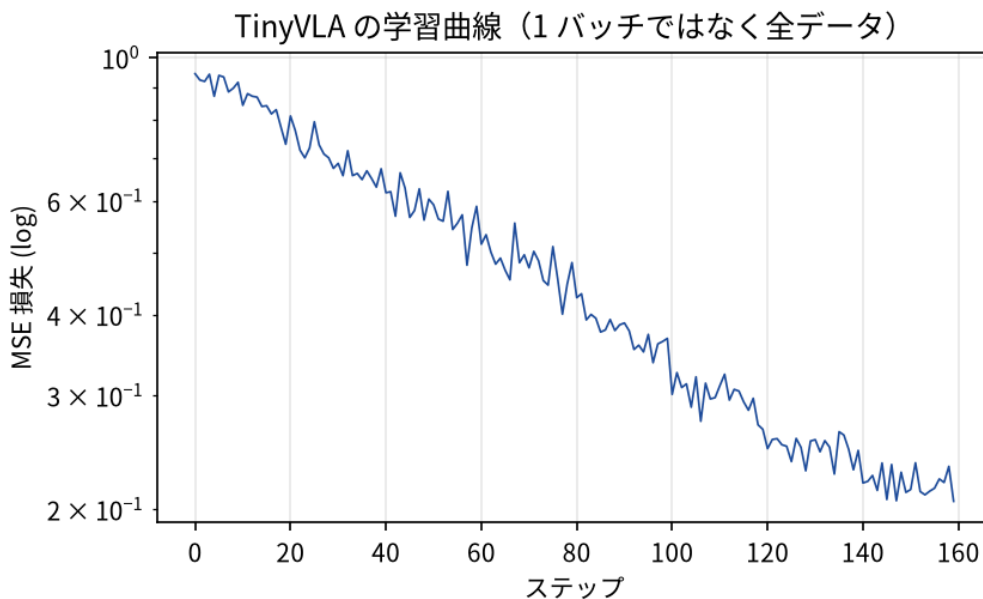


図 5: TinyVLA の学習曲線（1 バッチではなく全データ）。下がっても 閉ループで解けるとは限らない のが本章の教訓。図は `scripts/make_figures.py` の `fig_loss_and_rollout`。

まとめ

- **模倣学習 (BC)** = エキスパートの 観測 → 行動 を教師あり回帰で真似る。新しいのは「教師データの出所 = エキスパート」だけ。
- まず `state[3]` → `action[3]`、次に `image[3, 64, 64]` → `action[3]` を M1 の学習ループの応用で実装した。
- **オフライン MSE が下がっても閉ループで崩れる**。原因は **分布シフト** と **誤差蓄積**。「お手本は失敗の直し方を見せてくれない」。
- 対策は `generate_episodes(action_noise=...)` の **ノイズ注入 (DART/Dagger 風)**: 実行だけ乱してラベルはクリーンにし、「外れ → 戻す」のお手本を増やす。実験で成功率が上向く傾向を確認した。
- どの章でも **1 バッチに過学習できるか** で学習機構の健全性を最初に確かめる。
- 次章 M3 では、本物の VLA に必要な **データの作法** (正規化・時間窓・行動チャンク・トークナイザ) を学び、`SyntheticVLADataset` を完成させます。

行動表現とデータ

この章のゴール

- **正規化 (normalization)** をマスターする: `Normalizer.fit / normalize / denormalize`、なぜ「平均 0・分散 1」が嬉しいのか、**逆正規化を推論で使う** 理由。
- **時間方向の窓と行動チャンク (action chunking)** を理解する: `extract_action_chunk` と `pad_mask`、そして「**なぜまとめて予測するのか**」(`pi0 / ACT / SmolVLA` の発想)。
- **文字トークナイザ (CharTokenizer)** で言語指示を ID 列に変換する (`PAD=0`、語順の話は M4 への伏線)。
- 仕上げに `SyntheticVLADataset` が返す `dict` の **各 shape** を完全に把握し、**LeRobot 風データ辞書** への伏線を張る。

4.1 なぜ「行動表現とデータ」を独立した章にするのか

M2 では「観測 → 行動」をそのまま回帰しました。でも本物の VLA を作るには、**入力と出力の「整え方」** が成否を分けます。この章で次の 4 つを順に押さえれば、M4 は「部品を組むだけ」になります。

整え方	なぜ要るか
正規化	行動 $[d_x, d_y, \text{grip}]$ は次元ごとにスケールが違う (d_x, d_y は小さな差分、 <code>grip</code> は 0/1)。生のままだと不安定。
行動チャンク	1 ステップずつ出すと推論回数が多く、行動もガタつく。 まとめて 8 ステップ 出すと滑らか。
トークン化	文字列のままではネットに入らない。 ID 列 に変換する。
データ辞書の形	モデル・損失・評価が食い違わないよう、 1 サンプルの形 (shape) を固定する。

4.2 正規化 (normalization)

次元ごとに $(x - \mu)/\sigma$ で標準化し、逆変換 $x \cdot \sigma + \mu$ も持ちます。NumPy / Torch 両対応です。

実装を読む `src/vla_learn/datasets/normalization.py` → `Normalizer`

`fit` が各列の `mean / std` を推定し、`normalize / denormalize` が往復変換。`__init__` の `self.std = np.where(self.std < eps, 1.0, self.std)` が **0 割り防止** の肝 (節 4.2.3)。`normalize / denormalize` は `isinstance(x, torch.Tensor)` で分岐し、Torch 側は `device` まで合わせます。

4.2.1 なぜ「平均 0・分散 1」が嬉しいのか

ニューラルネットの重み初期化や最適化 (Adam など) は、**入力/出力が 0 付近で分散 1 くらい** のときに最も素直に働きます。理由を 3 つだけ。

- **次元間のスケール差を消す**: d_x, d_y (例: ± 0.08) と `grip` (0 or 1) が混在すると、MSE は大きい次元に引っ張られます。標準化すると各次元が同じ土俵に乗ります。
- **勾配が暴れにくい**: 入力が大きいと活性化が飽和したり勾配が発散したりします。
- **学習率を共有できる**: 全次元のスケールが揃うので、1 つの学習率で全部うまく回ります。

4.2.2 `fit / normalize / denormalize` を触る

```
import numpy as np
from vla_learn.datasets import generate_episodes, Normalizer
```

```

eps = generate_episodes(n_episodes=200, seed=0)
actions = np.concatenate([ep["actions"] for ep in eps], axis=0) # [sumT, 3]
print("生の行動 平均:", actions.mean(0), " 標準偏差:", actions.std(0))

norm = Normalizer.fit(actions) # 各列(次元)の mean/std を推定
print("学習した mean:", norm.mean, " std:", norm.std)

a = actions[:3] # 適当に 3 サンプル [3,3]
z = norm.normalize(a) # 正規化 (平均0・分散1の空間へ)
back = norm.denormalize(z) # 逆正規化 (元の空間へ)
print("正規化後:", z)
print("往復誤差(最大):", np.abs(back - a).max()) # ほぼ 0 になるはず

```

出力例 (数値はぶれます) :

```

生の行動 平均: [-0.004  0.004  0.532] 標準偏差: [0.065 0.064 0.499]
学習した mean: [-0.004  0.004  0.532] std: [0.065 0.064 0.499]
正規化後: [[ 1.29 -0.89 -1.07] ...]
往復誤差(最大): 0.0

```

補足

grip 次元は $\mu \approx 0.53, \sigma \approx 0.50$ のように 0/1 の比率を反映します。 d_x, d_y は平均ほぼ 0・標準偏差 0.06 程度。正規化でどの次元も「だいたい $\pm 1 \sim 2$ 」に収まります。往復誤差は float32 なので 0.0~1e-7 程度。数学的には厳密に元へ戻ります。

4.2.3 std=0 の罫 (実装が守ってくれる)

ある次元が常に同じ値だと $\sigma = 0$ で **0 割り** になります。 `Normalizer` は内部で防いでいます:

```

# normalization.py より (抜粋)
self.std = np.where(self.std < eps, 1.0, self.std) # 0 割り防止 (eps=1e-6)

```

4.2.4 【最重要】逆正規化を「推論」で使う

ここが初心者のつまずきポイントです。学習は正規化空間、推論は逆正規化 という分担を図にします:

[学習時]

生の行動 a —normalize—► 正規化行動 z —(教師)—► モデルは z を当てるよう学習
損失 = MSE(モデル出力, z)

[推論時 (閉ループ)]

モデル出力 z_{hat} —denormalize—► 生の行動 a_{hat} —► `env.step(a_hat)` に渡す
↑ これを忘れると、環境は ± 2 みたいな巨大な dx, dy を受け取り破綻する

- **学習** は正規化空間 (平均 0・分散 1) で行います (d_y などの小さな差分を扱いやすくするため)。
- **推論** ではモデル出力を **必ず逆正規化** して、生のワールド座標差分に戻してから環境へ渡します。
- これを忘れると、環境は桁違いの行動を受け取り、即座にクリップされて全く動かない・暴れる、となります。

つまずき / バグの定番

逆正規化忘れ は VLA 実装で最も多いバグの一つです。学習中は損失が綺麗に下がるのに、閉ループだとロボットが微動だにしない／暴れる、という症状が出ます。状態 `state` も同様に「学習入力」

正規化、環境から来た生 state は推論時に正規化してから入れる」が基本です。リポジトリでもこの分担を守り、推論時は PolicyWrapper が逆正規化してから返します (M4 の evaluation/rollout.py)。

4.3 時間方向の窓と行動チャンク (action chunking)

ある時刻 t から `chunk_len` ステップ分の行動 をまとめて取り出します。 `chunk_len` の既定は 8 です。

実装を読む `src/vla_learn/datasets/temporal.py` → `extract_action_chunk`

全文 40 行未満。 `n_valid = min(chunk_len, T - t)` で有効長を決め、 `chunk[:n_valid]` に `actions[t:t+n_valid]` を入れ、足りない分 `chunk[n_valid:]` を最後の行動 `actions[T-1]` で埋め、その位置の `pad_mask` を 0 のままにします (1=有効, 0=パディング)。この `pad_mask` が損失計算 (節 4.6) で効きます。

4.3.1 なぜ「まとめて」予測するのか

1 ステップずつ予測する素朴版 (M2) には弱点がありました。

- **推論回数が多い**: 毎ステップ重いモデルを呼ぶ。実機では遅延になる。
- **行動がガタつく**: 各ステップ独立に予測すると、連続性が保証されず震えやすい。

そこで VLA の多くは「これから `chunk_len` (例 8) ステップ分の行動」を 1 回でまとめて出します。これを **行動チャンク (action chunking)** と呼びます。利点は次の通りです。

利点	中身
推論回数が減る	8 ステップ分を 1 回で出し、しばらくそれを実行できる (receding horizon)。
行動が滑らか	連続する複数ステップを一括設計するので、軌道が自然につながる。
時間的な一貫性	「掴んでから運ぶ」のような短い手順を、ひとまとまりで学べる。

行動チャンク $[B, 8, 3]$: 先頭だけ実行→再観測 (receding horizon)、終端は `pad_mask=0`

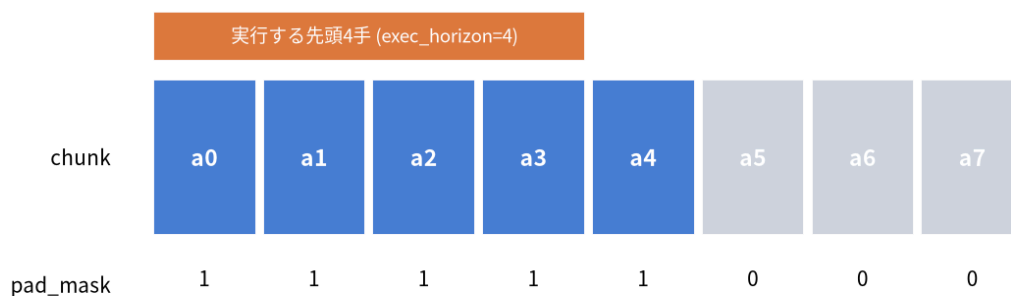


図 6: 行動チャンク $[B, 8, 3]$ 。先頭だけ実行して再観測する receding horizon と、終端の `pad_mask=0`。実装は `src/vla_learn/datasets/temporal.py` の `extract_action_chunk`。

座学とのつながり

ACT (Action Chunking with Transformers)、pi0、SmolVLA はいずれも「ひとまとまりの行動系列」を生成します。本リポジトリの `chunk_len=8` はその最小版です。M5 では、このチャンクを **flow matching** で生成します (= $[B, \text{chunk_len}, \text{action_dim}]$ のサンプルを生成モデルで作る)。

4.3.2 extract_action_chunk の中身と pad_mask

```
# temporal.py (全文に近い抜粋)
def extract_action_chunk(actions, t, chunk_len):
    T, action_dim = actions.shape
    chunk = np.zeros((chunk_len, action_dim), dtype=np.float32)
    pad_mask = np.zeros((chunk_len,), dtype=np.float32)

    n_valid = min(chunk_len, T - t)
    chunk[:n_valid] = actions[t : t + n_valid]
    pad_mask[:n_valid] = 1.0
    if n_valid < chunk_len:
        chunk[n_valid:] = actions[T - 1] # 最後の行動で埋める (静止に近く無害)
    return chunk, pad_mask
```

ポイント:

- 戻り値は chunk: [chunk_len, action_dim] と pad_mask: [chunk_len]。
- エピソード **終端付近** では $t + \text{chunk_len}$ が T を超えます。足りない分は **最後の行動で埋め**、その位置の pad_mask を 0 にします (1 = 有効ステップ、0 = パディング)。
- pad_mask は損失計算で「**パディングは数えない**」ために使います (節 4.6)。

4.3.3 図で見る窓とパディング

chunk_len=4、エピソード長 T=6 の例です。窓を t ごとにスライドさせ、終端で末尾を複製します。

```
actions:  a0 a1 a2 a3 a4 a5          (T=6, 各 a は [dx,dy,grip])

t=0 → chunk=[a0 a1 a2 a3] pad_mask=[1 1 1 1]   (全部有効)
t=1 → chunk=[a1 a2 a3 a4] pad_mask=[1 1 1 1]
t=2 → chunk=[a2 a3 a4 a5] pad_mask=[1 1 1 1]
t=3 → chunk=[a3 a4 a5 a5] pad_mask=[1 1 1 0]   ← 末尾 a5 を複製、最後はパディング
t=4 → chunk=[a4 a5 a5 a5] pad_mask=[1 1 0 0]
t=5 → chunk=[a5 a5 a5 a5] pad_mask=[1 0 0 0]
      └ 有効 ┘ └ padding ┘
```

実際に動かして shape と中身を確認しましょう:

```
import numpy as np
from vla_learn.datasets import extract_action_chunk

actions = np.arange(6 * 3, dtype=np.float32).reshape(6, 3) # [T=6, 3] わかりやすい連番
for t in (0, 3, 5):
    chunk, pad_mask = extract_action_chunk(actions, t, chunk_len=4)
    print(f"t={t}  chunk.shape={chunk.shape}  pad_mask={pad_mask}")
```

出力例:

```
t=0  chunk.shape=(4, 3)  pad_mask=[1. 1. 1. 1.]
t=3  chunk.shape=(4, 3)  pad_mask=[1. 1. 1. 0.]
t=5  chunk.shape=(4, 3)  pad_mask=[1. 0. 0. 0.]
```

4.4 言語のトークン化 (CharTokenizer)

本物の VLA は BERT/Llama のサブワード・トークナイザを使いますが、ここでは **文字を ID に変換するだけ** の最小実装にします (日本語でも空白に依存せず動きます)。

実装を読む src/vla_learn/datasets/tokenizer.py → CharTokenizer

from_corpus が [PAD_TOKEN] + sorted(chars) で語彙を作り (index 0 を PAD に固定)、max_len をコーパス最長に自動設定。encode は長さ max_len 固定の ID 列を返します (不足は PAD、超過は切り詰め)。語彙を **ありうる全指示文** から作るので **未知語 (OOV) が出ない** のがミソです。

要点:

- **PAD = 0** (埋め草)。語彙の index 0 を PAD に固定しています。
- 語彙は **ありうる指示文すべて** から作るので、未知語 (OOV) は出ません (all_instruction_strings() で全パターンを列挙 → from_corpus)。
- encode(text) は **長さ max_len 固定** の ID 列を返します (不足は PAD、超過は切り詰め)。

```
from vla_learn.envs import all_instruction_strings
from vla_learn.datasets import CharTokenizer

corpus = all_instruction_strings()          # ありうる全指示文
tok = CharTokenizer.from_corpus(corpus)      # max_len は corpus 最長に自動設定
print("語彙サイズ vocab_size:", tok.vocab_size)  # 例: 30 (テンプレ・色名でぶれる)
print("max_len:", tok.max_len)              # 例: 17

s = "青のブロックを青のゴールに置いて"
ids = tok.encode(s)
print("文字数:", len(s), " ids 長さ:", len(ids))  # ids 長さは常に max_len
print("PAD は 0 で末尾を埋める:", ids[-5:])
print("復元:", tok.decode(ids))                # PAD(0) を除いて元に戻る
```

出力例 (語彙サイズは色名・テンプレ依存。ぶれます):

```
語彙サイズ vocab_size: 30
max_len: 17
文字数: 16   ids 長さ: 17
PAD は 0 で末尾を埋める: [7, 25, 1, 5, 0]
復元: 青のブロックを青のゴールに置いて
```

補足

max_len=17 = コーパス中で最長の指示文の文字数。上の指示文は 16 文字なので、末尾に PAD が 1 つ付いて 17 になります。もっと短い指示文なら PAD はさらに増えます。vocab_size は使われる文字種の数 + PAD で、テンプレート・色名により変わります。

つまずき / バグの定番

語順の話 (M4 への伏線)。いまの encode は ID の **並び** を保ちます。これを後で「単純な平均プーリング」で潰すと、「赤を青ゴールへ」と「青を赤ゴールへ」が同じベクトルになり、どの色をどこへ運ぶか (grounding) を区別できなくなります。だから M4 の TextEncoder は **位置埋め込み + Transformer** で **語順を区別** します。ここでは「ID 列には順序情報がある」とだけ覚えておけば十分です。

4.5 すべてを束ねる SyntheticVLADataset

ここまでの正規化・チャンク・トークン化と、M2 の「state→image レンダリング」を **1 つの Dataset** にまとめたのが SyntheticVLADataset です。__getitem__ が返す **1 サンプルの dict と shape** を完全に把握しましょう。

実装を読む src/vla_learn/datasets/synthetic_dataset.py → SyntheticVLADataset.__getitem__

`__getitem__` は (episode_idx, t) の平坦インデックスから 1 サンプルを組み立てます: `render_world` で画像を **都度レンダリング**、`state_normalizer.normalize` で状態を正規化、`tokenizer.encode` で言語を ID 化、`extract_action_chunk` → `action_normalizer.normalize` で行動チャンクを正規化。M3 の部品が全部ここで合流します。

```
import torch
from torch.utils.data import DataLoader
from vla_learn.envs import all_instruction_strings
from vla_learn.datasets import (
    generate_episodes, build_normalizers, SyntheticVLADataset, CharTokenizer,
)

eps = generate_episodes(n_episodes=50, seed=0)
tok = CharTokenizer.from_corpus(all_instruction_strings())
action_norm, state_norm = build_normalizers(eps) # 行動・状態の Normalizer を一括作成
ds = SyntheticVLADataset(eps, tok, chunk_len=8,
                        action_normalizer=action_norm,
                        state_normalizer=state_norm)

sample = ds[0]
for k, v in sample.items():
    print(f"{k:10s} shape={tuple(v.shape)} dtype={v.dtype}")
```

出力例:

```
image      shape=(3, 64, 64) dtype=torch.float32
state      shape=(3,)      dtype=torch.float32
tokens     shape=(17,)     dtype=torch.int64
action     shape=(8, 3)    dtype=torch.float32
pad_mask   shape=(8,)      dtype=torch.float32
```

`__getitem__` の各項目 (M0 の観測・行動の定義に対応) :

キー	shape	dtype	中身 / どこから
image	[3, 64, 64]	float32	視覚入力。 <code>render_world</code> で都度レンダリング
state	[3]	float32	$[a_x, a_y, gripper]$ (正規化済み)。 <code>state_normalizer.normalize</code>
tokens	[max_len]	int64	言語指示の文字 ID 列 (右パディング、PAD=0)。 <code>tokenizer.encode</code>
action	[8, 3]	float32	行動チャンク (正規化済み)。 <code>extract_action_chunk</code> → <code>normalize</code>
pad_mask	[8]	float32	1=有効 / 0=パディング。 <code>extract_action_chunk</code>

補足

なぜ image だけ保存せず都度レンダリングするのか。画像を全部ディスクに置くとギガ単位になります。そこで **低次元の状態だけ保存** し、`__getitem__` で state → image を描画します (`save_dataset` / `load_dataset`)。「状態 → 画像」のパイプライン自体も VLA の学びです (M2 の `render_world` と同じ仕組み)。

Tiny Tabletop 2D：観測画像 [3,64,64] と言語指示

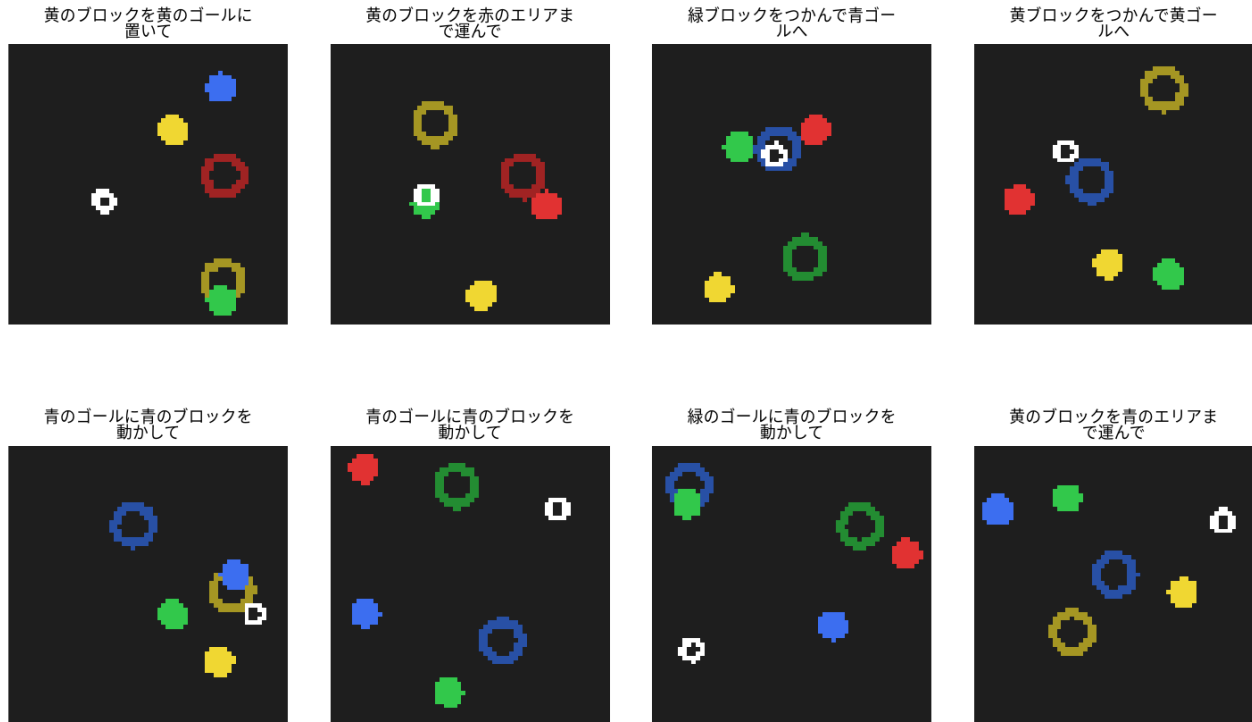


図 7: SyntheticVLADataset が `render_world` で描く観測画像 [3,64,64] と言語指示の例。塗り円がブロック、リングがゴール、白がグリッパ。実装は `src/vla_learn/envs/render.py`。

4.5.1 DataLoader でバッチにすると先頭に B が付く

`DataLoader` は同じ形のサンプルを `batch_size` 個積み、各テンソルの先頭に次元 **B** を足します。これが M4 でモデルに渡る形です。

```
batch = next(iter(DataLoader(ds, batch_size=4, shuffle=True)))
for k, v in batch.items():
    print(f"{k:10s} {tuple(v.shape)}")
```

出力例:

```
image      (4, 3, 64, 64)
state      (4, 3)
tokens     (4, 17)
action     (4, 8, 3)
pad_mask   (4, 8)
```

覚え方: 画像は $[B, C, H, W]$ (チャンネル先頭)、行動チャックは $[B, T, A]$ ($T=\text{chunk_len}=8$, $A=\text{action_dim}=3$)、ベクトル系 (state) は $[B, D]$ 。

4.6 仕上げ: `pad_mask` を使った損失 `masked_mse`

行動チャックの末尾はパディングで埋まっているので、そのまま MSE を取ると **存在しないステップ** まで誤差に数えてしまいます。 `masked_mse` は `pad_mask=0` を除外します。

実装を読む `src/vla_learn/functional.py` → `masked_mse`

全文 20 行強。 `mask.unsqueeze(-1).expand_as(se)` で $[B, C]$ のマスクを $[B, C, A]$ に広げ、 `(se * mask3).sum() / mask3.sum().clamp(min=1.0)` で **有効ステップ数で割る** のが肝。 `clamp(min=1.0)` は「有効ステップが 0 のとき 0 割りしない」保険。 M5 の flow 損失でも同じ `pad_mask` を使います。

```
# functional.py (全文に近い抜粋)
def masked_mse(pred, target, mask=None):    # pred,target: [B,C,A]  mask: [B,C]
    se = (pred - target) ** 2              # [B, C, A]
    if mask is None:
        return se.mean()
    mask3 = mask.unsqueeze(-1).expand_as(se) # [B, C, A] に拡張
    return (se * mask3).sum() / mask3.sum().clamp(min=1.0)
```

「マスクで割る」ところが肝です。有効ステップ数で平均するので、パディングの多寡で損失のスケールが変わりません。実際に確かめます:

```
import torch
from vla_learn.functional import masked_mse

pred = torch.zeros(2, 4, 3)
target = torch.ones(2, 4, 3)
mask = torch.tensor([[1., 1., 1., 0.],    # 4ステップ中3つ有効
                    [1., 1., 0., 0.]])    # 2つ有効
print("マスク無し:", masked_mse(pred, target).item())    # 全要素の平均 = 1.0
print("マスク有り:", masked_mse(pred, target, mask).item()) # 有効分だけ = やはり 1.0
```

出力例:

```
マスク無し: 1.0
マスク有り: 1.0
```

補足

この例はどちらも 1.0 ですが、予測がパディング位置で大きく外れている場合に差が出ます。パディングを数えないことで、「実在するステップの再現」だけを正しく評価できます (演習で確認します)。

4.7 LeRobot 風データ辞書への伏線

この章で作った dict (image / state / tokens / action / pad_mask) は、本リポジトリ内部の形式です。M6 では、これを **LeRobot 風のデータ辞書** へ対応づけます。名前が違っただけで中身は対応します。

本教材の内部 dict	LeRobot 風 (概念対応)	意味
image	observation.image	視覚
state	observation.state	固有受容
tokens / instruction	task	言語指示 (LeRobot は文字列で持つことが多い)
action (chunk)	action	行動 (フレーム列として保存)

LeRobot ではエピソードを **フレーム列** (各時刻の observation / action / task のレコード) として保存します。本教材の map_episode_to_frames (M6 の export_lerobot.py、lerobot 無しでも動く) がこの変換の入口です。いまは「内部 dict と LeRobot 辞書は名前が違っただけで中身は対応する」と覚えておけば十分です。

座学とのつながり

SmolVLA など LeRobot 直結の VLA も、入力は「画像 + 状態 + 言語(task)」、出力は「action」です。本教材の dict はその **最小縮小版**。だから M4/M5 で TinyVLA/FlowVLA を作れば、M6 で SmolVLA を「同じ部品の大規模版」として読めます。

まとめ

- **正規化**: `Normalizer.fit/normalize/denormalize`。学習は正規化空間、**推論は逆正規化して環境へ**（重要）。 $\sigma = 0$ は実装が 0 割り防止。
- **行動チャンク**: `extract_action_chunk(actions, t, chunk_len) -> (chunk[C,A], pad_mask[C])`。終端は最後の行動で **パディング** し `pad_mask=0`。まとめて出す理由＝**推論回数減・滑らかさ・一貫性** (pi0/ACT/SmolVLA)。
- **トークナイザ**: `CharTokenizer`（文字→ID、PAD=0、OOV なし、長さ `max_len` 固定）。ID 列は **語順** を保つ（M4 で Transformer が活かす）。
- **SyntheticVLADataset** の 1 サンプル :
`image[3,64,64] / state[3] / tokens[L] / action[8,3] / pad_mask[8]`。DataLoader で先頭に B が付く。
- **masked_mse** は `pad_mask` でパディングを除外して **有効ステップで平均** する。
- これらは **LeRobot 風辞書**（`observation / action / task`）の最小版で、M6 で対応づける。
- 次章 M4 では、この **SyntheticVLADataset** をそのまま学習データに、画像 + 言語 + 状態 → 行動チャンクを出す **TinyVLA** (MSE 回帰版) をスクラッチで組み、`masked_mse` で学習し、閉ループで成功率を測ります。

最小 VLA を自作する（MSE 回帰版 TinyVLA）

この章のゴール

- 画像 + 言語 + 状態 → 行動チャンク を出す **TinyVLA**（MSE 回帰版）を、3つのエンコーダ + 融合 MLP + 行動ヘッドの **部品を組んで** 自作し、各テンソルの shape を完全に把握する。
- VLA を「動く」ものにするための **3つの設計教訓** を、実装と図で結びつけて理解する: (1) 画像は **flatten** で空間情報を保つ、(2) 言語は **Transformer + 位置埋め込み** で語順を区別する、(3) **FiLM** で言語が視覚を変調する（`condition_vision=False` や `image_pool="avg"` で崩れる）。
- `masked_mse` で学習ループを回し、`scripts/train_mse.py` + `configs/m4_mse.json` で学習、`scripts/eval_policy.py` で **閉ループ評価（成功率）** まで通す。
- 「**損失は下がるのに動かすと失敗**」という現象を、M2 の distribution shift と結びつけて理解する。

ここが本書の山場です。M3 までで **入力と出力の整え方**（正規化・行動チャンク・トークン化）が揃いました。この章では、その `SyntheticVLADataset` をそのまま学習データにして、

$$\text{image} [3, 64, 64] + \text{instruction} + \text{state} [3] \Rightarrow \text{action} [8, 3]$$

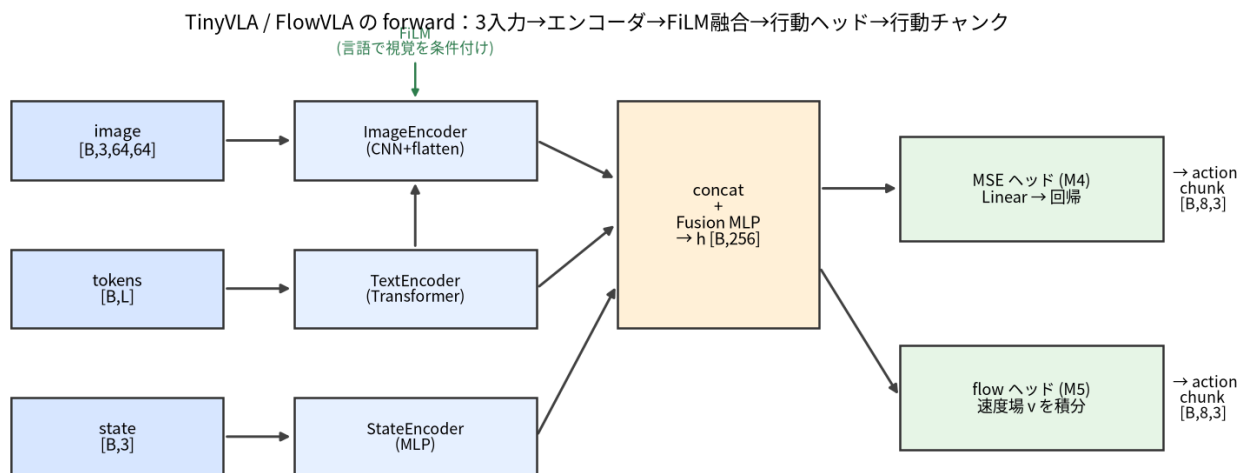
を出す小さな方策 (policy) を **スクラッチで** 組み、学習し、環境で動かして成功率を測ります。本物の VLA（`SmolVLA /  $\pi 0$` など）も骨格は同じ「**3つの感覚を融合して行動を出す**」です。M4 では行動ヘッドを「ただの全結合 + MSE」にします。M5 では、この **ヘッドだけを flow matching に差し替え** ます。

座学とのつながり

既習の VLM は「画像 + 言語 → テキスト」でした。VLA は出力が **行動** に変わるだけで、「複数モダリティを 1 本のベクトルに融合する」発想は同じです。M4 はその最小形です。flow matching として行動生成を作り直すのは M5 ですが、まずは決定論ヘッドで「3 感覚の融合」を完成させます。

5.1 全体像 — VLABackbone（3 エンコーダ + 融合）

TinyVLA は **VLABackbone（条件ベクトル h を作る）** と **head（ $h \rightarrow$ 行動チャンク）** の 2 段構成です。まず鳥瞰図で、3つの入力はどうエンコーダを通り、FiLM で融合され、ヘッドから行動チャンクに化けるかを掴みます。



同じ backbone (青→橙) を共有し、右端のヘッドだけ MSE↔flow で差し替える

図 8: TinyVLA / FlowVLA の forward パイプライン。3 入力 (画像・トークン・状態) を各エンコーダで符号化し、言語ベクトルで視覚を FiLM 変調しながら concat + Fusion MLP で条件ベクトル $h [B, 256]$ を作る。右端のヘッドだけが M4 (MSE 回帰) と M5 (flow) で differ。実装は `src/vla_learn/models/tiny_vla.py`。

各テンソルの shape (B =バッチ、 L =トークン長、 C =`chunk_len`=8、 A =`action_dim`=3) :

段	入力	出力	出力 shape
TextEncoder	tokens $[B, L]$	言語ベクトル l	$[B, 128]$
ImageEncoder (<code>cond=l</code>)	image $[B, 3, 64, 64]$	視覚ベクトル v	$[B, 128]$
StateEncoder	state $[B, 3]$	状態ベクトル s	$[B, 64]$
concat	$[v, l, s]$	融合前ベクトル	$[B, 320]$
Fusion MLP	$[B, 320]$	条件ベクトル h	$[B, 256]$
head	$h [B, 256]$	行動チャンク	$[B, 8, 3]$

融合次元は $320 = 128(\text{画像}) + 128(\text{言語}) + 64(\text{状態})$ です。この足し算は手で言えるようにしておきましょう。

実装を読む `src/vla_learn/models/tiny_vla.py` → `VLABackbone.forward`

3 エンコーダ + 融合の心臓部です。forward の順番が大事で、まず言語 l を作り、それを画像エンコーダに `cond=l` として渡してから視覚 v を得て、最後に `concat([v, l, s])` を融合します。

```

def forward(self, image, state, tokens):
    l = self.text_encoder(tokens)          # [B, txt_dim] ... まず言語を符号化
    v = self.image_encoder(image, cond=l)  # [B, img_dim] ... その言語で視覚を条件付け (FiLM)
    s = self.state_encoder(state)          # [B, state_dim]
    h = torch.cat([v, l, s], dim=-1)       # [B, img+txt+state] = [B, 320]
    return self.fusion(h)                  # [B, hidden] = [B, 256]
  
```

ポイントは言語 l を先に作り、それを画像エンコーダに `cond` として渡すことです。こうして「言語が視覚の見え方を変える」(=後述の FiLM) を実現します。図の上端、ImageEncoder に降りてくる緑の矢印がこの `cond=l` です。

5.1.1 実際に shape を確かめる

部品の出力 shape を、手を動かして確認しましょう。

```

import torch
from vla_learn.models.image_encoder import ImageEncoder
from vla_learn.models.text_encoder import TextEncoder
from vla_learn.models.state_encoder import StateEncoder

B, L, VOCAB = 4, 17, 30
image = torch.rand(B, 3, 64, 64)
state = torch.rand(B, 3)
tokens = torch.randint(0, VOCAB, (B, L))

txt = TextEncoder(vocab_size=VOCAB) # out_dim=128 (既定)
img = ImageEncoder(out_dim=128, cond_dim=128) # FiLM 有効 (cond_dim を渡す)
stt = StateEncoder(out_dim=64)

l = txt(tokens) # 先に言語
v = img(image, cond=l) # その言語で視覚を条件付け
s = stt(state)
print("text :", tuple(l.shape)) # (4, 128)
print("image:", tuple(v.shape)) # (4, 128)
print("state:", tuple(s.shape)) # (4, 64)
print("fused:", l.shape[-1] + v.shape[-1] + s.shape[-1]) # 320

```

5.2 3つの設計教訓 — なぜこの作りなのか

VLAを「とりあえず動くニューラルネット」にするだけなら、画像も言語も雑に潰してしまえます。しかしそれだと**タスクを成功させられません**。この章でいちばん大事なのは、次の3つの「効かせ方」です。いずれも TinyVLA のコンストラクタ引数で**オン/オフを切り替えられ**、消すと成功率がはっきり落ちます。

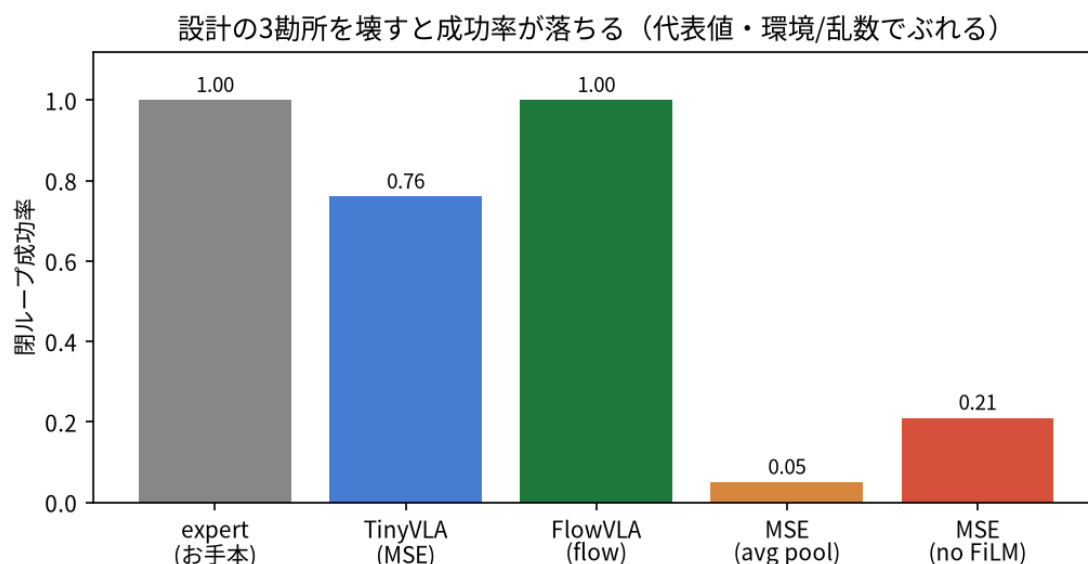


図 9: 3 勘所を壊すと閉ループ成功率が落ちる（代表値・環境/乱数でぶれる）。 `image_pool="avg"`（位置を捨てる）と `condition_vision=False`（FiLMを外す）で大きく低下する。計測は `scripts/eval_policy.py`。

5.2.1 教訓 1: 画像は flatten で「位置」を保つ（avg は位置を捨てる）

このタスクは「指定色のブロックの“場所”へ動く」ものです。だから視覚特徴は「どこに何があるか」を保たねばなりません。ImageEncoder は CNN で $64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$ と空間を縮めた最後の特徴マップ `[B, 64, 4, 4]` を、

- `pool="flatten"` (既定) : そのまま **flatten** して `[B, 64*4*4] = [B, 1024]` にする → **4×4 の空間配置が残る**
- `pool="avg"` (比較版) : GlobalAveragePooling で `[B, 64]` に潰す → **位置が消える** (「何があるか」は残るが「どこか」が消える)

```
# image_encoder.py (抜粋)
if self.pool == "flatten":
    x = x.flatten(1)          # [B, 64*4*4] ... 空間配置を保持
else:
    x = self.gap(x).flatten(1) # [B, 64] ... 位置を平均で消す (比較用)
return self.fc(x)             # [B, out_dim]
```

実装を読む `src/vla_learn/models/image_encoder.py` → `ImageEncoder.forward`

CNN 4 ブロックで空間を 1/16 に縮め、`pool` で `flatten` / `avg` を切り替える本体です。`flatten` 分岐が `[B, 64, 4, 4]` の空間配置を保つ様子と、`avg` 分岐が位置を平均で消す様子を見比べてください。

直感: `avg` は「画面のどこかに赤がある」までしか言えません。`[dx, dy]` (どちらへ動くか) を出すには位置が要るので、`avg` 版は方向の手がかりを失い、成功率が落ちます (前掲の図で `MSE (avg pool)`)。

座学とのつながり

本物の VLA は事前学習済み ViT (SigLIP 等) を使い、パッチごとのトークン列で空間情報を保ちます。ここで `flatten` が果たす役割は、その「**空間を潰さない**」ことの最小版です。

5.2.2 教訓 2: 言語は語順を区別する (平均プーリングの落とし穴 → Transformer + 位置埋め込み)

最も罠にはまりやすいのが言語です。「埋め込みを平均するだけ」にすると、

「赤のブロックを青のゴールに置いて」と「青のブロックを赤のゴールに置いて」

は **同じ文字の集合** なので **同じベクトル** になってしまいます。どちらの色が「運ぶ対象」か区別できず、grounding (言語と対象の対応付け) が原理的に不能になります。そこで `TextEncoder` は **位置埋め込み + 1 層の Transformer エンコーダ** で語順を考慮します:

```
# text_encoder.py (抜粋)
pos = torch.arange(L, device=tokens.device).unsqueeze(0).expand(B, L) # [B, L]
x = self.token_embed(tokens) + self.pos_embed(pos) # 位置情報を足す
pad_mask = tokens == PAD_ID # PAD は無視
x = self.encoder(x, src_key_padding_mask=pad_mask) # 1 層 Transformer
keep = (~pad_mask).float().unsqueeze(-1)
pooled = (x * keep).sum(1) / keep.sum(1).clamp(min=1.0) # PAD を除いた平均
return self.fc(pooled) # [B, out_dim]
```

実装を読む `src/vla_learn/models/text_encoder.py` → `TextEncoder.forward`

位置埋め込みを足してから Transformer に通す本体です。最後に PAD を除いた平均でプーリングしますが、これは **Transformer 通過後** の文脈表現の平均なので、素の埋め込みの平均とは別物 (語順が効く) である点を確認します。

「最後は平均してるじゃないか」と思うかもしれませんが、Transformer を通した後の各位置ベクトルは **周囲の語と位置を織り込んだ文脈表現** なので、語順の違いがちゃんと反映されます。

5.2.3 教訓 3: FiLM で言語が視覚を変調する (`condition_vision=False` だと grounding 崩壊)

3つの感覚を **ただ concat する** だけでは、実は弱いのです。「赤を運べ」と言われたとき、視覚側が **赤を探す目つき** になってほしい。これを実現するのが **FiLM (Feature-wise Linear Modulation)** です。言語ベクトル `cond` から、特徴マップの **チャンネルごとの (scale, shift)** を作って畳み込み特徴を変調します:

```
# image_encoder.py (抜粋)
class FiLM(nn.Module):
    def __init__(self, cond_dim, num_channels):
        super().__init__()
        self.to_scale_shift = nn.Linear(cond_dim, 2 * num_channels)

    def forward(self, x, cond): # x:[B,C,H,W], cond:[B,cond_dim]
        gamma, beta = self.to_scale_shift(cond).chunk(2, dim=-1) # [B,C],[B,C]
        return x * (1 + gamma[:, :, None, None]) + beta[:, :, None, None]
```

`ImageEncoder` は中間2箇所 (`[B,32,16,16]` と `[B,64,8,8]` の後) に FiLM を挿し、`forward(image, cond=l)` のときだけ効かせます。`VLABackbone` の `condition_vision` でオン/オフします:

```
cond_dim = txt_dim if condition_vision else None
self.image_encoder = ImageEncoder(out_dim=img_dim, pool=image_pool, cond_dim=cond_dim)
```

- `condition_vision=True` (既定): 言語が視覚を変調 → 「名指しされた色」に視覚を向けられる。
- `condition_vision=False` (比較版): `cond_dim=None` になり FiLM を作らないため、**画像エンコーダは言語をまったく見ません**。言語は concat 時の `l` としてしか入らず、**対象選択 (どの色を運ぶか) の steering がほぼ効かなくなり、grounding が崩壊** します。

つまずき / バグの定番

開発時、FiLM を外すと「**言語で対象を選べる率がほぼ 0**」になった一方、FiLM を入れると同じ画像でも指示の色を切り替えると対象が切り替わるようになりました。「concat してあるから言語は届いているはず」は通用しません。**どこで効かせるか** (早い段階で視覚を変調する) が grounding を左右します。前掲の図の `MSE (no FiLM)` がこの崩壊です。

5.2.4 3 つまとめ

教訓	引数	既定 (良い)	比較版 (悪化)	失われるもの
空間情報	<code>image_pool</code>	"flatten"	"avg"	「どこに」あるか (方向が出せない)
語順	(<code>TextEncoder</code> 内蔵)	Transformer+位置埋め込み	平均プーリング	「赤→青」と「青→赤」の区別
言語条件付け	<code>condition_vision</code>	<code>True</code> (FiLM)	<code>False</code>	言語による対象選択 (grounding)

これら無しだと **損失は下がっても成功率が落ちます**。だから3つとも「効かせる」のが既定値なのです。

5.3 TinyVLA 本体 — head と forward

`VLABackbone` が条件ベクトル `h [B,256]` を作ったら、あとは行動チャンクに変換するだけです:

```

class TinyVLA(nn.Module):
    def __init__(self, vocab_size, chunk_len=8, action_dim=3, hidden=256, **backbone_kwargs):
        super().__init__()
        self.backbone = VLABackbone(vocab_size, hidden=hidden, **backbone_kwargs)
        self.chunk_len = chunk_len
        self.action_dim = action_dim
        self.head = nn.Linear(hidden, chunk_len * action_dim)  # 256 → 8*3 = 24

    def forward(self, image, state, tokens):
        h = self.backbone(image, state, tokens)  # [B, 256]
        out = self.head(h)  # [B, 24]
        return out.view(-1, self.chunk_len, self.action_dim)  # [B, 8, 3]

```

head は $256 \rightarrow \text{chunk_len} \times \text{action_dim} = 24$ の **ただの全結合** で、view で `[B, 8, 3]` に整形します。これが「決定論的に行動チャンクを 1 つ出す」MSE 版の正体です。`**backbone_kwargs` で `image_pool` や `condition_vision` を素通しできる点に注目（教訓の実験で使います）。

5.3.1 forward と parameter 数を確認

```

import torch
from vla_learn.models import TinyVLA, count_parameters
from vla_learn.datasets import CharTokenizer
from vla_learn.envs import all_instruction_strings

tok = CharTokenizer.from_corpus(all_instruction_strings())
model = TinyVLA(vocab_size=tok.vocab_size, chunk_len=8)
print("vocab_size =", tok.vocab_size)
print("params      =", f"{count_parameters(model):,}")

B = 4
image = torch.rand(B, 3, 64, 64)
state = torch.rand(B, 3)
tokens = torch.randint(0, tok.vocab_size, (B, 17))
out = model(image, state, tokens)
print("forward out:", tuple(out.shape))  # (4, 8, 3)

```

出力例（パラメータ数は語彙サイズ等でぶれます）：

```

vocab_size = 30
params      = 422,168
forward out: (4, 8, 3)

```

補足

規模感: 約 **0.42M パラメータ** の「Tiny」VLA です。CPU で数分学習でき、本物の VLA（数億～数十億）の **同じ部品の縮小版** として全工程を体験できます。参考までに `image_pool="avg"` だと約 0.30M (299,288)、`condition_vision=False` だと約 0.40M (397,400) になります（位置や FiLM の重みが減るため）。これらの値は環境でぶれます。

5.4 学習ループ — masked_mse で教師あり回帰

5.4.1 損失: なぜ masked_mse か

教師は「正規化済みの行動チャンク」`action [B,8,3]` です。TinyVLA の出力 `pred [B,8,3]` をこれに近づけます。ただし M3 で見たとおり、チャンク末尾は **パディング** されていることがあるので、`masked_mse` で `pad_mask=0` のステップを損失から外します。

```
# functional.py (全文に近い抜粋)
def masked_mse(pred, target, mask=None):    # pred,target: [B,C,A]  mask: [B,C]
    se = (pred - target) ** 2              # [B, C, A]
    if mask is None:
        return se.mean()
    mask3 = mask.unsqueeze(-1).expand_as(se) # [B, C, A] に拡張
    return (se * mask3).sum() / mask3.sum().clamp(min=1.0) # 有効ステップで平均
```

実装を読む src/vla_learn/functional.py → masked_mse

本書の損失の中核です。pad_mask を [B,C,A] に広げて掛け、有効ステップ数で割るだけ。学習コードでは from vla_learn.training.losses import masked_mse で使いますが、中身はこの functional のものの再公開です。M5 の flow 損失も、最後はこの masked_mse を**速度**に対して呼ぶだけになります。

5.4.2 学習ループの中核

学習ループ本体は trainer.py の run_training です。心臓部だけ抜き出すと、M1 で習った標準形そのものです：

```
import torch
from torch.utils.data import DataLoader
from vla_learn.datasets import (
    generate_episodes, build_normalizers, SyntheticVLADataset, CharTokenizer,
)
from vla_learn.envs import all_instruction_strings
from vla_learn.models import TinyVLA
from vla_learn.training.losses import masked_mse
from vla_learn.utils import set_seed, get_device

set_seed(0)
device = get_device() # CPU/GPU 自動

# --- データ (M2/M3 の部品をそのまま) ---
episodes = generate_episodes(n_episodes=200, seed=0, action_noise=0.03)
tok = CharTokenizer.from_corpus(all_instruction_strings())
action_norm, state_norm = build_normalizers(episodes)
ds = SyntheticVLADataset(episodes, tok, chunk_len=8,
                        action_normalizer=action_norm, state_normalizer=state_norm)
loader = DataLoader(ds, batch_size=64, shuffle=True)

# --- モデル・最適化 ---
model = TinyVLA(vocab_size=tok.vocab_size, chunk_len=8).to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

# --- 学習ループ ---
model.train() # 学習モード (重要)
for epoch in range(5): # デモ用に 5 epoch
    running, nb = 0.0, 0
    for batch in loader:
        batch = {k: v.to(device) for k, v in batch.items()} # device を揃える (重要)
        pred = model(batch["image"], batch["state"], batch["tokens"]) # [B,8,3]
        loss = masked_mse(pred, batch["action"], batch["pad_mask"])
        opt.zero_grad(); loss.backward(); opt.step()
        running += loss.item(); nb += 1
    print(f"epoch {epoch}  loss={running/nb:.5f}")
```

実装を読む src/vla_learn/training/trainer.py → run_training

データ生成 → トークナイザ/正規化 → Dataset/DataLoader → モデル → 最適化ループ → 保存 → 閉ループ評価まで、VLA学習の全工程が1ファイルに収まっています。M5との違いは `_compute_loss` とモデル構築の **2箇所だけ** で、ループ本体は共通です。

出力例（数値はぶれます。下がってればOK）：

```
epoch 0  loss=0.61432
epoch 1  loss=0.18044
epoch 2  loss=0.10231
epoch 3  loss=0.07515
epoch 4  loss=0.06120
```

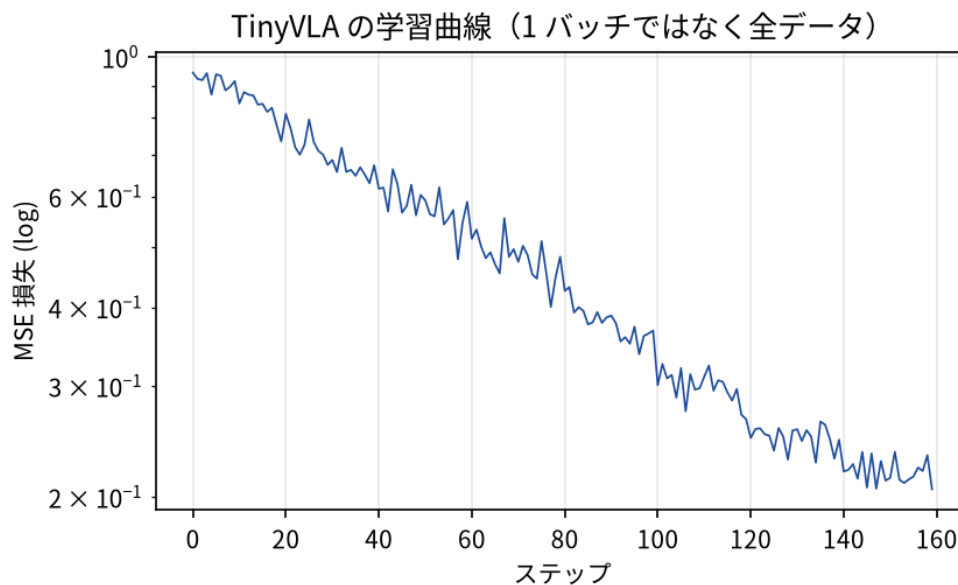


図 10: 全データでの学習曲線（縦軸 log）。MSE は単調に近く下がる。M5 の flow 損失は 毎ステップ乱数を引くため、これほど滑らかには下がらない（移動平均で見ると）。生成は `scripts/make_figures.py`。

つまずき / バグの定番

初心者がつまずく 3 点（演習のバグ修正でも問います）：

- **device を揃える**: batch のテンソルをモデルと同じ device に `.to(device)` する。忘れると `cpu` と `cuda` の不一致で落ちます。
- **.train() / .eval()**: 学習中は `model.train()`。推論・評価では `model.eval()`（このモデルに Dropout/BN は無いものの、習慣として徹底）。
- **正規化**: 教師 action は **正規化済み**。推論時は逆正規化して環境へ（後述）。

補足

M5 との関係: この学習ループで **MSE と flow の違いは「モデル構築」と「損失計算」の 2 箇所だけ** です。trainer.py の `_compute_loss` は `model_type=="flow"` のとき `model.flow_loss(...)` を呼び、そうでなければ `masked_mse(...)` を使います。いまは「ヘッドと損失を差し替えるだけで枠組みは共通」と覚えておけば十分です。

5.5 スクリプトで学習する — train_mse.py + configs/m4_mse.json

実運用では Python を直書きせず、設定ファイルとスクリプトで回します。スモークテスト（ごく小規模で配線確認）：

```
uv run python scripts/train_mse.py --config configs/smoke.json
```

本番設定（configs/m4_mse.json）。中身は次のとおりです：

```
{
  "model_type": "mse",
  "n_episodes": 1500,
  "n_objects": 3,
  "n_goals": 2,
  "chunk_len": 8,
  "epochs": 30,
  "batch_size": 128,
  "lr": 0.001,
  "eval_episodes": 100,
  "exec_horizon": 4,
  "seed": 0,
  "out_dir": "checkpoints/mse"
}
```

```
uv run python scripts/train_mse.py --config configs/m4_mse.json
```

```
# 一部だけ上書きしたいとき（CLI 引数が config より優先）
```

```
uv run python scripts/train_mse.py --config configs/m4_mse.json --epochs 30 --n-episodes 1500
```

run_training は最後に **学習済み方策を保存し、そのまま閉ループ評価** まで行います。出力例（数値はぶれます）：

```
[setup] device=cpu model_type=mse
[data] 1500 episodes 生成 (action_noise=0.03)
[data] 12030 学習サンプル / vocab=30
[model] mse | パラメータ数 = 422,168
[train] epoch 0 loss=0.70218
[train] epoch 10 loss=0.09531
[train] epoch 29 loss=0.05604
[save] checkpoints/mse/policy.pt
[eval] success_rate=0.760 final_dist=0.160 steps=27.4
```

補足

実測の目安（CPU 計測。n_objects=3, n_goals=2, action_noise=0.03, 1500ep×30epoch, exec_horizon=4）：学習 loss はおよそ 0.7 → 0.056 まで下がり、**閉ループ成功率はおよそ 7～8 割**（success_rate≈0.76）、final_distance≈0.16 でした。ただし **環境・乱数で数ポイントはぶれます**。success_rate が 0.7 前後～0.8 台に入っていれば想定とおりです（解析エキスパートは 100% 成功なので、その下にギャップがあるのは正常です）。

5.6 閉ループ評価 — 損失ではなく成功率を見る

「損失が下がった」は **お手本との一致度** にすぎません。本当に知りたいのは「環境で動かしてタスクを成功できるか」です。rollout.py がこれを担います。

5.6.1 PolicyWrapper — obs から行動チャンクへ（正規化・device 込み）

PolicyWrapper は学習済みモデルを「obs → 行動チャンク（生の値）」に変換します。要点は **state** を正規化して入れ、出力を逆正規化して返す ことです：

```
# rollout.py (PolicyWrapper.predict_chunk 抜粋)
state_np = self.state_norm.normalize(obs["state"].astype(np.float32)) # 入力 state は正規化
...
a = self.model(img, state, tokens) # [1,C,3] (正規化空間)
a = self.action_norm.denormalize(a)[0].cpu().numpy() # 生の行動に戻して返す
```

実装を読む src/vla_learn/evaluation/rollout.py → PolicyWrapper.predict_chunk

M3 の最重要点の実物です。**逆正規化を忘れると、環境は ± 2 のような巨大な dx, dy を受け取り破綻します。** 学習は正規化空間、環境とのやり取りは生の空間、という分担をここが守ります。M5 ではこの中の 1 行が model.sample(...) に変わるだけ（後述）。

5.6.2 receding horizon と exec_horizon

行動チャンクは「**chunk を予測** → **先頭 exec_horizon ステップだけ実行** → **観測し直して再予測**」を繰り返して使います（receding horizon, 後退ホライズン）：

```
# rollout.py (rollout_episode 抜粋)
while not done:
    chunk = policy.predict_chunk(obs) # [C, 3]
    for k in range(min(exec_horizon, chunk.shape[0])):
        obs, _, done, info = env.step(chunk[k]) # 先頭 exec_horizon ステップだけ実行
        if done:
            break
```

exec_horizon=4 なら「8 ステップ予測して 4 ステップ実行し、また観測し直す」です。**全 8 ステップを実行しきらないのがコツ**（理由は次節）。

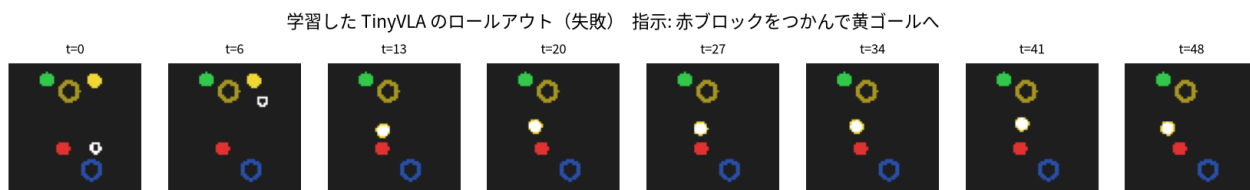


図 11: 学習した TinyVLA のロールアウト連続フレーム。白がグリッパ。指示色のブロックを指示色のゴールへ運ぶ（この例は失敗ケース）。success_rate は多数エピソードの平均。実装は src/vla_learn/evaluation/rollout.py。

5.6.3 評価を走らせる

学習で保存した checkpoints/mse/policy.pt を読み込んで評価します：

```
uv run python scripts/eval_policy.py --ckpt checkpoints/mse/policy.pt --n-episodes 100
```

出力例（数値はぶれます）：

```
==== 評価結果 ====
success_rate: 0.76
mean_final_distance: 0.16
mean_steps: 27.4
n_episodes: 100
```

success_rate が「指定色のブロックを指定色のゴールへ運べた割合」です。**7〜8 割**あたりが目安です。

5.7 「損失は下がるのに動かすと失敗」 — distribution shift の再来

ここは VLA でいちばん大事な落とし穴です。学習 loss が 0.056 まで下がっても、成功率は 100% にはなりません。なぜか。

M2 で見た **distribution shift** (分布シフト) が原因です。学習データは **エキスパートが通った軌道** だけ。ところが本番では、自分の予測がわずかにズレ、**お手本が一度も通らなかった状態** に入ります。そこでの行動は学習していないのでさらにズレ、誤差が積み重なって失敗します。masked_mse は「各ステップの平均誤差」を測るだけで、**この連鎖的なズレ (閉ループでの破綻) は測れません**。だから「loss は低いのに失敗」が起きます。

この教材が打っている **2 つの緩和策** を、引数と結びつけて押さえましょう：

- **action_noise** (既定 0.03) : データ生成時にエキスパート行動へ小さなノイズを混ぜます (M2 の DAGger 風の発想)。すると「少しズレた状態」も訓練分布に入り、本番で軌道を外しても **復帰** しやすくなります。0.0 にすると見た目の loss は下がりやすいのに、閉ループは脆くなりがちです。
- **exec_horizon** (既定 4) : 8 ステップ全部を実行しきらず、**4 ステップで観測し直す** ことで、古い観測に基づく行動の積み重ねを断ち、ズレを早めにリセットします。大きくする (例 8) ほど再観測が減って distribution shift に弱くなり、小さすぎる (例 1) と再観測は増えますが推論回数が増え、チャンクの滑らかさの利点が薄れます。

座学とのつながり

MSE の低さ ≠ タスク成功。閉ループ成功率こそが VLA の通信簿です。M5 の flow matching は、行動分布の **多峰性** (同じ状況で複数の正解経路がある場合) を扱える点で、この決定論 MSE 版より頑健になり得ます。まずは「MSE 版で 7~8 割」を体で覚えるのが目的です。

5.8 章末チェック — 1 バッチに過学習できるか

学習ループ・shape・損失・最適化のどれかにバグがあると、**そもそも学習が回っていません**。これを最速で炙り出すのが「**小さな 1 バッチに過学習できるか**」テストです (機械学習デバッグの鉄則)。正しければ loss は限りなく 0 に近づきます。

```
import torch
from torch.utils.data import DataLoader
from vla_learn.datasets import (
    generate_episodes, build_normalizers, SyntheticVLADataset, CharTokenizer,
)
from vla_learn.envs import all_instruction_strings
from vla_learn.models import TinyVLA
from vla_learn.training.losses import masked_mse
from vla_learn.utils import set_seed

set_seed(0)
eps = generate_episodes(n_episodes=8, seed=0)
tok = CharTokenizer.from_corpus(all_instruction_strings())
an, sn = build_normalizers(eps)
ds = SyntheticVLADataset(eps, tok, 8, an, sn)
batch = next(iter(DataLoader(ds, batch_size=16, shuffle=True))) # 固定の 1 バッチ

model = TinyVLA(vocab_size=tok.vocab_size, chunk_len=8)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
first = None
for i in range(200):
    pred = model(batch["image"], batch["state"], batch["tokens"])
```

```

loss = masked_mse(pred, batch["action"], batch["pad_mask"])
opt.zero_grad(); loss.backward(); opt.step()
if first is None:
    first = loss.item()
print(f"first={first:.4f} last={loss.item():.4f} (last は first の 1/5 未満が目安)")

```

出力例（数値はぶれます）：

```

first=0.6149 last=0.0087 (last は first の 1/5 未満が目安)

```

補足

これはリポジトリの `tests/test_overfit_tiny_batch.py`（`test_mse_overfits_one_batch`）と同じ趣旨です。学習ループを自作したら、まずこの確認をしてから本番データに進むのが安全です。`uv run pytest -k overfit` でも走らせられます。なお `run_training` には `overfit_one_batch=True` という設定もあり、1 バッチだけを繰り返し学習するモードを `TrainConfig` から使えます。

まとめ

- **TinyVLA** = **VLABackbone**（3 エンコーダ + 融合 MLP）+ **head**（全結合）。`forward(image,state,tokens) → [B,8,3]`。融合は `concat([v,l,s]) = [B,320] → Fusion → h[B,256] → head → [B,8,3]`。約 0.42M パラメータ。
- **3つの設計教訓**: (1) 画像は `image_pool="flatten"` で空間情報を保持、(2) 言語は Transformer + 位置埋め込みで語順を区別、(3) FiLM（`condition_vision=True`）で言語が視覚を変調。いずれも消すと損失は下がっても成功率が落ちる。
- **学習**: `masked_mse(pred, action, pad_mask)` を Adam で最小化。device を揃える / `.train()` `.eval()` / 正規化を忘れない。
- **スクリプト**: `train_mse.py --config configs/m4_mse.json` で学習 + 自動評価。実測の目安は成功率およそ 7~8 割（ぶれる）。
- **閉ループ評価**: `PolicyWrapper`（state 正規化・行動逆正規化・device）+ `receding horizon`（`exec_horizon=4`）。**損失の低さ ≠ 成功**。
- **distribution shift**: 「loss は低いのに失敗」は M2 の分布シフトの再来。`action_noise` と `exec_horizon` で緩和する。
- **デバッグ**: 何よりもまず **1 バッチに過学習** できるか確認する。

次章 M5 では、この TinyVLA の **行動ヘッドだけ** を flow matching ヘッドに差し替えて FlowVLA を作ります。VLABackbone も学習ループの骨格も **そのまま流用** し、変わるのは「ヘッドと損失」だけです。

flow matching 化 (MSE ヘッド → rectified flow ヘッド)

この章のゴール

- M4 の TinyVLA (MSE 回帰版) の **行動ヘッドだけ** を rectified flow (= linear flow) に差し替え、FlowVLA を 自作・学習・評価する。
- **なぜ MSE では足りないのか** (多峰性 (multimodality) を平均で潰す問題) を具体例で腹落ちさせる (本タスク自体は、あえて多峰性が出ないように単純化してある点も明示します)。
- 既習の **拡散 / フローの座学** (スコア・ノイズ除去・確率フロー) を、**実装の数行に回収** する。
- **時刻埋め込み (SinusoidalTimeEmbedding)** と **速度ネット (velocity net)** の役割・入出力 shape を押さえる。
- 推論の **Euler 積分** ($\tau=0 \rightarrow 1$, flow_steps) を実装し、flow_steps を変えると何がかわるかを実験する。

補足

この章は diffusion / flow の **座学 (数式・概念) は理解済み** という前提で書きます。概念の再講義はしません。「まず動かす→理解」の方針どおり、座学の各概念が flow_head.py のどの行に化けるかを 1 対 1 で対応させます。

6.1 この章で「足す」のは 1 点だけ

M4 で作った VLA は、こうでした:

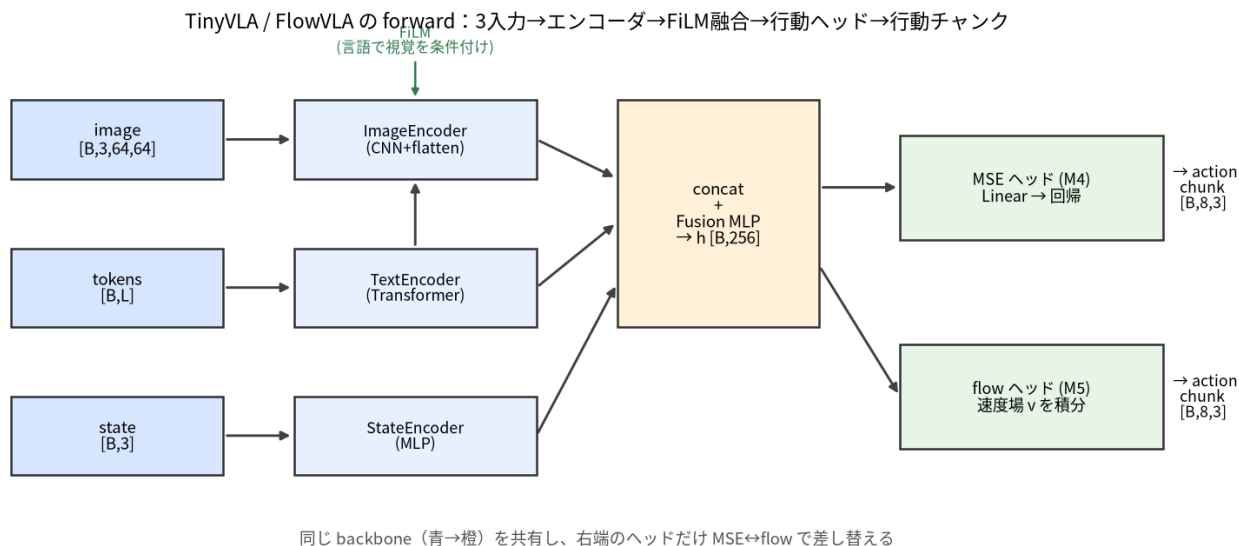


図 12: M4 と共通の forward。M5 で変えるのは右端の **行動ヘッド** だけ。左側のエンコーダ (FiLM 付き VLABackbone) は一字も変えずそのまま共有する。実装は src/vla_learn/models/flow_head.py。

M5 で変えるのは **右端の行動ヘッド** だけです。左側のエンコーダ (FiLM 付き VLABackbone) は **一字も変えずそのまま共有** します。

	行動ヘッド	学習の損失	エン コーダ (VLABackbone)
M4 TinyVLA	<code>nn.Linear(hidden, C*A)</code> (決定論的に 1 点を出す)	<code>masked_mse(pred, action)</code>	共有 (FiLM・Transformer・ flatten)
M5 FlowVLA	速度ネット $v(a_\tau, \tau, h)$	flow matching loss	同じものを共有

なぜこの設計が嬉しいか。「観測をどう理解するか (perception)」と「行動をどう生成するか (action head)」を分離しているからです。grounding (言語で対象を選ぶ) の作り込みは M4 で済んでいて、M5 はその上に「平均で潰さない行動生成」を載せ替えるだけで得られます。これはおもちゃの話ではなく、 π_0 / SmolVLA が現に取っている構造です。

座学とのつながり

既習の座学との対応 (先に結論) : M4 の MSE は「条件付き平均 $\mathbb{E}[a|h]$ を点推定する」回帰。M5 の flow matching は「条件付き分布 $p(a|h)$ からサンプリングできる生成モデル」を作ります。拡散の「ノイズから画像を生成」を、ここでは「ノイズから行動チャンクを生成」に使うだけです。

補足

「ヘッドだけ差し替え」と言っても、FlowVLA には時刻埋め込みと速度ネット `vnet` (3 層 MLP) が新たに加わるため、パラメータ数は TinyVLA の約 0.42M (422,168) → 約 0.58M (576,280) に増えます (エンコーダ部は共有)。値は環境でぶれます。

6.2 なぜ MSE では不十分か (多峰性を平均で潰す)

6.2.1 多峰性とは (なぜ MSE は平均で潰れるのか)

`masked_mse` は出力を教師の平均に近づけます。教師 (エキスパート) の行動が 1 つの局面に対して複数あり得る (= 多峰) とき、平均は「どれでもない中間」になります。一般のロボットタスクでは、この多峰性が頻繁に起きます。

例えば「エージェントが障害物を避けてゴールへ向かう」局面を考えます。左から回っても右から回っても到達できる = 正解の行動が 2 つ (2 峰) あります。データに「左に回る軌跡」と「右に回る軌跡」の両方が (別エピソードとして) 入ると、ある瞬間の状態 h に対し教師行動は $dx < 0$ (左) と $dx > 0$ (右) の 2 つの峰を持ちます。MSE は両者を 1 点で当てようとして、その平均 $dx \approx 0$ (まっすぐ突っ込む) を学びます。これはどちらの正解でもなく、障害に当たる最悪手です。「掴むか / もう一步寄ってから掴むか」「複数の対象のどれを先に運ぶか」でも同じ多峰が起きます。 π_0 / SmolVLA など実用 VLA が flow / diffusion を使う最大の理由がこれです。

つまずき / バグの定番

ただし本教材のタスクは、あえて多峰性が出ないよう単純化してあります。現行の `Tabletop2DEnv` には障害物も衝突判定もなく、エキスパート (`expert.py`) は対象へまっすぐ近づく決定論方策 (経路の分岐も障害物回避もなく、各軸を `MAX_STEP` でクリップしながら対象 → ゴールへ詰める) です。物体の色も重複しません (指示は一意)。つまり、ある状態 h に対する正解行動はほぼ 1 つ (単峰)。だからこそ M4 の MSE 版でも 7~8 割動きました (失敗の主因は多峰性ではなく、M2 で見た分布シフトです)。

ではなぜここで flow を学ぶのか。狙いは2つです。

1. **多峰を扱える行動ヘッドの作り方** を、最小実装で体得する（座学の flow matching を自分の手で動かす）。
2. 同じ **backbone** のまま **MSE ヘッド** を生成ヘッドに差し替えても性能が落ちない（むしろ閉ループがやや安定する）ことを確認する。

flow の真価がはっきり出るのは **多峰タスク** で、本タスクはそこへ進む前の安全な足場です。

6.2.2 数式で 1 行

回帰の最適解は条件付き平均です:

$$\arg \min_f \mathbb{E}[\|f(h) - a\|^2] = f^*(h) = \mathbb{E}[a | h] \quad (\text{平均、多峰だと谷の底})$$

flow matching が学ぶのは平均ではなく、 $p(a|h)$ から実際にサンプルを引く手続きです。だから2峰なら、ある推論では左の峰、別の推論では右の峰、と **片方の正解** を選べます。

座学とのつながり

座学の言い換え: MSE は L2 回帰 = 「ガウス1個でフィット」。多峰分布をガウス1個で表すと平均に山が立ち、真の峰が消える。flow / diffusion は分布そのものを表現できる、という既習の話を、ここでは行動 a について行います。

6.3 rectified flow の実装対応（座学 → コード）

ここからが本題です。座学で知っている **rectified flow** (= linear / conditional flow matching) が、`flow_head.py` のどの行に化けるかを、1対1で対応させます。

6.3.1 まっすぐな道とその速度

ノイズ $a_0 \sim N(0, I)$ から目標行動 a_1 （正規化済みの教師チャンク）へ、**直線で結ぶ** 道を引きます:

$$\text{経路: } a_\tau = (1 - \tau) \cdot a_0 + \tau \cdot a_1 \quad (\tau: 0 \rightarrow 1)$$

$$\text{速度: } v^* = \frac{da_\tau}{d\tau} = a_1 - a_0 \quad (\tau \text{ によらず一定})$$

$\tau = 0$ でノイズ a_0 、 $\tau = 1$ で目標 a_1 。直線なので速度は **始点と終点だけで決まる定数** $a_1 - a_0$ です。ネットワークの仕事は「いま a_τ にいて時刻が τ 、条件が h のとき、進むべき速度 v を答える」こと:

$$\text{損失: } L = \|v_{\text{pred}}(a_\tau, \tau, h) - (a_1 - a_0)\|^2 \quad (\text{pad は masked_mse で除外})$$

この直線経路と一定速度を図で見てください。

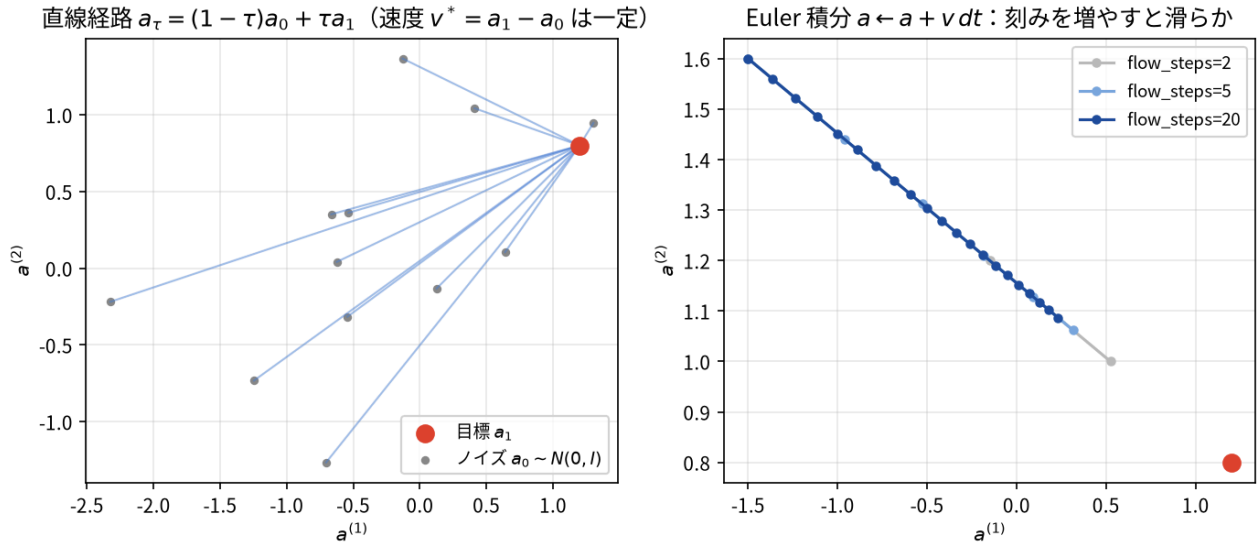


図 13: 左: rectified flow の直線経路 $a_\tau = (1 - \tau)a_0 + \tau a_1$ 。多数のノイズ a_0 から目標 a_1 へ各々まっすぐ向かう (速度 $v^* = a_1 - a_0$ は経路上一定)。右: 学習後に Euler 積分でノイズ 1 点から目標へ歩く様子。flow_steps を増やすと刻みが細くなる。生成は scripts/make_figures.py。

6.3.2 flow_loss の数行と座学の対応

実装 (FlowVLA.flow_loss) はこれだけです:

```
def flow_loss(self, image, state, tokens, action, pad_mask=None):
    h = self.encode(image, state, tokens) # [B, hidden] ← M4 と同じ backbone
    a1 = action # [B, C, A] 正規化済みの目標チャック
    a0 = torch.randn_like(a1) # [B, C, A] ノイズ (標準正規)
    tau = torch.rand(a1.shape[0], device=a1.device) # [B] ~ U(0, 1)
    a_tau = (1 - tau)[:, None, None] * a0 + tau[:, None, None] * a1 # [B, C, A] 経路上の点
    v_pred = self.velocity(a_tau, tau, h) # [B, C, A] ネットの予測速度
    v_target = a1 - a0 # [B, C, A] 真の速度 (一定)
    return masked_mse(v_pred, v_target, pad_mask) # スカラ
```

実装を読む src/vla_learn/models/flow_head.py → FlowVLA.flow_loss

この章の心臓部です。encode は M4 と同じ VLABackbone、最後の masked_mse も M3/M4 と同じ関数。新しいのは「ノイズ a_0 を引き、直線上の点 a_τ を作り、真の速度 $a_1 - a_0$ を当てる」3 行だけ、という最小性を確認します。

座学の各概念がどの行か:

座学の概念	コード	補足
ノイズ分布 $p_0 = N(0, I)$	<code>a0 = torch.randn_like(a1)</code>	行動チャックと同じ shape のノイズ
データ分布 p_1 (目標)	<code>a1 = action</code>	正規化済み教師 (M3)
時刻サンプル $\tau \sim U(0, 1)$	<code>tau = torch.rand(B)</code>	バッチ内の各サンプルに別々の τ
補間 (確率パス) a_τ	<code>a_tau = (1 - tau) * a0 + tau * a1</code>	直線 パス (= rectified)
真の速度場 $v^* = a_1 - a_0$	<code>v_target = a1 - a0</code>	直線なので閉形式・一定
速度のマッチング損失	<code>masked_mse(v_pred, v_target, pad_mask)</code>	L2 で速度を回す

座学とのつながり

拡散座学（スコア / ノイズ除去）との対応: 拡散モデルでは「ノイズを加えた x_t から、加えたノイズ ε （またはスコア $\nabla \log p$ ）を予測」しました。rectified flow は、その「ノイズ予測」を「**確率フロー ODE の速度場 v 予測**」に置き換えた形です。直線パスを使うと速度は $a_1 - a_0$ という定数になり、拡散の複雑なノイズスケジュールも、スコアから サンプラへの変換も要りません。 $v = a_1 - a_0$ を **L2 で当てるだけ** で生成器が手に入る——これが「rectified（まっすぐにした）flow」が実装上うれしい理由で、座学の確率フロー ODE の最小実装そのものです。

6.3.3 [:, None, None] が要る理由（つまずきの定番）

`tau` は `[B]`、`a0 / a1` は `[B, C, A]`（3次元）です。`tau * a0` を素直に書くと**形が合わず**ブロードキャストに失敗します（または意図しない次元で勝手に合ってしまう）。`tau[:, None, None]` で `[B, 1, 1]` に整形すると、`C` と `A` の両方向へ正しく放送され、**サンプルごとに1つの τ がチャック全体へ効きます**。

```
tau          # [B]
tau[:, None] # [B, 1]
tau[:, None, None] # [B, 1, 1] ← これで [B, C, A] と掛けられる
```

つまずき / バグの定番

これは演習 M5 の「バグ修正」に出します。`[:, None, None]` を忘れると、形エラーか、バッチとチャックが混線した無意味な a_τ になります。`tau` のような「サンプルごとに1つのスカラー」を高次元テンソルへ掛けるときは、末尾に `None`（= `unsqueeze`）を足して軸数を合わせる、という作法を覚えておきましょう。

6.4 時刻埋め込みと速度ネット（shape を完全に追う）

6.4.1 SinusoidalTimeEmbedding：連続時刻をベクトルに

速度ネットは MLP（全結合）です。MLP に「 $\tau = 0.3$ 」という生のスカラー1個を入れても、時刻の情報をうまく使えません。そこで τ を **高次元ベクトル** に展開してから渡します。発想は Transformer の位置符号（M4 の `TextEncoder` で出た正弦波）と同じで、違うのは「離散の位置 `pos = 0, 1, 2, ...`」ではなく「**連続の時刻 $\tau \in [0, 1]$** 」を符号化する点だけです。

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim=64):
        super().__init__()
        assert dim % 2 == 0
        self.dim = dim

    def forward(self, tau):          # tau: [B]
        half = self.dim // 2
        freqs = torch.exp(-math.log(1000.0) * torch.arange(half, device=tau.device) / (half - 1)) # [half]
        ang = tau[:, None] * freqs[None, :] * (2 * math.pi) # [B, half]
        return torch.cat([torch.sin(ang), torch.cos(ang)], dim=-1) # [B, dim]
```

役割: いろいろな周波数の `sin/cos` で τ を表すと、**近い時刻は近いベクトル、別の時刻は区別できる** 表現になります。これで速度ネットは「 τ の前半（ノイズ寄り）か後半（目標寄り）か」を読み取れます。`shape` は入力 `tau` が `[B]` → 出力 `[B, time_dim]`（既定 `time_dim=64`）。`dim` が偶数なのは `sin` 半分・`cos` 半分を `cat` するためです。

6.4.2 velocity：速度ネットの入出力

速度ネットは「**いまの行動 a_τ ・時刻 τ ・条件 h** 」を1本のベクトルに連結して MLP に通す」だけです：

```
def velocity(self, a, tau, h):
    # a:[B,C,A], tau:[B], h:[B,hidden] → [B,C,A]
    B = a.shape[0]
    x = torch.cat([a.flatten(1), h, self.time_embed(tau)], dim=-1)
    return self.vnet(x).view(B, self.chunk_len, self.action_dim)
```

実装を読む src/vla_learn/models/flow_head.py → FlowVLA.velocity

速度ネットの入出力です。3つ（行動を平らにした `a.flatten(1)` ・条件 `h` ・時刻埋め込み）を連結して `vnet` に通し、出力を `[B,C,A]` に戻すだけ。in_dim の内訳（次表）を手で言えるかが、実装を読めているかの試金石です。

連結する3つのshape（`C=8, A=3, hidden=256, time_dim=64` の既定値で）：

部品	中身	shape	要素数
<code>a.flatten(1)</code>	いまの行動チャンクを平らに	<code>[B, C*A]</code>	$8 \times 3 = 24$
<code>h</code>	観測の条件ベクトル（backbone 出力）	<code>[B, hidden]</code>	256
<code>self.time_embed(tau)</code>	時刻埋め込み	<code>[B, time_dim]</code>	64
x（連結後）	速度ネットへの入力	<code>[B, in_dim]</code>	$24+256+64 = 344$

`self.vnet` の構造（`__init__` 内）：

```
in_dim = chunk_len * action_dim + hidden + time_dim    # 24 + 256 + 64 = 344
self.vnet = nn.Sequential(
    nn.Linear(in_dim, velocity_hidden), nn.ReLU(inplace=True),    # 344 -> 256
    nn.Linear(velocity_hidden, velocity_hidden), nn.ReLU(inplace=True), # 256 -> 256
    nn.Linear(velocity_hidden, chunk_len * action_dim),          # 256 -> 24
)
```

出力 `[B, 24]` を `.view(B, 8, 3)` で **行動チャンクと同じ形** `[B, C, A]` に戻します。つまり速度ネットは「行動空間の点 a_τ 」を入れて「同じ行動空間の速度 v 」を返す写像です。

補足

ここを手で言い当てる問題を演習 Q1~Q3 に置きます。in_dim=344（実物 FlowVLA.vnet[0].in_features も344）、time_emb=[B,64]、出力 [B,8,3] がスラスラ出れば実装は読めています。

6.5 推論： $\tau=0 \rightarrow 1$ を Euler 積分してサンプルする

学習は「速度を当てる」だけでした。生成（推論）は、その速度場に沿って **ノイズから目標まで歩く** ことです。直線パスの速度場を、 $a_0 \sim N(0, I)$ から $\tau = 0 \rightarrow 1$ へ **小さな前進（オイラー法）** で積分します：

```
@torch.no_grad()
def sample(self, image, state, tokens, n_steps=10):
    self.eval()
    h = self.encode(image, state, tokens)    # [B, hidden]
    B = h.shape[0]
    a = torch.randn(B, self.chunk_len, self.action_dim, device=h.device) #  $a_0 \sim N(0, I)$ 
    dt = 1.0 / n_steps
    for i in range(n_steps):
        tau = torch.full((B,), i * dt, device=h.device) # 現在時刻  $\tau = i/n\_steps$ 
```

```
a = a + self.velocity(a, tau, h) * dt
return a
```

```
# オイラー前進:  $a \leftarrow a + v \cdot dt$ 
# [B, C, A] (正規化空間)
```

実装を読む src/vla_learn/models/flow_head.py → FlowVLA.sample

推論の本体です。a を $\tau = 0$ のノイズで初期化し、 $dt = 1/n_{\text{steps}}$ 刻みで n_{steps} 回 $a \leftarrow a + v \cdot dt$ と前進。ループ終了時の a が $\tau = 1$ 近傍 = 目標分布からのサンプル (正規化空間) です。前掲図の右パネルがこの歩みです。

読み方:

- a を $\tau = 0$ のノイズで初期化し、 $dt = 1/n_{\text{steps}}$ 刻みで n_{steps} 回進めます。
- 各ステップで「今の a ・今の τ ・条件 h」から速度を引き、 $a \leftarrow a + v \cdot dt$ で前へ。
- ループ終了時、a は $\tau = 1$ 近傍 = **目標分布からのサンプル** (正規化空間の行動チャンク)。
- @torch.no_grad() と self.eval() で、推論中は勾配を作らず評価モードにします。

戻り値は **正規化空間** なので、環境に渡す前に action_norm.denormalize(...) で生の [dx,dy,grip] に戻します。これは PolicyWrapper (rollout.py) がやってくれます。M4 との違いはこの 1 箇所だけです:

```
# rollout.py (抜粋) : flow と mse でここだけ分岐
if self.model_type == "flow":
    a = self.model.sample(img, state, tokens, n_steps=self.flow_steps) # [1,C,3]
else:
    a = self.model(img, state, tokens) # [1,C,3]
a = self.action_norm.denormalize(a)[0].cpu().numpy() # 生の行動へ
```

6.5.1 flow_steps は「生成の刻みの細かさ」

n_{steps} (= flow_steps) は **積分ステップ数** です。多いほど積分誤差が減りますが、その分 **推論が重く** なります (速度ネットを n_{steps} 回呼ぶ)。注意: **教師にした path は直線** ですが、**学習した速度場 v は条件付きの近似** なので、必ずしも直線の場合ではありません (だから刻み数が効きます)。それでも本タスクは目標が単純なため、少ないステップでもそれなりに当たります。flow_steps=1/5/10/50 を変えて成功率・生成の質を比べる実験を演習 Q7 で行います。直感:

- flow_steps=1: いきなり $a_0 + v \cdot 1$ で 1 歩。粗い。
- flow_steps=10 (既定): 実用的な刻み。m5_flow.json の値。
- flow_steps=50: なめらかだが遅い。改善が頭打ちになりやすい。

補足

理想的な速度場なら 1 歩でも直線で着きますが、学習した **近似** 速度場では刻み数が効きます (前掲図の右パネルで flow_steps を変えた軌跡の粗密がこれにあたります)。

6.6 学習と評価 (M4 との差分は本当に 2 箇所だけ)

6.6.1 trainer は MSE と共有、分岐は _compute_loss と「モデル構築」のみ
trainer.py は **MSE / flow** で同じファイルです。違うのは 2 箇所だけ。

(1) モデル構築 (run_training 内):

```
model = (FlowVLA if cfg.model_type == "flow" else TinyVLA)(**model_kwargs).to(device)
```

(2) 損失計算 (_compute_loss):

```
def _compute_loss(model, model_type, batch):
    if model_type == "flow":
```

```

return model.flow_loss(
    batch["image"], batch["state"], batch["tokens"], batch["action"], batch["pad_mask"]
)
pred = model(batch["image"], batch["state"], batch["tokens"]) # [B,C,A]
return masked_mse(pred, batch["action"], batch["pad_mask"])

```

実装を読む src/vla_learn/training/trainer.py → `_compute_loss`

M4 と M5 を分ける唯一の「損失側」がここです。model_type=="flow" なら model.flow_loss(...)、それ以外は masked_mse(...)。学習ループ本体（zero_grad → backward → step、Dataset/DataLoader、正規化、保存、閉ループ評価）は **一字も変わりません**。

「観測理解は共有し、行動ヘッドだけ差し替える」という設計が、コードの差分にもそのまま現れています。

6.6.2 学習を回す

```

# 設定ファイルで (推奨)
uv run python scripts/train_flow.py --config configs/m5_flow.json

# 一部だけ上書きしたいとき (flow_steps や epochs を変える)
uv run python scripts/train_flow.py --config configs/m5_flow.json --flow-steps 5 --epochs 40

```

configs/m5_flow.json の主な中身:

```

{
  "model_type": "flow",
  "n_episodes": 1500, "n_objects": 3, "n_goals": 2,
  "chunk_len": 8, "epochs": 40, "batch_size": 128, "lr": 0.001,
  "flow_steps": 10, "eval_episodes": 100, "exec_horizon": 4, "seed": 0,
  "out_dir": "checkpoints/flow"
}

```

学習ログのイメージ (数値はぶれます。CPU で数分。重い学習を写経する必要はありません) :

```

[setup] device=cpu model_type=flow
[model] flow | パラメータ数 = 576,280
[train] epoch 0 loss=...
[train] epoch 39 loss=...
[eval] success_rate=... final_dist=... steps=...

```

つまづき / バグの定番

flow の loss は MSE と数字の意味が違います。 flow_loss は毎ステップ τ と a_0 を乱数で引くので、同じバッチでも値が上下にゆれます。「単調にきれいに下がる曲線」を期待せず、**数十~数百ステップの移動平均**で「下がっているか」を見てください。MSE の loss (教師との二乗誤差) と flow の loss (速度の二乗誤差) を **直接比較しても意味はありません**。

6.6.3 評価 (eval_policy.py、flow_steps)

```

uv run python scripts/eval_policy.py --ckpt checkpoints/flow/policy.pt # 既定
flow_steps=10
uv run python scripts/eval_policy.py --ckpt checkpoints/flow/policy.pt --flow-steps 5 # 積分ステップを変えて比較

```

PolicyWrapper は `model_type="flow"` を見て `sample(..., n_steps=flow_steps)` を呼びます。--flow-steps の既定は 10。1 や 5 に変えると、Euler 積分の刻み数が変わります（理想的な速度場なら 1 歩でも直線で着きますが、学習した 近似 速度場では刻み数が効きます）。評価指標（`success_rate / mean_final_distance / mean_steps`）は M4 と同じ意味です（閉ループ:チャンク予測 → 先頭 `exec_horizon` ステップ実行 → 再観測、を繰り返す）。

補足

到達点の目安: M4 の MSE 版 TinyVLA が同条件で **閉ループ成功率およそ 7~8 割** でした。**FlowVLA は同条件で MSE 版と 同等以上** で、ある検証では成功率ほぼ 10 割（`flow_steps=5/10` で 1.00、`final_dist`≈0.08）が出ました。ただし、この差を「多峰性を扱えるから」と単純に結論づけてはいけません。本タスクはほぼ単峰なので、ここで観察できるのは「同じ FiLM 付き backbone のまま行動ヘッドを生成版へ差し替えても **劣化せず、むしろ閉ループがやや安定する**」という経験的事実です。値は 環境・乱数・PC・seed・flow_steps で大きくぶれるので、断定せず複数 seed で傾向を見て、手元の値を控えておきましょう。

6.7 本物の VLA とのつながり（次章 M6 へ）

FlowVLA は おもちゃではなく、 π_0 / SmolVLA の骨格の最小版 です。対応はこうです:

自作 FlowVLA (M5)	π_0 / SmolVLA (M6 で精読)
VLABackbone (小さな画像/言語/状態エンコーダ + FiLM)	大規模 VLM (事前学習済み) + state トークン
velocity ネット (小さな MLP)	action expert (条件付き flow/diffusion ヘッド、より大きい)
flow_loss (rectified flow, L2)	同じ flow matching 損失 (パス・損失の発想は同一)
sample (Euler 積分)	同じく ODE/サンブラで行動を生成
行動チャンク (<code>chunk_len=8</code>)	action chunking (まとめて予測、receding horizon 実行)

つまり「flow matching + action chunking + (VLM で条件付け)」という **部品の組み合わせは同じ** で、本物は各部品が大規模・事前学習済みになるだけ、という地図が描けます。M6 では SmolVLA を精読して「この `velocity` が action expert に、この `h` が VLM 出力に対応する」と読み解きます。

6.8 章末: 1 バッチ過学習 で健全性を確かめる

flow でも、まず **小さな 1 バッチを丸暗記できるか** を確認します (毎章必須のデバッグ)。ただし注意点が 1 つ:

つまずき / バグの定番

flow の `flow_loss` は τ と a_0 を毎回ランダムに引くので、**1 バッチでも loss が上下に揺れます**。「最終値が最初より 十分小さいか」「移動平均が下がっているか」で判断し、**1 ステップごとの増減に一喜一憂しない**。

確認の骨子 (詳しいコードは演習 Q8 / 解答に) :

```
# fixed = 1 バッチを固定して取り、それだけを繰り返し学習する
losses = []
for step in range(800):
```

```

loss = model.flow_loss(fixed["image"], fixed["state"], fixed["tokens"],
                       fixed["action"], fixed["pad_mask"])
opt.zero_grad(); loss.backward(); opt.step()
losses.append(loss.item())

import numpy as np
early = np.mean(losses[:50])    # 最初 50 ステップの平均
late  = np.mean(losses[-50:])   # 最後 50 ステップの平均
print(f"{early:.4f} -> {late:.4f}") # late が十分小さくなるはず（揺れるので平均で見る）

```

run_training には overfit_one_batch=True のスイッチもあり、1 バッチを繰り返す学習を内蔵しています。下がる = encode → velocity → flow_loss → backward → step の配線は正しい。下がらない = 学習ループ側のバグ（zero_grad 忘れ、[:, None, None] 忘れ、.train() 忘れ、正規化忘れ、mask の取り違い）を疑います。揺れていても移動平均が下がっていれば正常です。

まとめ

- MSE は条件付き **平均** を点推定するため、**多峰の局面**（左右どちらでも回れる等）で平均（突っ込む）に潰れる。
- rectified flow は $a_\tau = (1 - \tau)a_0 + \tau a_1$ 、真の速度 $v^* = a_1 - a_0$ を **L2 で当てる** だけ。座学の確率フロー ODE の最小実装。
- SinusoidalTimeEmbedding で連続時刻 τ をベクトル化し、velocity は $[a_flat, h, time_emb]$ （ $[B, 344]$ ）→ $[B, 8, 3]$ 。
- 推論は $a_0 \sim N(0, I)$ から $\tau = 0 \rightarrow 1$ を **Euler 積分**（sample, flow_steps）。flow_steps は精度↔速度のトレードオフ。
- **M4 との差分は「行動ヘッドと損失」だけ**。エンコーダ（FiLM 付き VLABackbone）は共有し、trainer の分岐も 2 箇所のみ。
- これは π_0 / SmolVLA の骨格の最小版。M6 で「同じ部品の大規模版」として精読する。

LeRobot と有名 VLA

この章のゴール

- 自作データを `LeRobotDataset` 形式（フレーム列・`observation.images.* / observation.state / action / task`）へ並べ替える流れと、その「変換ロジック」を `lerobot` 無しでも動く純粋関数として持つ設計眼を身につける。
- `SmolVLA` を精読し、あなたが M4/M5 で自作した `TinyVLA / FlowVLA` の部品と 1 対 1 で対応づけられるようになる。
- $\pi 0$ / $\pi 0.5$ ・`OpenVLA`・`GR00T`・`MolmoAct2` を「入力 → 表現/トークン化 → 行動ヘッド（離散 or flow） → 学習データ」の軸で読み、行動の出し方は「離散自己回帰」か「連続生成(flow/diffusion)」の二系統に大別できると掴む。
- 自作の最小 VLA が「どの部品を大規模・高機能に差し替えると、どの実在 VLA に化けるか」を地図として持つ。

この章は本書の到達点です。ここまでで、あなたは 画像 + 言語 + 状態 → 行動チャンク を出す VLA を、MSE 版（M4 の `TinyVLA`）と flow matching 版（M5 の `FlowVLA`）の 2 通りで自作しました。本章はその経験を 2 方向に「外へ」つなげます。1 つはデータの規格（`LeRobot`）に自作データを載せること、もう 1 つはモデルの地図——`SmolVLA` を精読し、 $\pi 0$ / `OpenVLA` / `GR00T` / `MolmoAct2` を「あなたが書いた部品の、どこかを強化した版」として読むことです。

補足

この章には 2 つの正確さの約束があります。読み進める前に頭に入れてください。(1) `LeRobot` の書き込み API はバージョンで変わります。後述のコードは「ある時点の一例」で、あなたの環境では動かないことがあります。だから本書は変換ロジック（`lerobot` 不要・単体テスト可能）を中心に据えます。(2) 有名 VLA の内部実装の細部は、原典（公式 repo / arXiv）でしか確定できません。本章は各事実に出典（repo のファイルパス / arXiv 番号）と参照日（2026-06）を添え、憶測の数値は書きません。公式 VLA は「読む教材（本書での実行は保証しません）」として扱います。

7.1 LeRobotDataset とは何か（フレーム列という考え方）

`LeRobot` は Hugging Face のロボット学習ライブラリで、`LeRobotDataset` はその標準データ形式です。本書で重要なのは中身の考え方だけなので、そこに絞ります。`LeRobotDataset` は、1 つのエピソードを「フレーム列」として表現します。各フレーム（＝各時刻のレコード）は、名前付きの特徴量 (features) を持つ辞書です。VLA でよく使う特徴量を、本書の自作データの何に当たるかと並べると次のようになります。

LeRobot の特徴量名	中身	本書での対応
<code>observation.images.<camera></code>	カメラ画像（複数台ぶん。<camera> はカメラ名）	1 台なので <code>observation.image</code>
<code>observation.state</code>	固有受容状態（関節角・エンドエフェクタ位置など）	<code>state</code> （ <code>[ax, ay, grip]</code> ）
<code>action</code>	その時刻の行動	<code>action</code> （ <code>[dx, dy, grip]</code> ）
<code>task</code>	言語指示（文字列で持つ）	<code>instruction</code>

押さえるべき点は 3 つです。(1) 実機ロボットは複数視点を使うのが普通なので、画像のキー名にカメラ名を含めます (例 `observation.images.top`, `observation.images.wrist`)。本書は 1 視点なので `observation.image` 1 つで十分です。(2) 言語指示は `task` (文字列) として各フレームに付く。トークン化は学習時にモデル側でやるので、データ側は文字列で持ちます。(3) エピソード境界・FPS・特徴量の `dtype/shape`・正規化統計はメタデータ (`meta/`) が管理します。

座学とのつながり

M3 で `SyntheticVLADataset` を作ったとき張った伏線がこれです。本書の内部 `dict` (`image / state / tokens / action`) は、名前が違っただけで `LeRobotDataset` の特徴量に対応します。「実装の核心は自作、データ規格は LeRobot に合わせる」という本書の境界線 (最初から `lerobot-train` に依存しない方針) の実演です。

7.1.1 ディスク上の実体とバージョン (v3.0 を例に)

`LeRobotDataset` は中身を 3 種類に分けて保存します。これは「画像は重い／低次元データは軽い／メタ情報は別」という都合に素直に従った形です (以下は `lerobot v3.0` 系の構成。バージョンで変わります。出典: [huggingface/lerobot](https://huggingface.co/lerobot) 公式ドキュメント `LeRobotDataset v3.0`、参照 2026-06)。

置き場所	形式	入るもの
<code>data/.../*.parquet</code>	Apache Parquet	低次元の時系列 (<code>observation.state / action / タイムスタンプ</code> 等)
<code>videos/.../*.mp4</code>	MP4 動画	カメラ画像 (フレームを動画にまとめて格納)
<code>meta/</code>	JSON / JSONL / Parquet	<code>info.json</code> (特徴量スキーマ・FPS), <code>stats.json</code> (正規化統計), <code>tasks.jsonl</code> (指示文↔ID), エピソード情報

補足

バージョン依存の正直な注意: `LeRobotDataset` のコードベース版は `v2.1` → `v3.0` と変わってきました (`v3.0` は概ね 1 ファイルに複数エピソードをまとめる方式で、`v2.1` の「1 エピソード 1 ファイル」とは構造が違う)。書き込み API (後述の `create / add_frame / save_episode` や、`v3.0` で必要になった `finalize()`) も版で変わります。正確な使い方は、あなたが入れた `lerobot` の版のドキュメントを必ず参照してください。出典: [huggingface/lerobot src/lerobot/datasets/](https://huggingface.co/lerobot/src/lerobot/datasets/) (`CODEBASE_VERSION`)、公式ドキュメント、参照 2026-06。

7.1.2 画像の格納形式 (HWC・uint8) — ここが M4 と違う

VLA を自作したとき、画像は `[3, 64, 64]` (CHW) の `float (0..1)` でした (M3 の `render_world` の出力)。これは `PyTorch` の `Conv` が CHW を食べるからです。一方、`LeRobotDataset` の画像特徴量はスキーマ上 `[H, W, 3]` (HWC) で記述され、保存・取り込みは `uint8 (0..255)` が基本です。これは画像/動画として保存・可視化する都合に合わせた、保存・交換のための形式です。



補足

もう一段の正直さ: `LeRobot` の `DataLoader` は、学習時には画像を再び CHW・float に直して返す実装になっています (内部で `uint8(HWC) → float32(CHW)` へ変換し `0..1` へ正規化)。つま

り「保存は HWC・uint8、学習で使う形は CHW・float」という往復があります。本書の export では CHW float → HWC uint8 の向き（保存する向き）を作ります。出典: huggingface/lerobot src/lerobot/datasets/、参照 2026-06。

7.2 変換ロジックを読む（map_episode_to_frames ・ lerobot 不要）

export の心臓部は scripts/export_lerobot.py の map_episode_to_frames です。これは「自作 episode → LeRobot 風フレーム辞書の列」へ変換する純粋な関数で、lerobot をインストールしていなくても動き、単体テストできます。

実装を読む scripts/export_lerobot.py → map_episode_to_frames

この関数 1 つが「自作データ → 規格の特徴量」への対応づけの本体です。render_world で [3,H,W] float を作り、np.transpose(..., (1,2,0)) * 255 で [H,W,3] uint8 に並べ替え・型変換し、task には instruction（文字列）をそのまま入れます。書き込み API ではなくこの変換こそが学習目標である、と意識して読んでください。

```
# scripts/export_lerobot.py より（抜粋）
def map_episode_to_frames(ep: dict, img_size: int = 64) -> List[dict]:
    frames = []
    T = ep["actions"].shape[0]
    for t in range(T):
        img_chw = render_world(...) # [3,H,W] float 0..1
        img_hwc = (np.transpose(img_chw, (1, 2, 0)) * 255).astype(np.uint8) # [H,W,3] uint8
        frames.append({
            "observation.image": img_hwc, # 保存は HWC・uint8
            "observation.state": ep["agent"][t].astype(np.float32), # [3]
            "action": ep["actions"][t].astype(np.float32), # [3]
            "task": ep["instruction"], # str（トークン化しない）
        })
    return frames
```

読みどころは3つです。(1) 画像は M3 と同じく「都度レンダリング」——ディスクには低次元の状態だけ保存し、export 時に render_world で作ります。(2) img_chw → img_hwc が、前節の CHW float → HWC uint8 そのものです。(3) task は文字列をそのまま入れます（トークン化はモデル側の仕事）。

つまづき / バグの定番

ここで「画像が [64,64,3] の uint8、値域 0~255」に変わることを自分の目で確認しておきましょう。M4 のモデル入力 [3,64,64] float (0..1) と取り違えると、permute や /255 の入れ忘れで「色が壊れる／学習が進まない」が起きます。map_episode_to_frames は lerobot 不要なので、generate_episodes(n_episodes=1)[0] を食わせれば手で即 shape 確認できます。

7.2.1 なぜ「変換ロジックを独立させる」設計が良いか

ライブラリの書き込み API は変わりやすいが、「自分のデータ → 規格が要求する特徴量」への対応づけは設計判断であって、変わりにくいものです。この2つを分離しておく——API が変わっても直すのは薄い書き込み層だけで済み、変換ロジックを単体テストできる（lerobot を入れずに CI で回せる）。export_lerobot.py の main は、変換を確認した後に lerobot の import を try で囲み、無ければ案内して安全に終了し、書き込みは「v2 系を想定した一例」を try で囲って失敗しても落とさず warn を出す設計にしています。

補足

これは VLA に限らず、外部規格に合わせるときの定石です。「**変わりやすい所**」と「**変わりにくい所**」を分けるという設計眼を、ここで養ってください。features (スキーマ) の dtype 名、task の渡し方、finalize() の要否などは版で変わります——**正確な使い方は、あなたが入れた lerobot のドキュメントに従ってください。**

7.3 SmolVLA 精読 — 自作 VLA との対応表

ここからモデルです。精読の対象に SmolVLA を選ぶのは、LeRobot に直結 (同じエコシステム) し、小型の OSS で、VLM + 状態トークン + action expert + flow matching + action chunking が一通り入っており、あなたが M4/M5 で作った部品とほぼ 1 対 1 で対応づけられるからです。

補足

出典: SmolVLA 論文 [arXiv:2506.01844](https://arxiv.org/abs/2506.01844) 「SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics」、公式実装 [huggingface/lerobot](https://huggingface.co/lerobot) (src/lerobot/policies/smolvla/)、参照 2026-06。以下はこれら一次情報に基づきますが、**内部実装の細部・最新の数値は必ず原典で確認してください。**

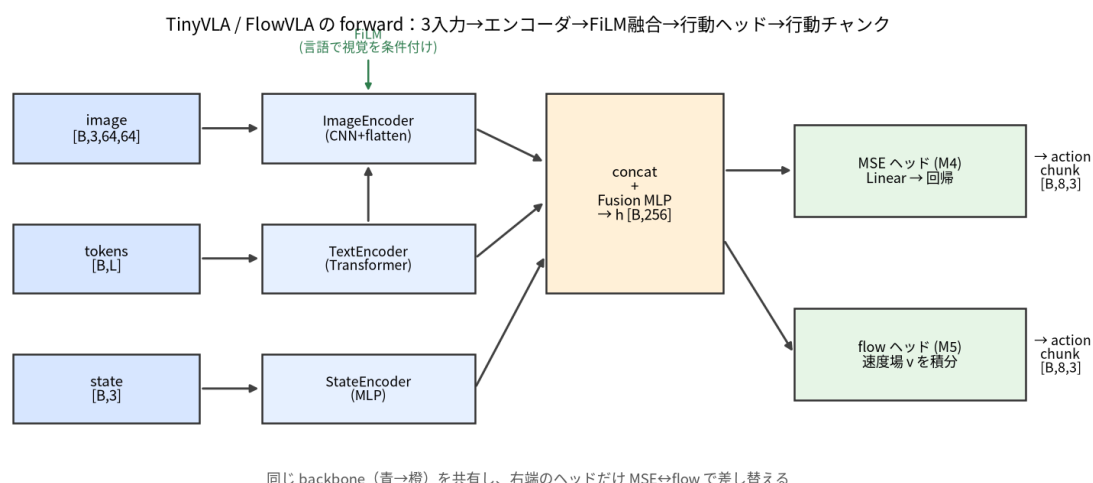


図 14: 自作 VLA の forward (3 入力 → エンコーダ → FiLM 融合 → ヘッド)。SmolVLA は前半 (融合) を事前学習 VLM に、後半 (行動生成) を専用の action expert(flow) に置き換えた格好。対応する自作実装は `src/vla_learn/models/tiny_vla.py` (VLABackbone) と `flow_head.py` (FlowVLA)。

7.3.1 SmolVLA の構成 (設計思想レベル)

設計思想レベルで言うと、SmolVLA は「事前学習済み VLM が画像・言語・状態を統合し、その文脈を条件に action expert (別の小さなネット) が flow matching で行動チャンクを生成する」流れです。確定している要点を並べます。

- **入力は「画像 + 言語(task) + 状態」**。画像は視覚エンコーダ (SigLIP 系) で視覚トークンに、言語は普通にトークン化、**状態 (proprioception) は線形層で射影してトークンとして入れます** (本書と同じ 3 入力)。複数カメラは各画像を順に処理して並べます。出典: `configuration_smolvla.py` (状態は `nn.Linear` で 1 トークンへ射影), 参照 2026-06。
- **VLM は事前学習済み**。既定のベースは **SmolVLM2** (HuggingFaceTB/SmolVLM2-500M-Video-Instruct、SigLIP 視覚 + SmolLM2 言語) で、SmolVLA は**その LLM の先頭 16 層だけ**を使います。出典: `configuration_smolvla.py` (`vlm_model_name`), 参照 2026-06。

- **action expert** が VLM の文脈を条件に**行動を生成**します。生成方式は **flow matching** (M5 で自作したもの)、出力は **連続値の行動チャンク**。既定の chunk は **50** (`chunk_size = 50`) です。出典: `configuration_smolvla.py` , 参照 2026-06。

補足

規模の数値 (不確実さの明記) : 論文 arXiv:2506.01844 は「約 **4.5 億 (450M)** パラメータ、うち約 **1 億 (100M)** が **action expert**」と本文に明記しています。一方ベース checkpoint 名は「500M」を含みますが、SmolVLA は**その 16 層 だけ**を使うため、**論文の 450M と checkpoint 名の 500M を混同しない**でください。正確な値は arXiv:2506.01844 と最新のモデルカードを参照。

7.3.2 実装を読む — flow matching の学習と推論

SmolVLA の中核は 2 クラスです。**精読の中心**として、ここを必ず開いてください。

実装を読む

`src/lerobot/policies/smolvla/modeling_smolvla.py` →

`SmolVLAPolicy.select_action / forward`

`SmolVLAPolicy` は LeRobot の policy ラップ。 `forward` (学習) は `VLAFlowMatching` の flow matching 損失を計算し、 `select_action` (推論) は**行動キュー**を持ち、空になったときだけ推論して**行動チャンクを生成**、1 手ずつ環境に返します。「学習＝損失、推論＝積分でチャンク生成、しかも毎ステップは推論しない」という**実運用の作法**が読めます。

実装を読む

`src/lerobot/policies/smolvla/modeling_smolvla.py` →

`VLAFlowMatching.forward / sample_actions`

flow matching の本体。学習は時刻 $\tau \sim \text{Beta}(1.5, 1)$ を引き、 $x_t = \tau \cdot \text{noise} + (1 - \tau) \cdot \text{action}$ を作り、目標速度 $u_t = \text{noise} - \text{action}$ を**速度予測の MSE** で学習。推論は $t=1 \rightarrow 0$ を $dt = -1/\text{num_steps}$ (既定 **10 ステップ**) で Euler 積分。**あなたが M5 で書いた `flow_loss / sample` と、設計が一致しているのが分かります。**

座学とのつながり

M5 の `FlowVLA.flow_loss` は $a_{\tau} = (1 - \tau) \cdot a_0 + \tau \cdot a_1$ 、 $v_{\text{target}} = a_1 - a_0$ でした (時刻は一様 $U(0, 1)$ 、推論は $\tau = 0 \rightarrow 1$)。SmolVLA は「**ノイズと目標の置き方の向き**」と「**時刻分布 (Beta)**」が違っただけで、**速度を予測して MSE で合わせ、積分で生成する**という骨格は完全に同じです。M5 を書けた時点で、この 2 ファイルは読めます。

7.3.3 【中核】 TinyVLA / FlowVLA との対応表

本章のいちばん大事な表です。**あなたが自作した部品が、SmolVLA では何に当たるか**を対応づけます。

役割	本書の自作部品 (M4/M5)	SmolVLA での対応 (概念)
視覚の符号化	<code>ImageEncoder</code> (小さな CNN、flatten で空間保持)	事前学習済み視覚エンコーダ (SigLIP 系) + 視覚トークン
言語の符号化	<code>TextEncoder</code> (文字埋め込み + 位置 + 1 層 Transformer)	事前学習済み言語モデル (VLM の言語側)
状態の符号化	<code>StateEncoder</code> (小さな MLP)	状態を線形射影した「状態トークン」
視覚 × 言語の融合	FiLM (言語で視覚を変調) + concat → 融合 MLP (<code>VLABackbone</code>)	VLM の注意機構で全トークンを統合
条件ベクトル	<code>VLABackbone</code> の出力 <code>h [B, hidden]</code>	VLM が出す文脈表現

行動ヘッド（生成）	FlowVLA.velocity + flow loss (M5)	action expert による flow matching
行動の形	行 動 チ ャ ン ク [B, chunk_len, action_dim] (chunk=8)	行動チャンク（連続値・複数ステップ。 chunk=50）
学習則	rectified flow の MSE（速度予測）	flow matching（速度予測の MSE）

この表の読み方は2つ。(1)「画像/言語エンコーダ → FiLM 融合 → flow ヘッド」(自作) が、SmolVLA では「事前学習 VLM → **action expert(flow)**」に対応します。つまり**骨格は同じ**で、SmolVLA は前半を**巨大な事前学習 VLM**に、後半を**専用の action expert**に置き換えた格好です。(2) あなたの VLABackbone が作る条件ベクトル h は、SmolVLA では VLM の文脈表現に当たります。

7.3.4 本書 TinyVLA との「距離」

正直に距離も書きます。主な違いは**事前学習の有無**（SmolVLA の VLM は大量の画像・言語で事前学習済み。本書はゼロから）、**規模**（SmolVLA は約 450M、本書は約 0.42M の TinyVLA / 約 0.58M の FlowVLA）、**入力の豊かさ**（実機は複数カメラ・高次元状態。本書は 1 視点・3 次元）、**データ**（実機データ vs 合成データ）です。逆に言えば、**設計の骨格（3 入力 → 融合 → flow で行動チャンク）は共通**。だから「小さく作って大きく読む」が成立します。

まとめ

M4 + M5 を作り切った時点で、SmolVLA の「読み方」は手に入っています。残りの差は主に「VLM を事前学習で賢くしてあるか」と「実機データで大規模に学習してあるか」です。

7.4 π_0 / $\pi_{0.5}$ — flow matching と action expert（実機データ規模）

補足

出典： π_0 = [arXiv:2410.24164](#)「 π_0 : A Vision-Language-Action Flow Model for General Robot Control」、 $\pi_{0.5}$ = [arXiv:2504.16054](#)、公式 repo [Physical-Intelligence/openpi](#) (src/openpi/models/)、参照 2026-06。

π_0 は **flow matching** で連続行動を生成する VLA です（M5 と同じ系統）。**action expert** という別の重みを持つネットを VLM に付け、状態と行動をトークンとして扱います。ベース VLM は **PaliGemma** (config では gemma_2b 相当 + SigLIP 視覚) で、**action expert** は別の小さな **Gemma** (既定 gemma_300m 相当)。論文も「ロボット用トークンに別の重みを使うと改善した」と述べています。行動はチャンク化 (config 既定の action_horizon = 50、タスク別 config で上書き)。 π_0 は、本書 FlowVLA の「事前学習 VLM + 専用 action expert + 実機大規模データ」版と読めます。

実装を読む src/openpi/models/pi0.py → embed_prefix / embed_suffix / compute_loss / sample_actions

自作との対応がいちばん綺麗に見えるファイル。embed_prefix が画像+言語（VLM 側）を、embed_suffix が状態+ノイズ行動+時刻（action expert 側）を埋め込みます。compute_loss は時刻 $t \sim \text{Beta}(1.5, 1)$ を引いて $x_t = t \cdot \text{noise} + (1-t) \cdot \text{action}$ 、目標 $u_t = \text{noise} - \text{action}$ で速度の MSE。sample_actions は $t=1 \rightarrow 0$ を既定 10 ステップで Euler 積分。M5 の flow_loss / sample と同じ部品配置です。

実装を読む src/openpi/models/pi0_config.py → Pi0Config (pi05 フラグ)

π_0 と $\pi_{0.5}$ の差を確認する場所。paligemma_variant / action_expert_variant でベース VLM と action expert のサイズが、action_horizon / action_dim で行動チャンクの形が決まります。

pi05: bool フラグが立つと挙動が変わる（後述）——「config の 1 フラグで派生を切り替える」設計が読めます。

7.4.1 $\pi 0.5$ の差分（確定事項と解釈の分離）

$\pi 0.5$ は $\pi 0$ を基盤に、**オープンワールド汎化**（学習に無い新しい家庭での動作）を狙った発展版です。確定している差分を、openpi の config と $\pi 0.5$ 論文から**事実**として挙げます。

- **config 上の差分**: `Pi0Config` の `pi05=True` で、(a) 状態入力を離散の言語トークンとして扱う、(b) action expert の時刻注入を `adaRMSNorm` で行う（ $\pi 0$ の sinusoidal 埋め込みに対して）、という違いが入ります。出典: openpi `src/openpi/models/pi0_config.py`（post-init の記述）、参照 2026-06。
- **学習目標のハイブリッド**: $\pi 0.5$ 論文は「**FAST トークナイザによる自己回帰サンプリング**（離散）と**フロー場の反復積分**（連続）の両方で行動を予測するよう学習」と述べています。**高レベル=離散トークン / 低レベル=連続生成**のハイブリッドです。出典: arXiv:2504.16054, 参照 2026-06。

補足

「 **$\pi 0.5$ を専用ファイルで読みたい**」とき（よくある疑問）: openpi には `pi05.py` という**単独ファイルは無く**、 $\pi 0.5$ は `pi0.py` を `pi05=True` で動かす実装です（差分がフラグとして `pi0.py` / `pi0_config.py` に散らばる）。一方 **Hugging Face LeRobot** には **$\pi 0.5$ 専用の policy**があり、 $\pi 0.5$ を**独立したクラス**として読めます。openpi = 研究の本家(JAX)、LeRobot = PyTorch で他 policy と統一 API、という住み分けです。 **$\pi 0.5$ だけを集中して読むなら、M5 と同じ PyTorch で書かれた LeRobot 版が分かりやすい**でしょう。

実装を読む `src/lerobot/policies/pi05/modeling_pi05.py` → `PI05Policy` / `PI05Pytorch.sample_actions`

LeRobot 版 $\pi 0.5$ の専用実装（PyTorch・openpi からの移植）。`PI05Pytorch.forward` が flow matching の損失、`sample_actions` が $t=1 \rightarrow 0$ の Euler 積分で行動チャンクを生成——**M5 で自作した `FlowVLA.flow_loss` / `FlowVLA.sample` とそのまま対応**します（ファイル内コメントにも“see openpi”と明記）。設定は同ディレクトリの `configuration_pi05.py`、LeRobot データとの入出力接続は `processor_pi05.py`。openpi の `pi05=True` 経路と同じ思想を、フラグ分岐ではなく**専用クラス**としてまとめた形で読めます。参照 2026-06。

つまずき / バグの定番

混同しやすい点（重要）: 「**知識絶縁 (knowledge insulation)**」という定式化は、 $\pi 0.5$ 論文 (arXiv:2504.16054) 本文ではなく、別の後続論文 arXiv:2505.23705 で**確立**されたものです。 $\pi 0.5$ 論文自体はこの語を使っていません。 $\pi 0.5$ に **knowledge insulation** を断定的に帰属させないでください。また $\pi 0.5$ のベース VLM 名・パラメータ数・学習データの**定量規模**を一次資料で**検証するのは難しい**——**断定せず原典参照**としてください。本書で確実に言えるのは「 $\pi 0$ 系は flow matching + 専用 action expert で連続行動を出し、 $\pi 0.5$ は離散+連続のハイブリッドを使う」までです。

7.5 OpenVLA — 離散トークン自己回帰 + 大規模 VLM

補足

出典: **arXiv:2406.09246**「OpenVLA: An Open-Source Vision-Language-Action Model」、公式 repo `openvla/openvla`（`prismatic/`、`vla-scripts/`）、参照 2026-06。

OpenVLA は行動を「離散トークン」に変換し、VLM に自己回帰で生成させる方式の代表です。各行動次元を 256 個のビンに離散化し、言語モデルの語彙のうち使用頻度の低いトークンを行動トークンに置き換えて扱います。入力画像 + 言語指示で、論文は固有受容状態を入力しない（単一画像のみ）と明記します。ベース VLM は Prismatic VLM (DINOv2 + SigLIP の視覚 + Llama-2 7B、計 7B)、学習データは Open X-Embodiment の約 97 万 (970k) 軌跡です。

実装を読む `prismatic/vla/action_tokenizer.py` → `ActionTokenizer`

連続行動を「単語」に量子化する核心。既定 `n_bins = 256`、`-1..1` を `np.linspace` で等分してビン中心を作り、ビン番号を `vocab_size - 番号` で語彙末尾のトークン ID に写す（「使用頻度の低いトークンは語彙の末尾」という前提）。あなたの head（連続値 Linear）を「256 クラス分類を `action_dim × chunk 回`」に置き換える発想が、ここで具体的にになります。

実装を読む `prismatic/models/vlas/openvla.py` → `OpenVLA.predict_action`

`predict_action(image, instruction, ...)` は行動トークンを自己回帰生成（`generate` を `max_new_tokens=action_dim`）し、`decode_token_ids_to_actions` で連続値に戻して逆正規化します。**state 引数がないことも読み取ってください**（本書は state を使う）。

補足

派生は分けて扱う: 推論を高速化・多画像化する OpenVLA-OFT や、行動チャンクを少ないトークンへ圧縮する FAST トークナイザは、いずれも基底 OpenVLA とは別の後続研究です（FAST は Physical Intelligence 由来）。混同しないでください。fine-tune は `vla-scripts/finetune.py` の LoRA（既定 rank 32、対象は全 linear）で、データは RLDS 形式。出典は上記 repo、参照 2026-06。

7.6 GR00T / MolmoAct2 — dual-system と action expert（LeRobot 統合）

7.6.1 GR00T — ヒューマノイド向け dual-system

補足

出典: GR00T N1 = arXiv:2503.14734 「GR00T N1: An Open Foundation Model for Generalist Humanoid Robots」、公式 repo NVIDIA/Isaac-GR00T (`gr00t/`)、参照 2026-06。

GR00T はヒューマノイド向けの基盤モデルで、特徴は dual-system 構成です。System 2（遅い）= vision-language module が視覚と言語で状況を解釈し、System 1（速い）= action module がリアルタイムで運動を生成します。System 1 の実装は Diffusion Transformer (DiT) を flow matching で学習したもの（連続行動）。両システムは密結合して学習し、状態は実体ごとの MLP で共有埋め込みに射影、行動はチャンク化（論文 $H = 16$ ）して出します。「遅い思考 + 速い反射」という人間の二重過程がそのまま設計になっているのが面白い点です。

実装を読む `gr00t/policy/gr00t_policy.py` → `Gr00tPolicy`

推論の入口。video / state / language の観測辞書を `AutoProcessor` で整形して `model.get_action(...)` を呼びます。LeRobot 形式の規約に沿ったデータを扱う一方、policy 自体は「整形済み観測辞書」を食べる点に注意（生ファイルを直接ではない）。本書 FlowVLA の sample に当たる「観測 → 行動チャンク」の実機版がここに当たります。

つまずき / バグの定番

版差に注意 (最重要の不確かさ): ベース VLM は世代で変わります。論文 N1 は NVIDIA Eagle-2 と明記しますが、2026-06 時点の repo は概ね N1.7 系で、Cosmos-Reason2-2B (repo の設定では `nvidia/Eagle-Block2A-2B-v2` という表記も見られ、公式の整理は未確定)。世代 (N1 / N1.5 / N1.6 / N1.7) で VLM 名も DiT 層数も異なります。主張は必ず「どの版か」を明示し、数値は原典で確認してください。また「end-to-end 学習」は事前学習時に VLM の言語側を凍結する記述があるため、「VLM の視覚側 + DiT を同時学習」と捉えるのが正確です。出典: arXiv:2503.14734、repo、参照 2026-06。

7.6.2 MolmoAct2 — 行動の「空間推論」と action expert (LeRobot policy)

補足

出典: 初代 **MolmoAct** = arXiv:2508.07917 「Action Reasoning Models that can Reason in Space」(allenai/molmoact, Ai2 の **Molmo** を基盤)。MolmoAct2 (後継・2026-06 時点の主演) = Ai2 が 2026-05 に公開、huggingface/lerobot に `policy_src/lerobot/policies/molmoact2/` として統合 (MolmoAct2Policy)。参照 2026-06。MolmoAct2 は新しいため、細部は repo / 公式発表で確認してください。

MolmoAct 系の独自性は「行動する前に空間で推論する」(Action Reasoning Model) 点です。初代は概ね 3 段階で、(a) 観測を **depth-aware perception token** (深度を意識した知覚トークン) にして 3D 的に接地し、(b) 中間計画を **画像上のウェイポイント列 (visual reasoning trace)** として描き、(c) 最後に **低レベル行動** (初代は 256 ビンの離散トークン、N=8 チャンク) を出します。観測 → 行動を直接写像する本書 TinyVLA / FlowVLA と違い、「観測 → 空間推論 (中間トークン) → 行動」と 1 段はさむ——chain-of-thought を行動に持ち込んだイメージです。

MolmoAct2 (後継) は LeRobot policy として読めるのが利点です。確定している範囲で要点を挙げます。

- **action expert + flow matching の連続生成も持つ:** 行動表現はハイブリッドで、`action_mode` を `discrete / continuous / both` で切り替え可能。離散側は OpenFAST トークナイザ、連続側は flow matching の denoiser。既定の推論は連続。出典: lerobot `src/lerobot/policies/molmoact2/` (MolmoAct2Policy, `hf_model/`) , 参照 2026-06。
- **VLM と action expert の橋渡しは KV-cache 経由** (VLM 各層の KV に action expert が cross-attend)。ベース VLM は Molmo2-ER (Molmo2 を身体性推論データで追加学習)。出典: 同 repo / Ai2 公式発表, 参照 2026-06。
- **depth トークン** (適応的な深度推論) は MolmoAct2-Think 変種の機能で、現状 LeRobot policy には未収録、とドキュメントに明記。出典: lerobot 公式ドキュメント, 参照 2026-06。

実装を読む `src/lerobot/policies/molmoact2/modeling_molmoact2.py` → MolmoAct2Policy

離散と連続を 1 つの policy で扱う実例。action_mode (discrete/continuous/both) と、VLM 各層の KV に cross-attend する action expert の接続 (KV-cache bridge) が読みどころ。`processor_molmoact2.py` は画像/言語/ 状態 (256 ビンに離散化して `<state_start>..<<state_end>` で囲む) の前処理を担います。

つまずき / バグの定番

世代の混同に注意: 2026-06 時点の主演は MolmoAct2、初代 MolmoAct は第 1 世代 (歴史) です。初代の状態 (proprioception) 入力是一次資料に明確な記述が見当たらず (主入力は視覚 + 指示)、

一方 MolmoAct2 の LeRobot processor は状態を離散化して取り込む——世代で前提が違うので、必ず原典/repo で「どの版の話か」を確認してください。

7.7 比較表 — 行動表現で並べる

設計思想レベルの地図です。数値や内部実装の断定は避け、版で変わる点は前節までの注意に従ってください。正確な値・最新情報は各 arXiv / 公式 repo（参照 2026-06）で確認します。

モデル	入力（状態）	画像 enc / 言語 backbone	行動表現・生成	chunk（既定）	fine-tune 単位
TinyVLA（自作 M4）	画+言+状態	小 CNN / 1 層 Transformer	連続・決定論的回帰(MSE)	8	全体（自作）
FlowVLA（自作 M5）	画+言+状態	小 CNN / 1 層 Transformer	連続・flow matching	8	全体（自作）
SmolVLA	画+言+状態	SigLIP / SmolLM2 (SmolVLM2)	連続・flow matching (action expert)	50	policy 微調整
$\pi 0 / \pi 0.5$	画+言+状態	SigLIP / Gemma (PaliGemma)	$\pi 0$:連続 flow / $\pi 0.5$:離散+連続	50（既定）	openpi 微調整
OpenVLA	画+言（状態なし）	DINOv2+SigLIP / Llama-2 7B	離散 256 ビン・自己回帰	単発（多くは 1 手）	LoRA(rank32)
GR00T	画+言+状態	Eagle 系 →Cosmos 系 (版依存)	連続・DiT + flow matching	16	微調整（版依存）
MolmoAct2	画 + 言 + 状態 (v2 で取込)	SigLIP 系 / Molmo2-ER	ハイブリッド (離散+flow)	8（初代）	LeRobot policy 微調整

補足

必要 GPU について: 本書の自作（約 0.4~0.6M）はノート PC / CPU でも回りますが、上記の実在 VLA はおおむね 数億~70 億パラメータ級で、微調整にも GPU（多くは VRAM 16GB 以上、7B 級は LoRA でも相応のメモリ）が要ります。正確な要件は各 repo の README を参照。本書はこれらを実行しません——「読む教材」として扱います。

この表で押さえるべきは「行動の出し方は離散自己回帰か連続生成(flow/diffusion)かに大別され、あなたは両系統の最小版（離散は卒業課題で少しだけ、連続は M5）を自分で書ける」という地図です。個々のセルの厳密さより、自作部品からの距離を読み取ってください。

7.7.1 【総まとめ】自作部品を「どう差し替えると、どのモデルになるか」

「あなたの自作部品の、どこを・どう差し替えると、そのモデルに化けるか」を 1 枚にまとめます。これが本章の地図の最後のピースです。

あなたの自作部品	これを…に差し替えると	→ このモデルに近づく
head（連続値 Linear）	行動を 256 ビンに離散化し、トークンとして自己回帰生成	OpenVLA / MolmoAct（初代）

flow ヘッド (FlowVLA)	action expert + 事前学習 VLM + 実機大規模データ	π_0 / SmolVLA
観測 → 行動を 直接 写像	観測 → 空間推論 トレース → 行動と1段はさむ	MolmoAct 系
単一の融合 MLP (VLABackbone)	System 2 (VLM の熟考) + System 1 (高速 flow) の二層	GR00T
ゼロから学習	大規模事前学習 VLM (SigLIP 系 + 言語モデル) を土台にする	実在 VLA すべてに共通

まとめ

腑に落ちてほしいのは——自作の最小 VLA は、どの部品を大規模・高機能に差し替えるかで、各実在 VLA に化けるということ。逆に言えば、有名 VLA はどれも「あなたが書いた部品の、どこかを強化した版」として読めます。これが「小さく作って大きく読む」の到達点です。

7.8 卒業課題 (capstone challenge)

ここまでで、自作 VLA → データ規格 (LeRobot) → 有名 VLA の地図がつながりました。最後は卒業課題で、自分の VLA をいじり倒して理解を確かなものにします。各課題に到達目標と評価方法を付けます (重い学習は不要。多くは「設定を変えて短く学習 → success_rate を見る」だけで観察できます。値は乱数でぶれるので、絶対値でなく傾向を読みます)。

1. **chunk_len / flow_steps** を変えて比較する。到達目標: 行動チャンク長と積分ステップ数が成功率・滑らかさに与える影響を、自分の言葉で説明できる。評価: evaluation の success_rate を 2~3 設定で比較し、傾向 (増やすと安定/飽和、等) を述べる。
2. **image_pool='avg' / condition_vision=False** で ablation する。到達目標: 「flatten で空間を保つ」「FiLM で言語接地する」がなぜ効くかを、壊した結果から説明できる。評価: 既定との success_rate 差を見て、部品を抜くと下がることを確認する。
3. 自作データを LeRobot 形式へ export してみる。到達目標: map_episode_to_frames の出力 shape ([H,W,3] uint8 / [3] float / 文字列 task) を説明でき、書き込み API がなぜ版依存で、変換ロジックと分離すべきかを言える。評価: lerobot 未導入でも、変換が通り「概算フレーム数」が出ることを確認 (書き込みはスキップで可)。
4. (発展) 離散トークン化ヘッドを自作して OpenVLA 流 (256 ビン) と比べる。到達目標: 連続値ヘッド (head) を「action_dim×chunk 回の 256 クラス分類」に置き換え、離散自己回帰の発想を体得する。評価: 連続版 (TinyVLA / FlowVLA) と離散版の成功率・学習挙動を比較し、長所短所を述べる。
5. (発展) FlowVLA の action chunk を拡張し、SmolVLA 風 (長い chunk) に近づける。到達目標: chunk_len を増やしたときの学習安定性・受信ホライズン (receding horizon) 実行への影響を観察できる。評価: chunk を伸ばして success_rate とロールアウトの滑らかさの変化を見る。

まとめ

- **LeRobot:** LeRobotDataset は フ レーム 列 で、observation.images.* / observation.state / action / task を持つ。画像はスキーマ上 HWC・保存は uint8 (学習は CHW・float に戻る)。変換 map_episode_to_frames は lerobot 無しで動く。書き込み API はバージョン依存 (v3.0 で finalize() 追加等) なので分離してある。

- **SmolVLA 精読:** 「VLM + 状態トークン + action expert + flow matching + action chunking」。自作の『画像/言語エンコーダ → FiLM 融合 → flow ヘッド』が『事前学習 VLM → action expert(flow)』に対応。M4+M5 を作れば読める。
- **概観:** 行動の出し方で 離散自己回帰 (OpenVLA / MolmoAct 初代) と 連続生成 (SmolVLA / $\pi 0$ / GR00T) に大別。 $\pi 0.5$ ・MolmoAct2 は離散+連続のハイブリッド、GR00T は dual-system。数値・細部・版は原典参照。
- **卒業課題**で自作 VLA をいじり、理解を確定させる。これで「VLA を読める」から「VLA をいじれる」へ進めます。